

Aprendizaje Supervisado

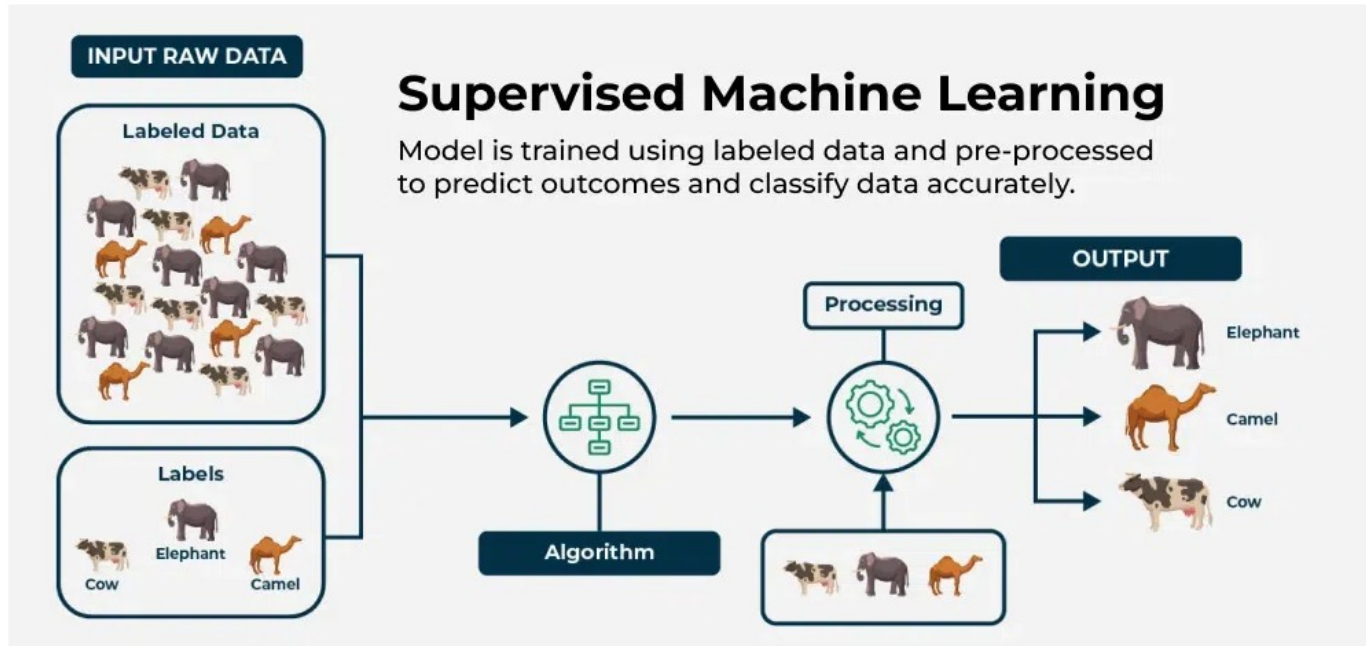
Fundamentos de Aprendizaje Automático

Universidad EAFIT

Medellín 2026

Recordemos...

- **Aprendizaje Supervisado:**
 - El enfoque mas común en Aprendizaje Automático.



Recordemos...

- **Aprendizaje Supervisado:**

- Comenzamos recolectando un conjunto de datos de objetos **etiquetados**.
- Entrenamos un modelo para que produzca predicciones precisas sobre este conjunto de datos.
- Cuando el modelo ve datos nuevos y similares, también será preciso.

Recordemos...

- Aprendizaje Supervisado:
 - **Aplicaciones:**
 - Clasificación de imágenes médicas.
 - Detección de SPAM.
 - Detección de objetos en la conducción autónoma.
 - Reconocimiento de imágenes.
 - Análisis de sentimiento.
 - Predicción de tendencias numéricas.

Regresión Lineal

- **Definición:**

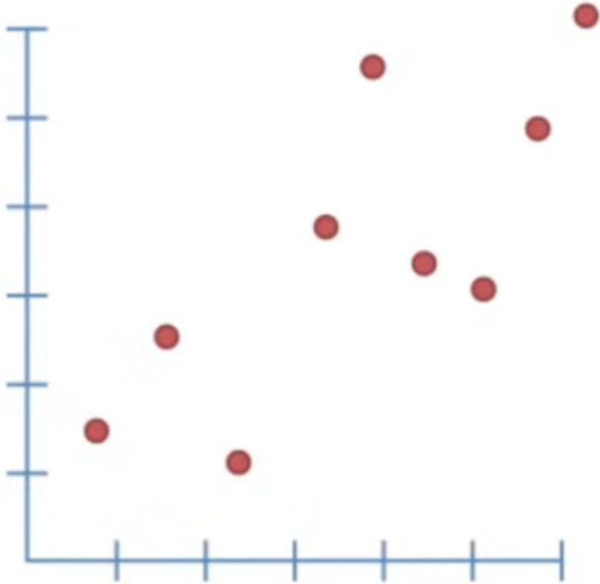
- Es un algoritmo de aprendizaje supervisado que busca encontrar la relación matemática entre una variable dependiente (y) y una o más variables independientes (x).
- Se usa para predecir valores numéricos, no categóricos.

- **Tipos:**

- Regresión Lineal Simple: Usa una sola variable x para predecir y . (Ej: Años de experiencia $\rightarrow$ Salario).
- Regresión Lineal Múltiple: Usa varias variables $x_1, x_2, x_3 \dots$ para predecir y . (Ej: Metros cuadrados + Ubicación + Antigüedad $\rightarrow$ Precio de casa).

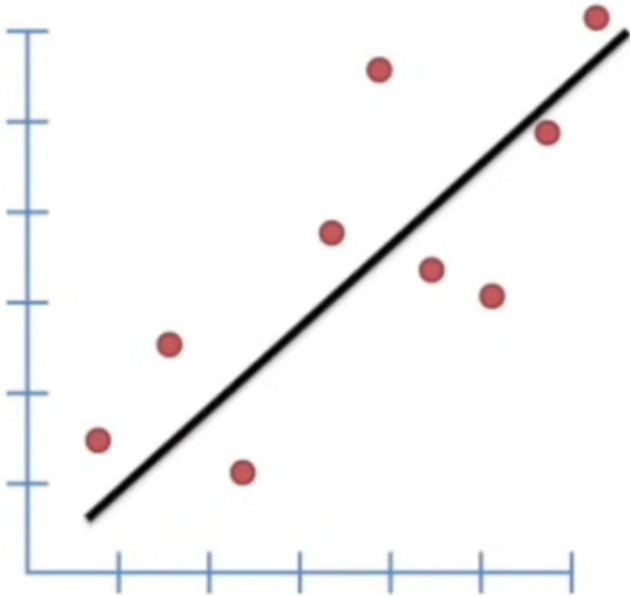
Regresión Lineal

- Suponga que tenemos unas medidas en 2D:



Regresión Lineal

- Suponga que tenemos unas medidas en 2D:

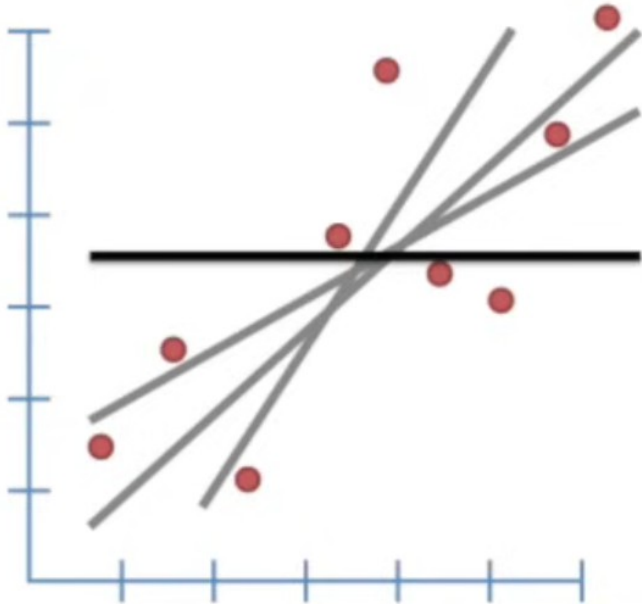


Podríamos proponer una línea que se aproxime a nuestros datos...

Pero...será la mejor?

Regresión Lineal

- Suponga que tenemos unas medidas en 2D:



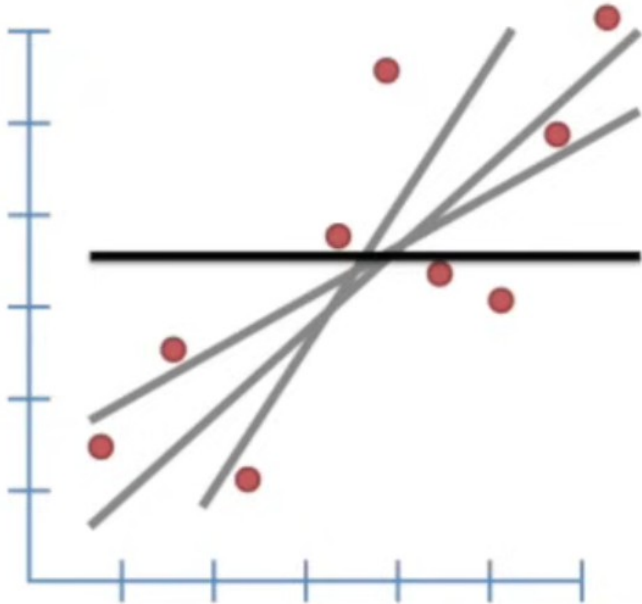
Podríamos proponer una línea que se aproxime a nuestros datos...

Pero...será la mejor?

Pueden haber muchas opciones...

Regresión Lineal

- Suponga que tenemos unas medidas en 2D:



Podríamos proponer una línea que se aproxime a nuestros datos...

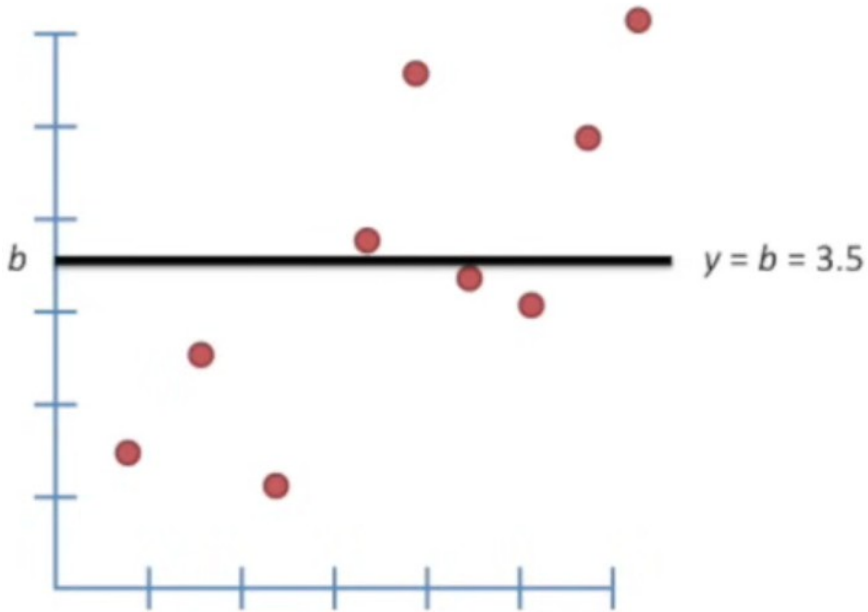
Pero...será la mejor?

Pueden haber muchas opciones...

Debemos buscar la **línea óptima** que **mejor se ajuste** a nuestros datos!

Regresión Lineal

- Suponga que tenemos unas medidas en 2D:

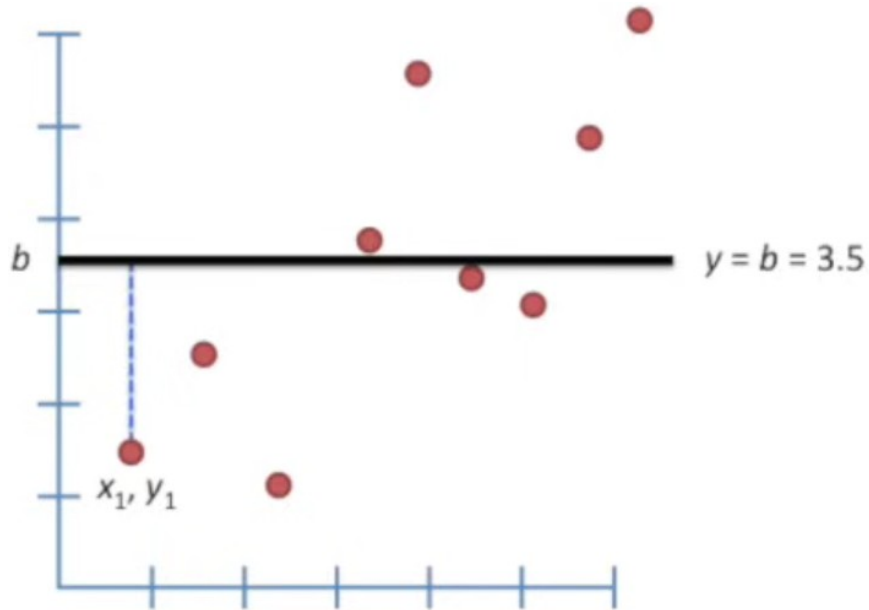


Empecemos con esta línea recta en paralelo con el eje X y que corta en $b=3.5$

Evaluemos que tan buena es esta línea...

Regresión Lineal

- Suponga que tenemos unas medidas en 2D:

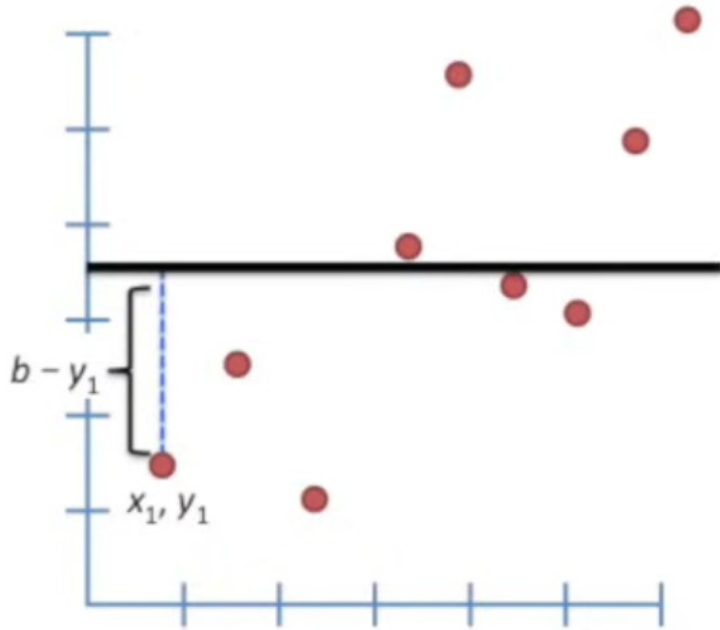


Empecemos con esta línea recta en paralelo con el eje X y que corta en $b=3.5$

Evaluemos que tan buena es esta línea...

Regresión Lineal

- Suponga que tenemos unas medidas en 2D:



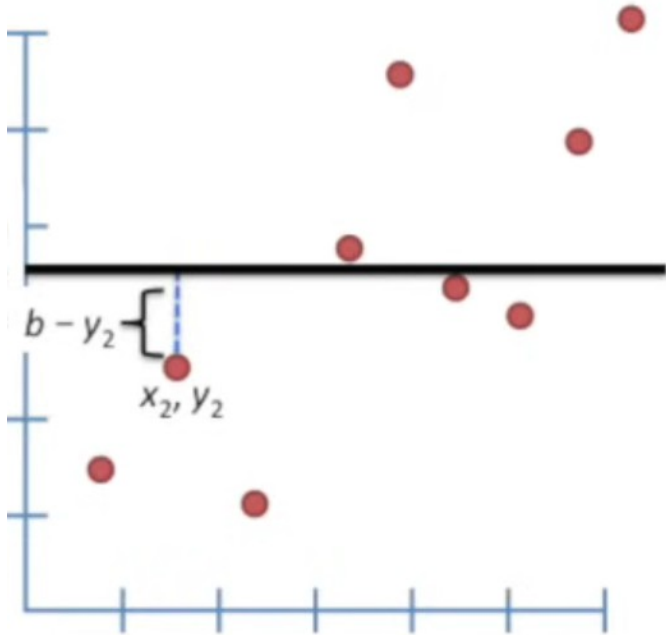
Empecemos con esta línea recta en paralelo con el eje X y que corta en $b=3.5$

Evaluemos que tan buena es esta línea...

Obtenemos la distancia desde el punto hasta la línea...

Regresión Lineal

- Suponga que tenemos unas medidas en 2D:



Empecemos con esta línea recta en paralelo con el eje X y que corta en $b=3.5$

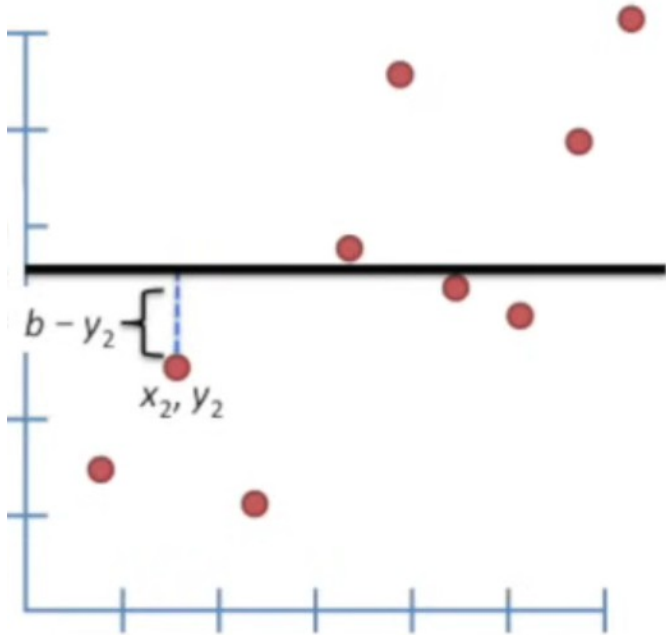
Evaluemos que tan buena es esta línea...

Obtenemos la distancia desde el punto hasta la línea...

Y realizamos lo mismo con los siguientes puntos

Regresión Lineal

- Suponga que tenemos unas medidas en 2D:

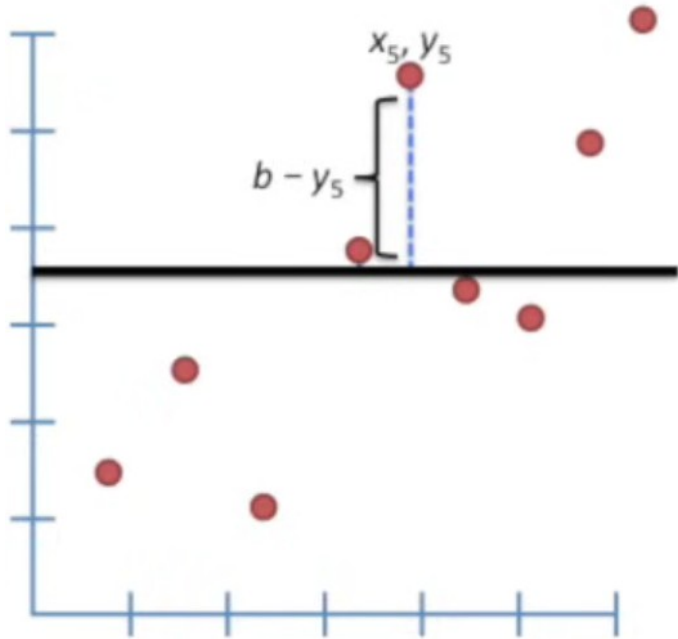


Suma total de los puntos hasta la línea hasta
ahora:

$$(b - y_1) + (b - y_2)$$

Regresión Lineal

- Suponga que tenemos unas medidas en 2D:



Suma total de los puntos hasta la línea hasta
Ahora:

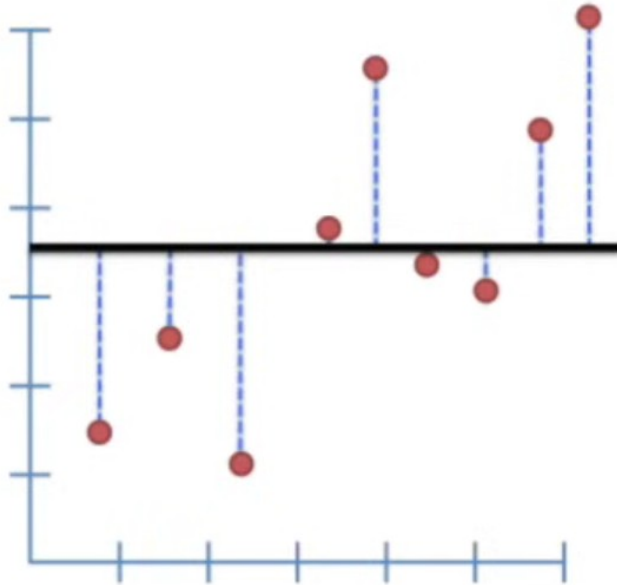
$$(b - y_1)^2 + (b - y_2)^2 + (b - y_3)^2 + (b - y_4)^2 + (b - y_5)^2$$

Elevamos al cuadrado para evitar diferencias
negativas!

Regresión Lineal

- Suponga que tenemos unas medidas en 2D:

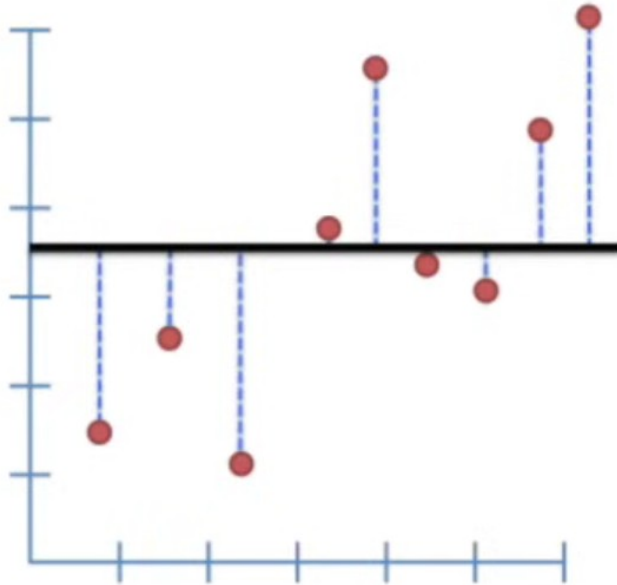
$$(b - y_1)^2 + (b - y_2)^2 + (b - y_3)^2 + (b - y_4)^2 + (b - y_5)^2 + (b - y_6)^2 + (b - y_7)^2 + (b - y_8)^2 + (b - y_9)^2$$



Regresión Lineal

- Suponga que tenemos unas medidas en 2D:

$$(b - y_1)^2 + (b - y_2)^2 + (b - y_3)^2 + (b - y_4)^2 + (b - y_5)^2 + (b - y_6)^2 + (b - y_7)^2 + (b - y_8)^2 + (b - y_9)^2$$



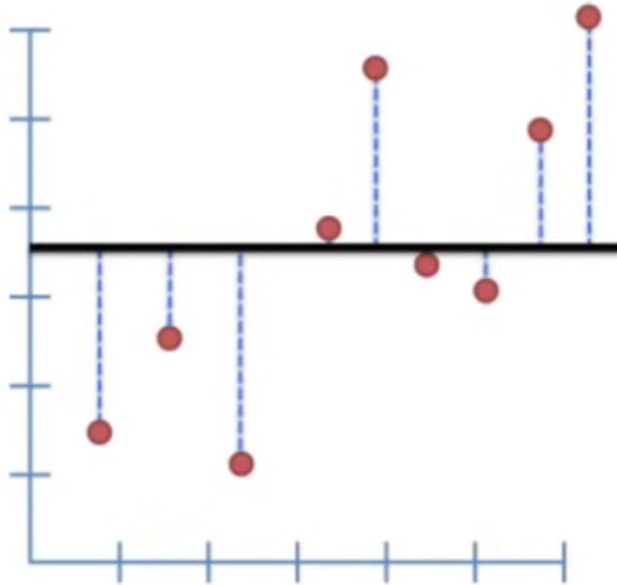
Suma de Cuadrados Residuales

Diferencia entre la data real y la línea propuesta

Regresión Lineal

- Suponga que tenemos unas medidas en 2D:

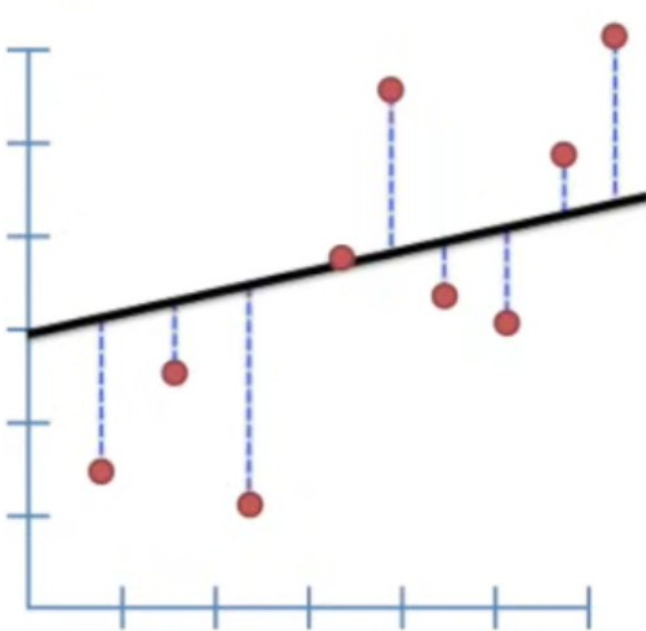
$$(b - y_1)^2 + (b - y_2)^2 + (b - y_3)^2 + (b - y_4)^2 + (b - y_5)^2 + (b - y_6)^2 + (b - y_7)^2 + (b - y_8)^2 + (b - y_9)^2$$



Resultado línea actual = 24.62

Regresión Lineal

- Suponga que tenemos unas medidas en 2D:



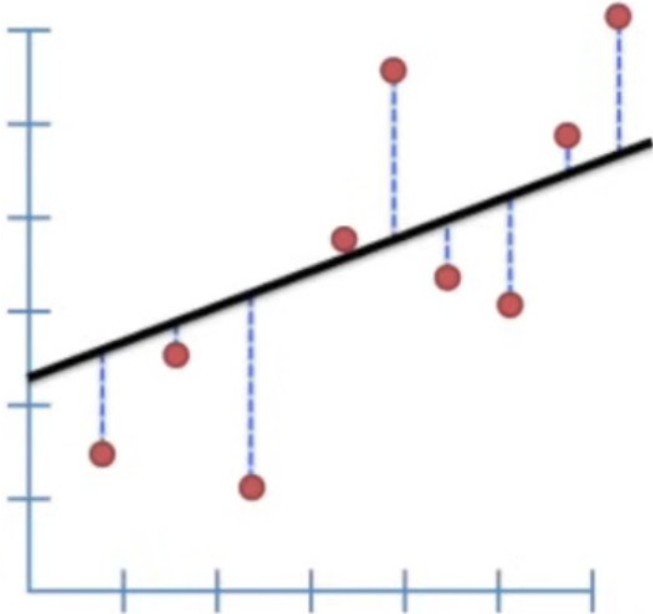
Si rotamos un poco la línea:

suma de cuadrados residuales = 18.72

Esta línea resulta **mejor** que la anterior!

Regresión Lineal

- Suponga que tenemos unas medidas en 2D:



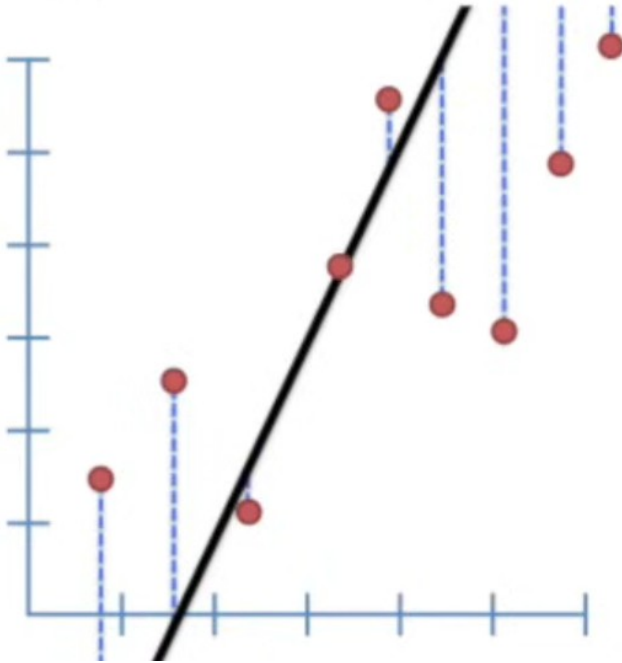
Si rotamos un poco mas la línea:

suma de cuadrados residuales = 14.05

Esta línea resulta **mejor** que la anterior!

Regresión Lineal

- Suponga que tenemos unas medidas en 2D:



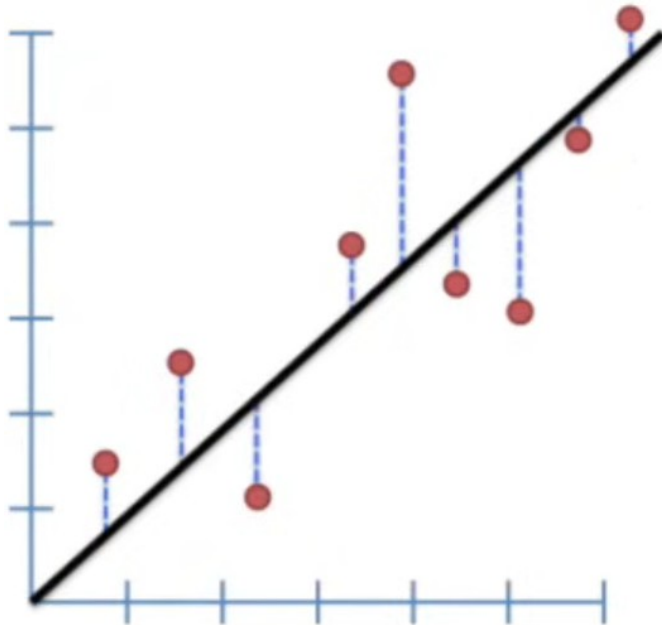
Si rotamos un poco mas la línea:

suma de cuadrados residuales = 31.71

Esta línea resulta **peor** que la anterior!

Regresión Lineal

- Suponga que tenemos unas medidas en 2D:



Ecuación de la recta:

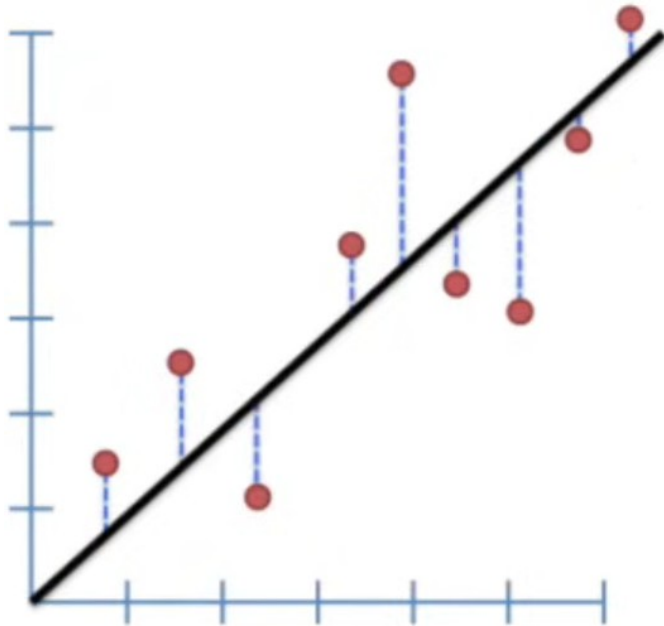
$$y = a \cdot x + b$$

Donde a es la pendiente
 b es el intercepto con el eje y .

Debemos encontrar los **valores óptimos de a y b** que minimicen la suma de los cuadrados residuales

Regresión Lineal

- Suponga que tenemos unas medidas en 2D:



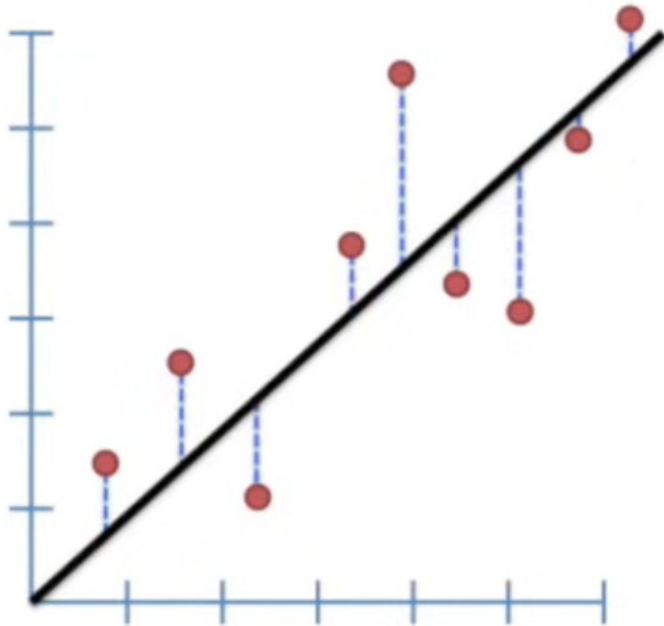
En general:

Suma de cuadrados residuales =

$$((a \cdot x_1 + b) - y_1)^2 + ((a \cdot x_2 + b) - y_2)^2 + \dots + ((a \cdot x_n + b) - y_n)^2$$

Regresión Lineal

- Suponga que tenemos unas medidas en 2D:



En general:

Suma de cuadrados residuales =

$$((a \cdot x_1 + b) - y_1)^2 + ((a \cdot x_2 + b) - y_2)^2 + \dots + ((a \cdot x_n + b) - y_n)^2$$

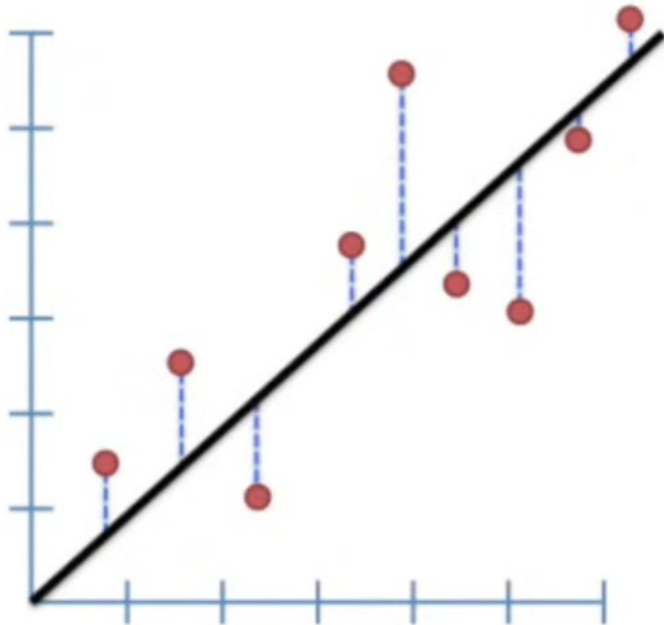
Donde:

$a \cdot x_1 + b$ es el valor de la línea en x_1

y_1 es el valor observado o medido

Regresión Lineal

- Suponga que tenemos unas medidas en 2D:



En general:

Suma de cuadrados residuales =

$$((a \cdot x_1 + b) - y_1)^2 + ((a \cdot x_2 + b) - y_2)^2 + \dots + ((a \cdot x_n + b) - y_n)^2$$

Donde:

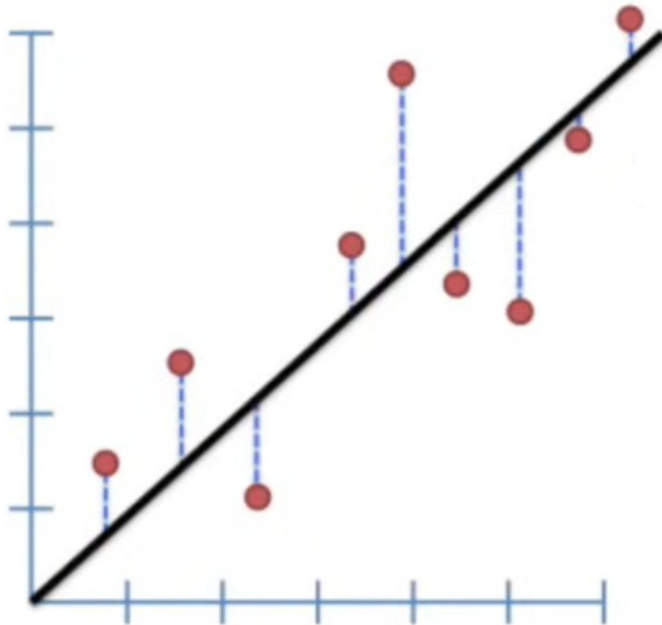
$a \cdot x_1 + b$ es el valor de la línea en x_1

y_1 es el valor observado o medido

Recordemos: estamos midiendo la distancia entre la línea propuesta (predicción) y el valor observado.

Regresión Lineal

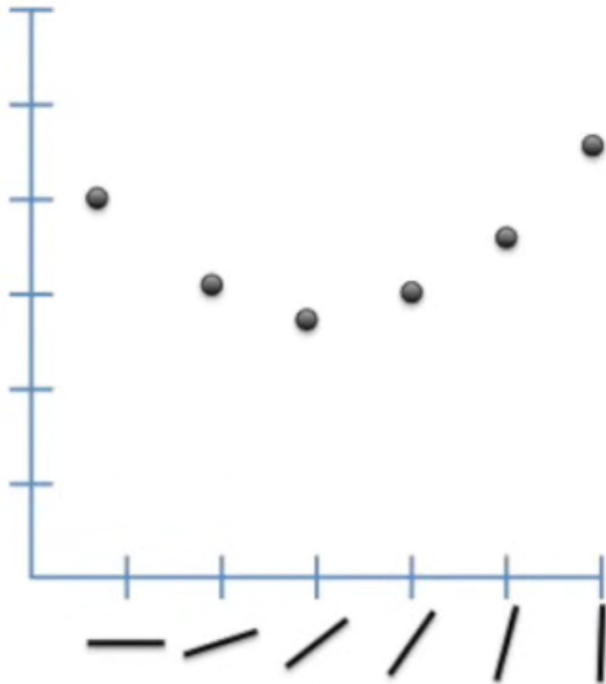
- Suponga que tenemos unas medidas en 2D:



Dado que queremos encontrar los valores de a y b que den la menor suma posible, entonces este método se denomina **Mínimos Cuadrados**.

Regresión Lineal

- Suponga que tenemos unas medidas en 2D:



Eje y: suma de cuadrados residuales.

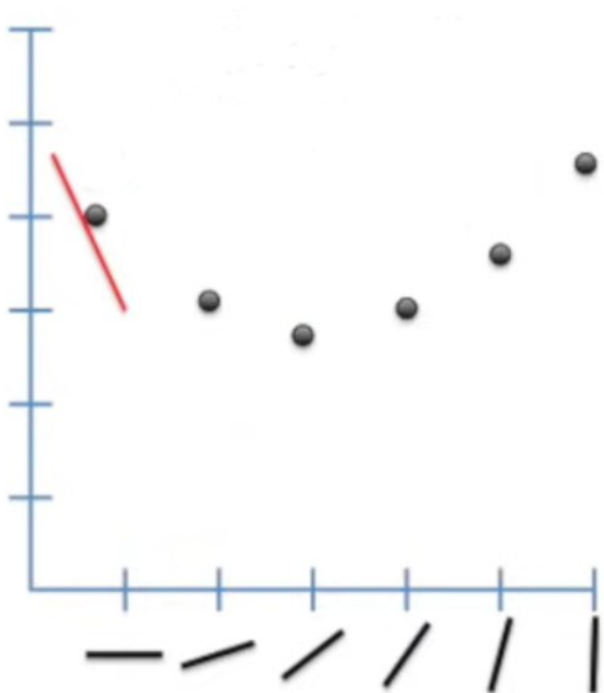
Eje x: líneas propuestas.

Cómo encontramos la rotación óptima (valores a y b)?

Rta. Derivamos la función.

Regresión Lineal

- Suponga que tenemos unas medidas en 2D:



Eje y: suma de cuadrados residuales.

Eje x: líneas propuestas.

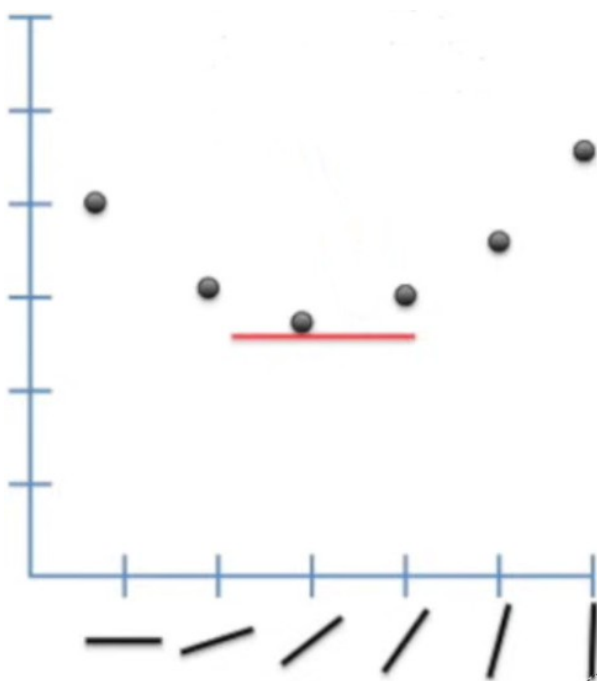
Cómo encontramos la rotación óptima (valores a y b)?

Rta. Derivamos la función en cada punto hasta que nos resulte lo mas cercana a cero.

En este primer punto la derivada sería negativa y muy distinta de cero.

Regresión Lineal

- Suponga que tenemos unas medidas en 2D:



Eje y: suma de cuadrados residuales.

Eje x: líneas propuestas.

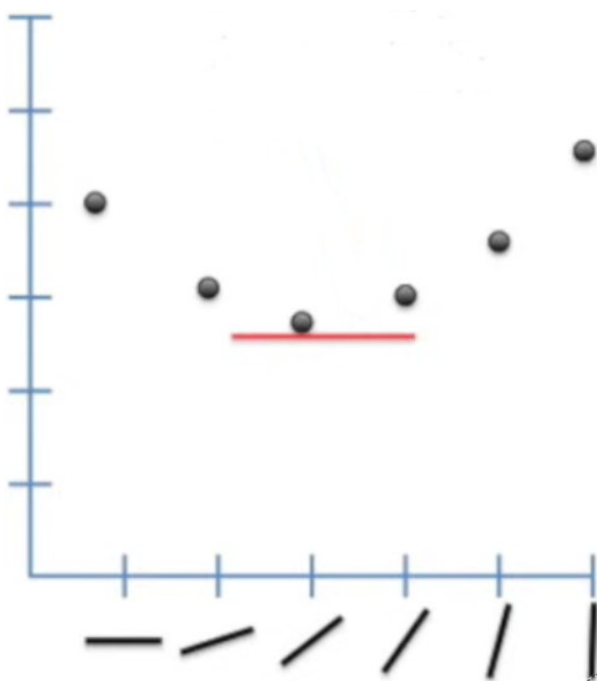
Cómo encontramos la rotación óptima (valores a y b)?

Rta. Derivamos la función en cada punto hasta que nos resulte lo mas cercana a cero.

En este tercer punto la derivada sería casi cero.

Regresión Lineal

- Suponga que tenemos unas medidas en 2D:



Eje y: suma de cuadrados residuales.

Eje x: líneas propuestas.

Cómo encontramos la rotación óptima (valores a y b)?

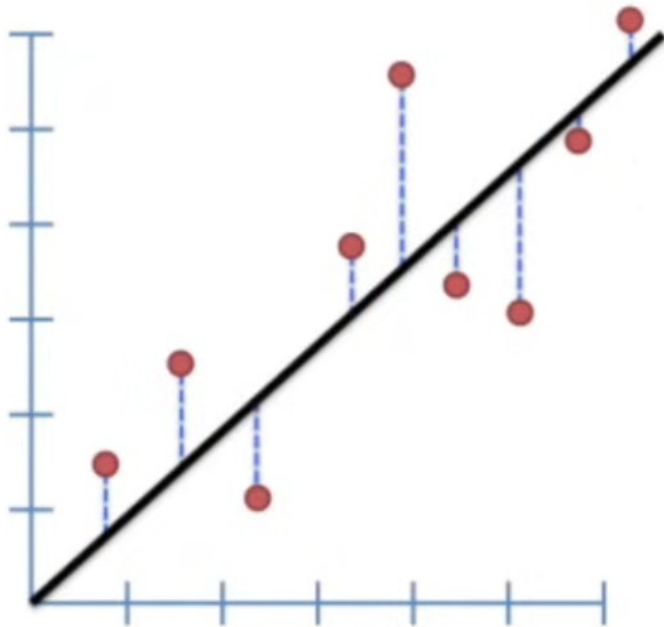
Rta. Derivamos la función en cada punto hasta que nos resulte lo mas cercana a cero.

En este tercer punto la derivada sería casi cero.

Este proceso de encontrar la derivada lo realiza un algoritmo llamado **Gradiente Descendente**.

Regresión Lineal

- Suponga que tenemos unas medidas en 2D:



Ecuación de la recta final:

$$Y = 0.77 * x + 0.66$$

Regresión Lineal

- **Algoritmo del Gradiente Descendente:**

- Recordemos...
 - Gradiente de una función:

$$\nabla f(\mathbf{r}) = \left(\frac{\partial f(\mathbf{r})}{\partial x_1}, \dots, \frac{\partial f(\mathbf{r})}{\partial x_n} \right)$$

- Norma de un vector:

$$\|\mathbf{v}\| = \sqrt{v_1^2 + v_2^2 + \dots + v_n^2}$$

$$z = x^2 + y^2;$$

$$\text{grad}_z =$$

$$2x$$

$$2y$$

Regresión Lineal

- **Algoritmo del Gradiente Descendente:**

Algorithm	Pseudocódigo del algoritmo del GD
-----------	-----------------------------------

```
1: Inicializar  $x_0$ 
2: Inicializar  $\alpha$ 
3: Inicializar convergencia
4: while  $\|\nabla f(x_0)\| > \text{convergencia}$  do
5:    $x_0 = x_0 - \alpha * \nabla f(x_0)$ 
6: end while
7:  $x^* \leftarrow x_0$ 
```

- Características del algoritmo:

- X_0 : valor de arranque.
 - Alfa: define el tamaño del paso (**Learning Rate**).
 - Convergencia: número por debajo del cual consideramos que la derivada es cero.
 - Caso 2D: primera derivada.
 - Caso 3D o ND: gradiente.

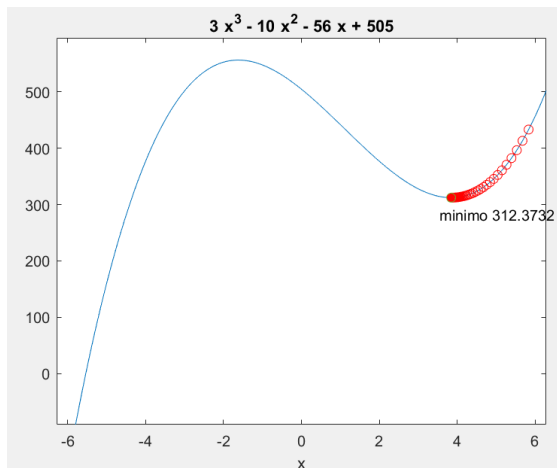
Regresión Lineal

- **Algoritmo del Gradiente Descendente:**

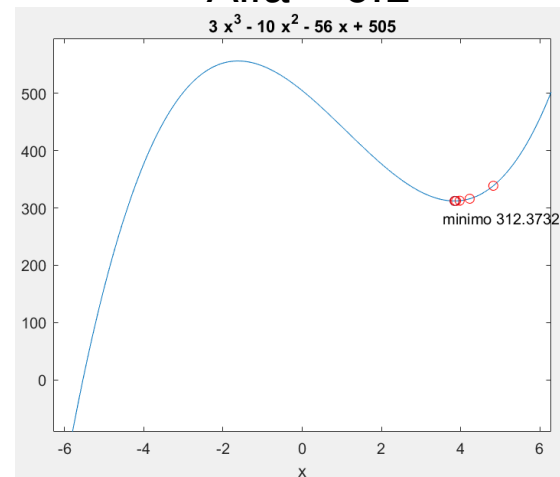
Algorithm Pseudocódigo del algoritmo del GD

```
1: Inicializar  $x_0$   
2: Inicializar  $\alpha$   
3: Inicializar convergencia  
4: while  $\|\nabla f(x_0)\| > \text{convergencia}$  do  
5:    $x_0 = x_0 - \alpha * \nabla f(x_0)$   
6: end while  
7:  $x^* \leftarrow x_0$ 
```

Alfa < 0.1



Alfa > 0.1



Regresión Lineal

- **Algoritmo del Gradiente Descendente:**

- Aplicación del GD a la regresión lineal:

Algorithm	Pseudocodigo del algoritmo del GD Modificado
1: Inicializar x_0	→ $[a_0, b_0]$
2: Inicializar α	
3: Inicializar convergencia	
4: while $\ \nabla f(x_0)\ > \text{convergencia}$ do	
5: $x_0 = x_0 - \alpha * \nabla f(x_0)$	
6: end while	
7: $x^* \leftarrow x_0$	→ $((a*x_1 + b) - y_1)^2 + ((a*x_2 + b) - y_2)^2 + \dots((a*x_n + b) - y_n)^2$

- X_0 : debe ser un vector columna de 2x1.
- El gradiente de la función resulta en un vector columna de 2x1.
 - Componente 1 del gradiente: Se calcula la derivada con respecto a “a”.
 - Componente 2 del gradiente: Se calcula la derivada con respecto a “b”.
- El nuevo X_0 resulta en un nuevo vector columna de 2x1.

Bono examen 1

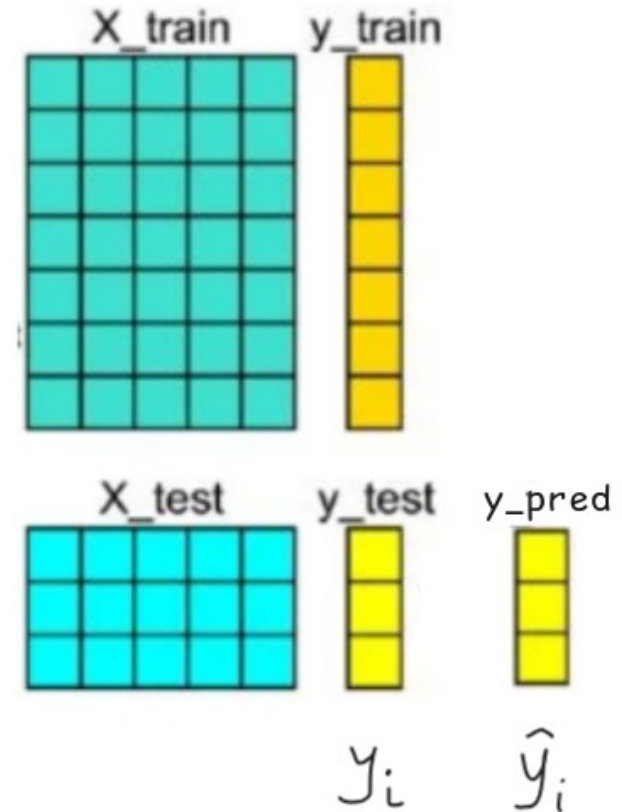
- Consulte la solución analítica para la Regresión Lineal Simple y Múltiple.
 - Un pdf con la explicación con sus propias palabras.

Regresión Lineal

- Métricas de la regresión lineal:
 - **Error Medio Absoluto (MAE):**
es el promedio de las distancias entre las predicciones y los valores reales.
 - Error = Real - Predicción

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

- Entre mas bajo, mejor.



Regresión Lineal

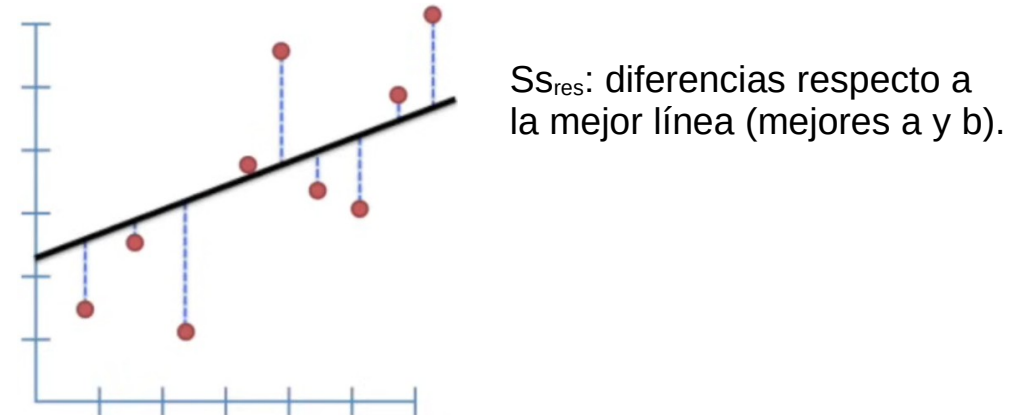
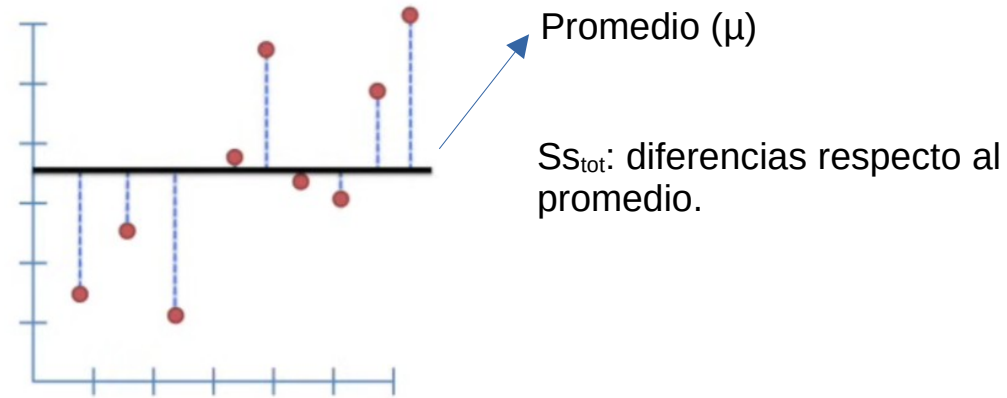
- Métricas de la regresión lineal:
 - **R^2 (Coeficiente de Determinación):** calcula qué tan "bien" encaja la predicción con los datos reales.
 - $R^2=1$: El modelo es perfecto. Explica el 100% de la variabilidad. Todas las predicciones caen exactamente sobre los puntos reales.
 - $R^2=0$: La predicción no explica para nada el comportamiento de los datos reales.
 - $R^2=0.85$: Significa que el 85% de los cambios en la variable "Y" se deben a las variables "X". El otro 15% es ruido o factores que no se está tomando en cuenta.

Regresión Lineal

- Métricas de la regresión lineal:
 - **R² (Coeficiente de Determinación):** calcula qué tan "bien" encaja la predicción con los datos reales.

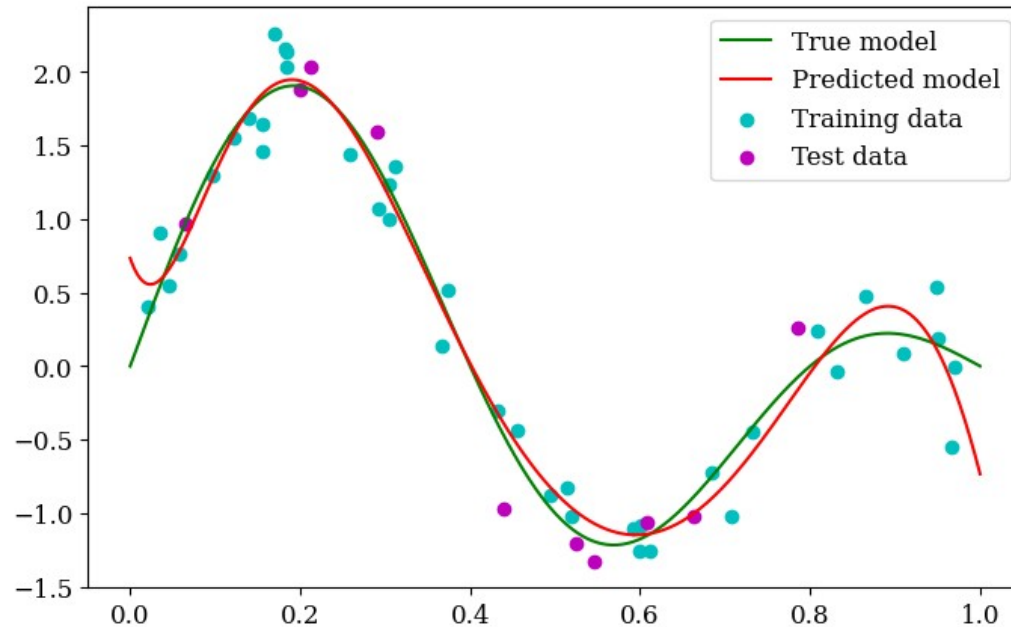
$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$

- SS_{res} (Suma de Cuadrados de los Residuos): El error acumulado del modelo.
- SS_{tot} (Suma de Cuadrados Totales): La varianza total de los datos originales (la distancia de los puntos respecto a su propio promedio).



Regresión Lineal

- Implementación:
 - Revisar el archivo ej_RL_basica.ipynb



Regresión Lineal

- Implementación:
 - Revisar el archivo ej_RL_multiple.ipynb

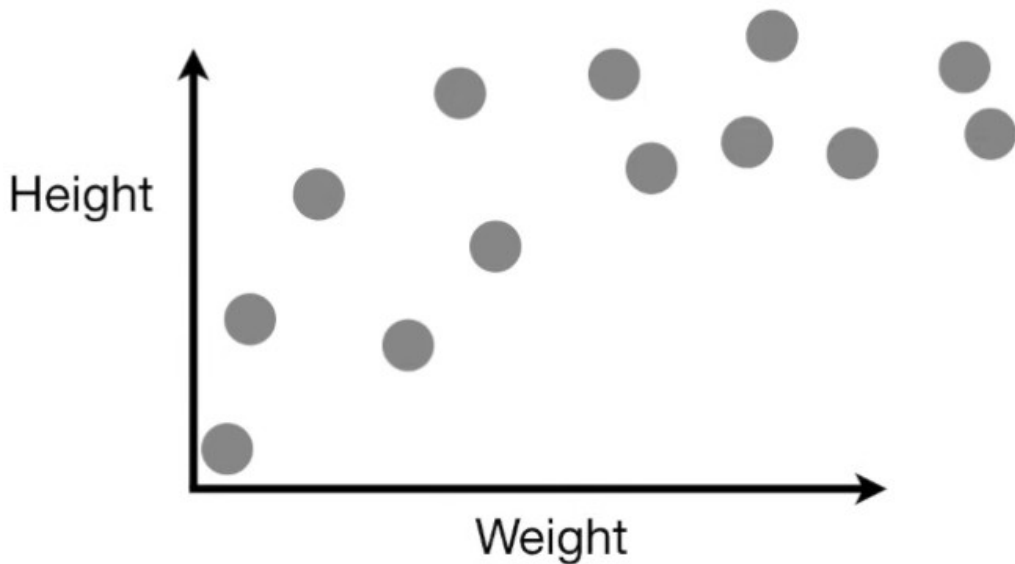
	area_m2	habitaciones	antiguedad_anios	distancia_centro_km	precio_millones
0	118.653425	5	28.454073	12.727885	4.806821
1	239.650004	4	3.175591	25.973785	7.847014
2	193.718728	3	1.841093	27.727327	5.975959
3	165.718282	4	6.692606	14.238154	6.615729
4	72.763914	5	0.683598	14.684691	3.590828

Sobreaajuste (Overfitting)

- Definición:
 - Ocurre cuando un modelo estadístico tiene una complejidad excesiva en relación con la cantidad y calidad de los datos disponibles, lo que le permite capturar no solo la señal subyacente del fenómeno, sino también el ruido aleatorio del conjunto de entrenamiento.

Sobreajuste (Overfitting)

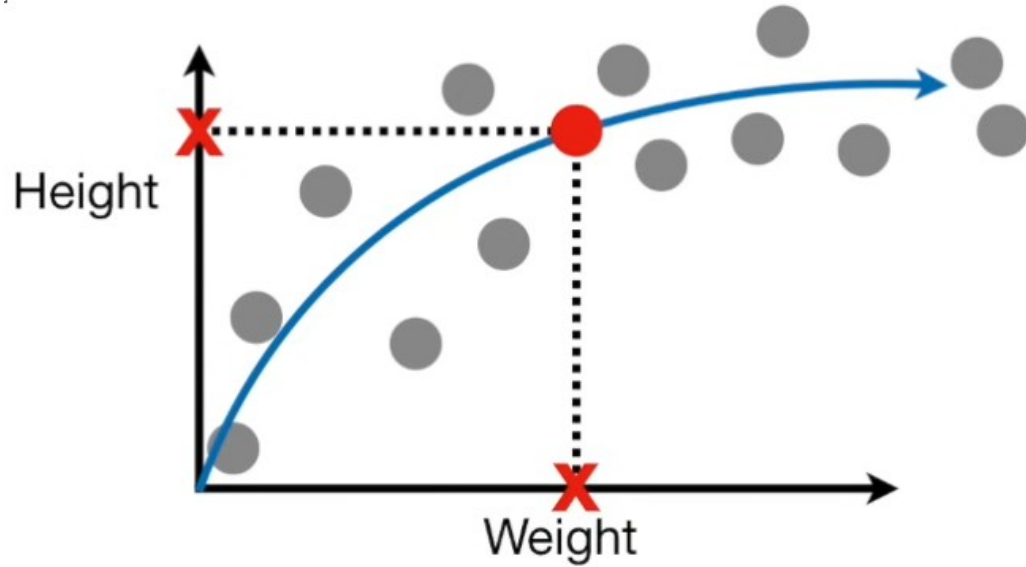
- Concepto general:



Suponga que queremos predecir el largo de un Ratón dado su peso.

Sobreajuste (Overfitting)

- Concepto general:

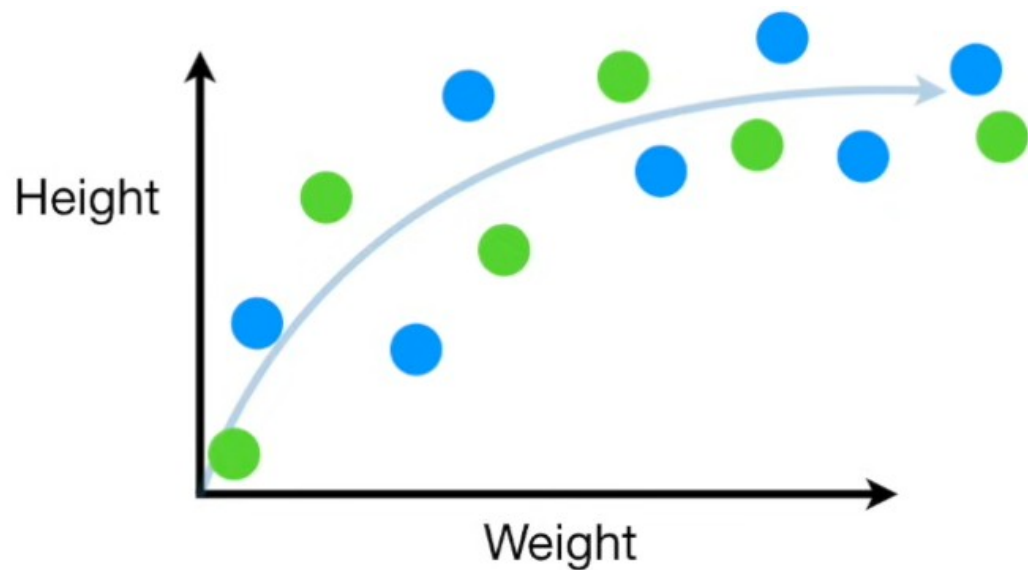


Suponga que queremos predecir el largo de un Ratón dado su peso.

Deseamos una curva que se aproxime a los datos reales

Sobreajuste (Overfitting)

- Concepto general:



Suponga que queremos predecir el largo de un Ratón dado su peso.

Deseamos una curva que se aproxime a los datos reales

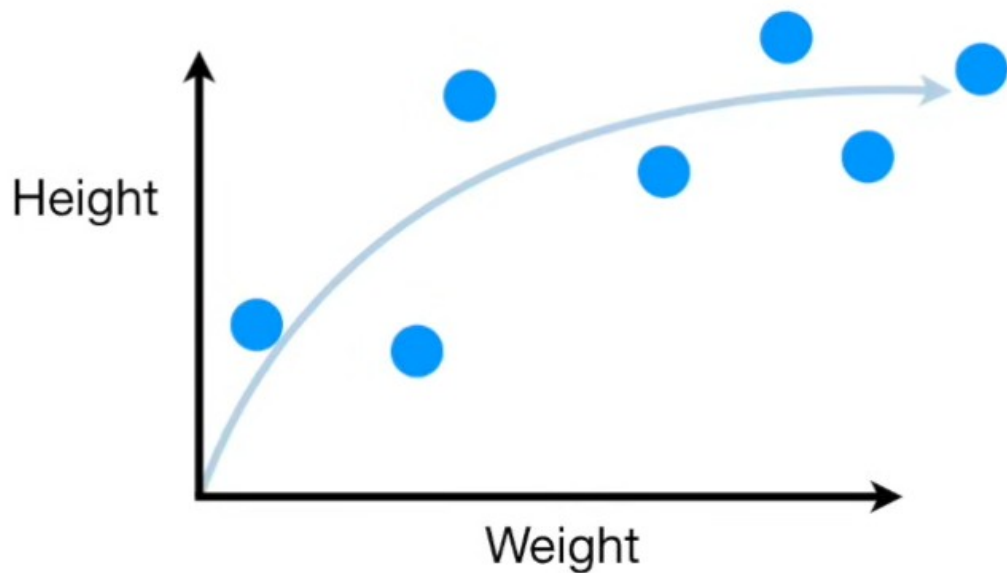
Dividimos los datos en entrenamiento y prueba:

Azules → entrenamiento

Verdes → prueba

Sobreajuste (Overfitting)

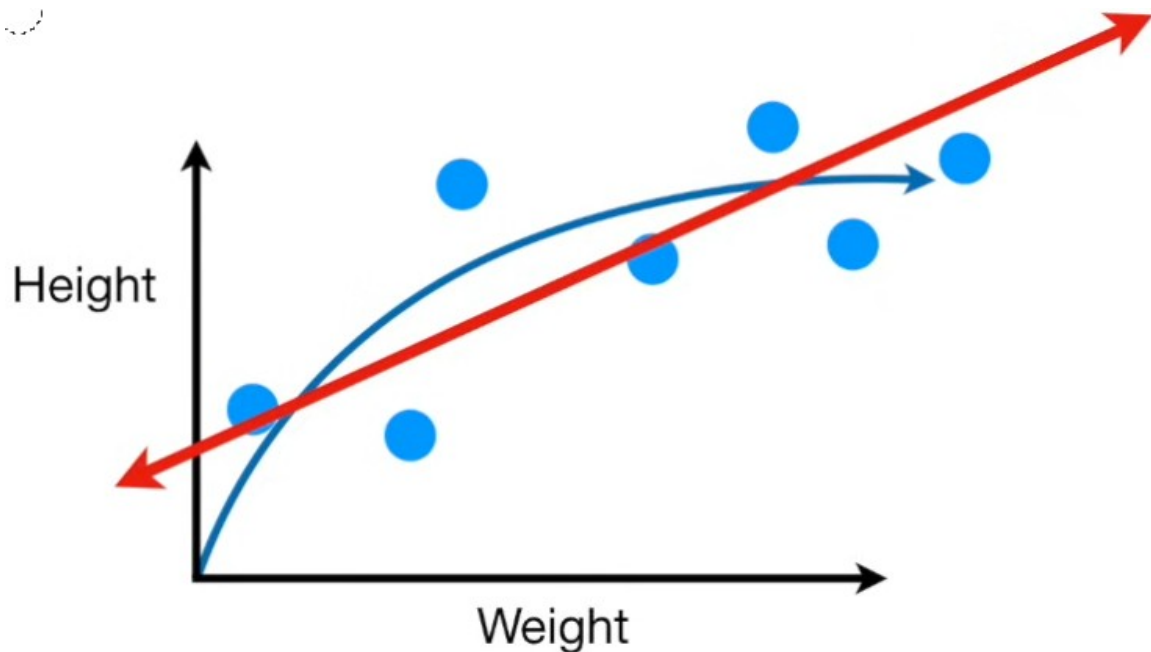
- Concepto general:



Dejemos lo de entrenamiento porque son sobre los cuales nuestro modelo va a aprender...

Sobreajuste (Overfitting)

- Concepto general:

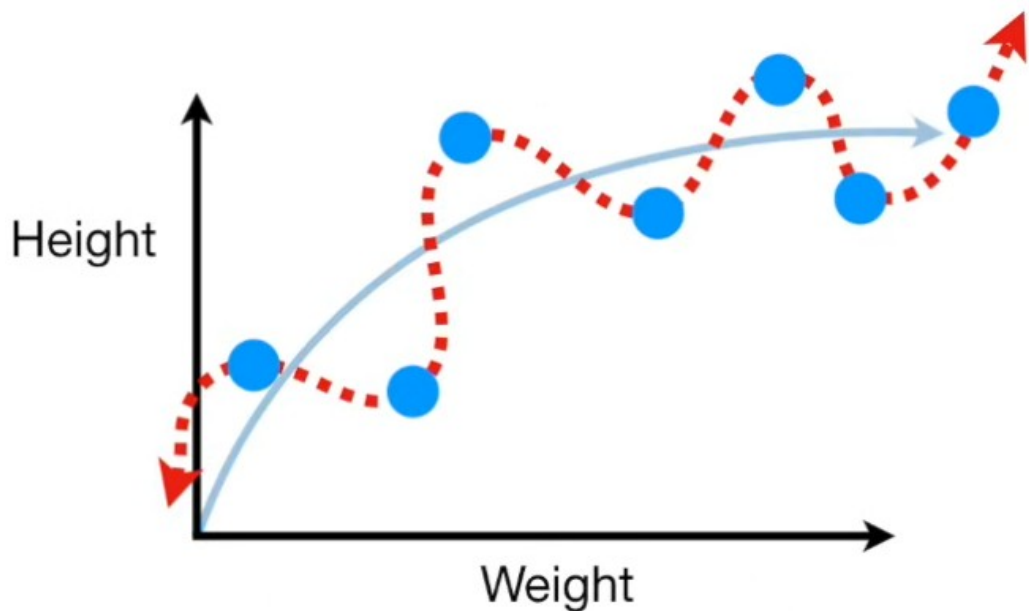


Dejemos los de entrenamiento porque son sobre los cuales nuestro modelo va a aprender...

Método 1: aplicamos una regresión lineal

Sobreajuste (Overfitting)

- Concepto general:



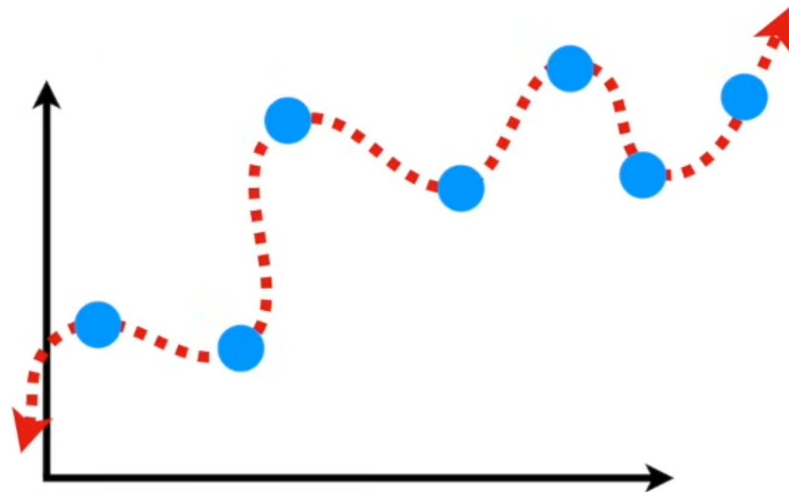
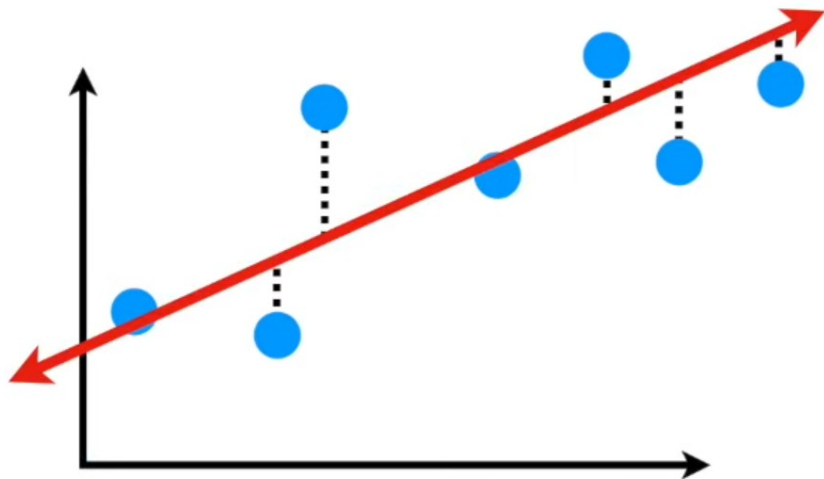
Dejemos los de entrenamiento porque son sobre los cuales nuestro modelo va a aprender...

Método 1: aplicamos una regresión lineal

Método 2: aplicamos otro método que se ajusta exactamente a los datos del entrenamiento

Sobreajuste (Overfitting)

- Concepto general:



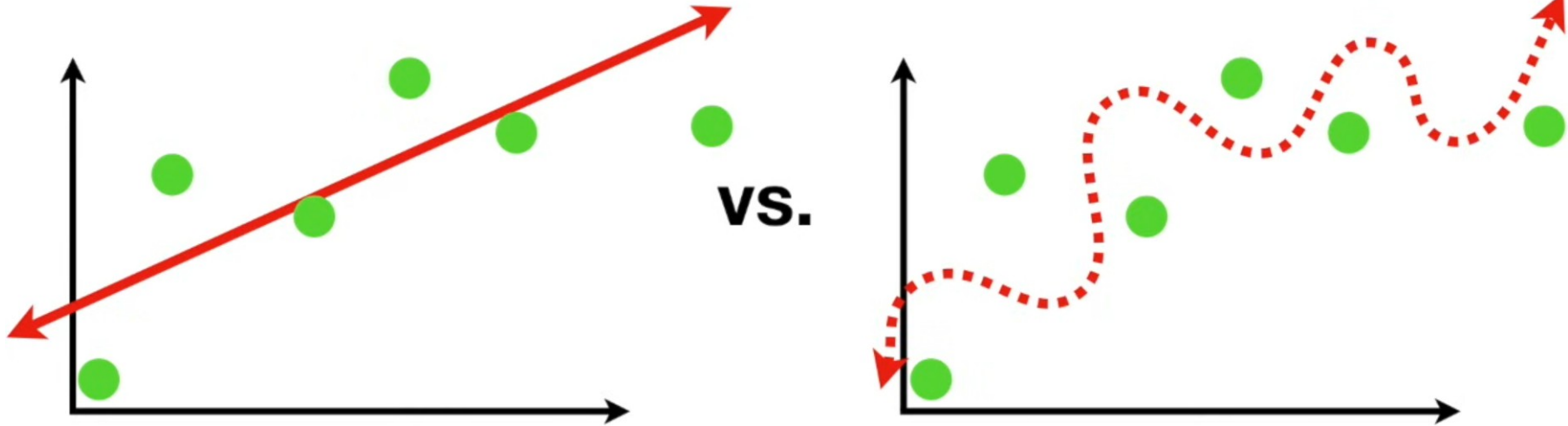
Calculemos la suma de cuadrados de los dos métodos para el entrenamiento:

Método 1: será un valor mayor a 0.

Método 2: será igual a 0.

Sobreajuste (Overfitting)

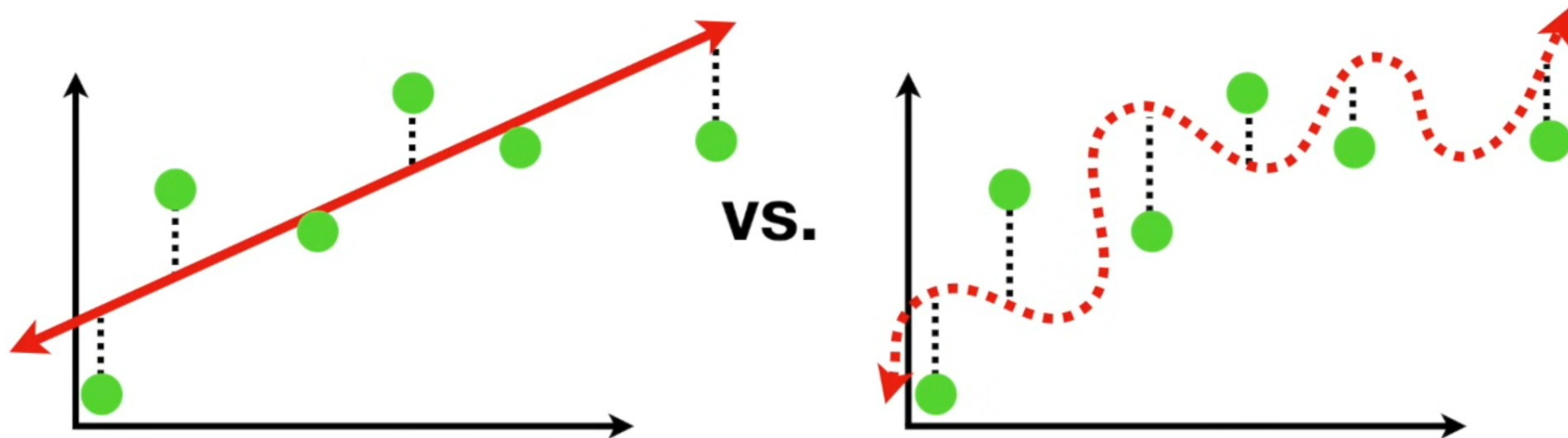
- Concepto general:



Calculemos la suma de cuadrados de los dos métodos para prueba:

Sobreajuste (Overfitting)

- Concepto general:

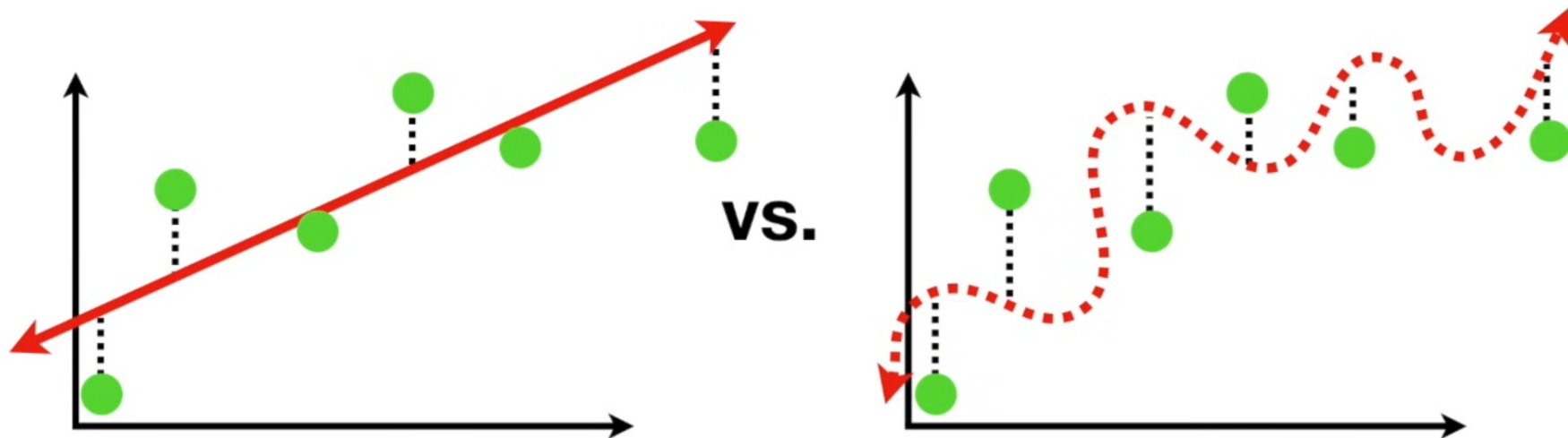


Calculemos la suma de cuadrados de los dos métodos para prueba:

El método 1 (la línea recta) gana, así el método 2 haya una labor perfecta en el entrenamiento.

Sobreajuste (Overfitting)

- Concepto general:



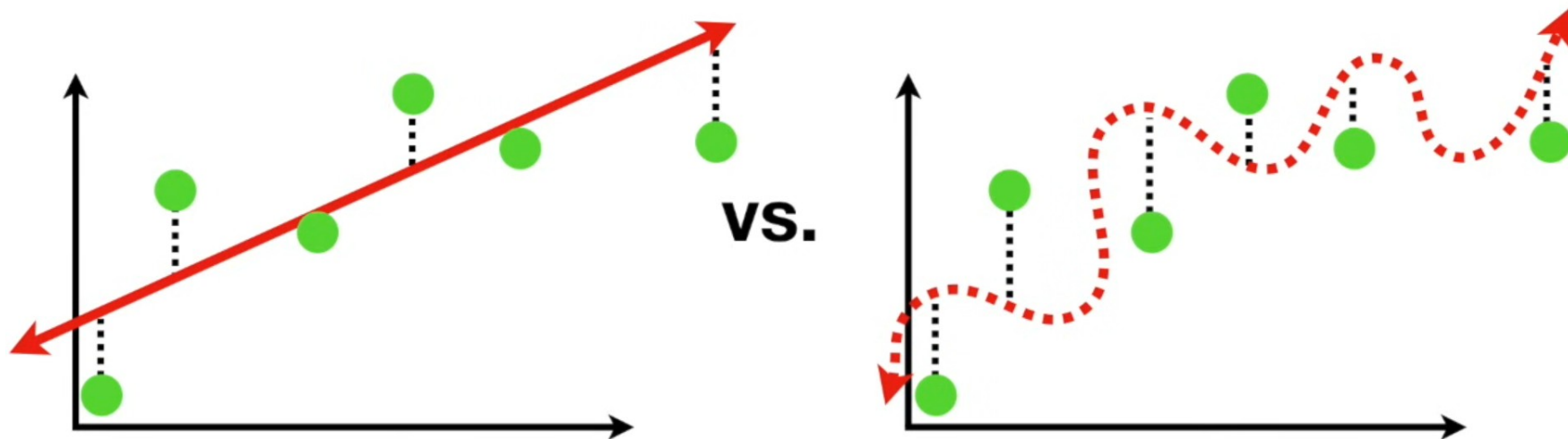
Calculemos la suma de cuadrados de los dos métodos para prueba:

El método 1 (la línea recta) gana, así el método 2 haya hecho una labor perfecta en el entrenamiento.

El método 1 será consistente en el tiempo contrario al método 2.

Sobreajuste (Overfitting)

- Concepto general:



Entonces, cuando un método se desempeña
Muy bien en entrenamiento pero muy mal en
La prueba, se dice que está **“Sobreajustado”**.

Sobreajuste (Overfitting)

- Conclusión:
 - Si para alguna métrica que estemos evaluando de algún modelo de aprendizaje automático, esta métrica difiere considerablemente para el entrenamiento y prueba, entonces muy probablemente se está presentando un *Sobreajuste*.

Sobreajuste (Overfitting)

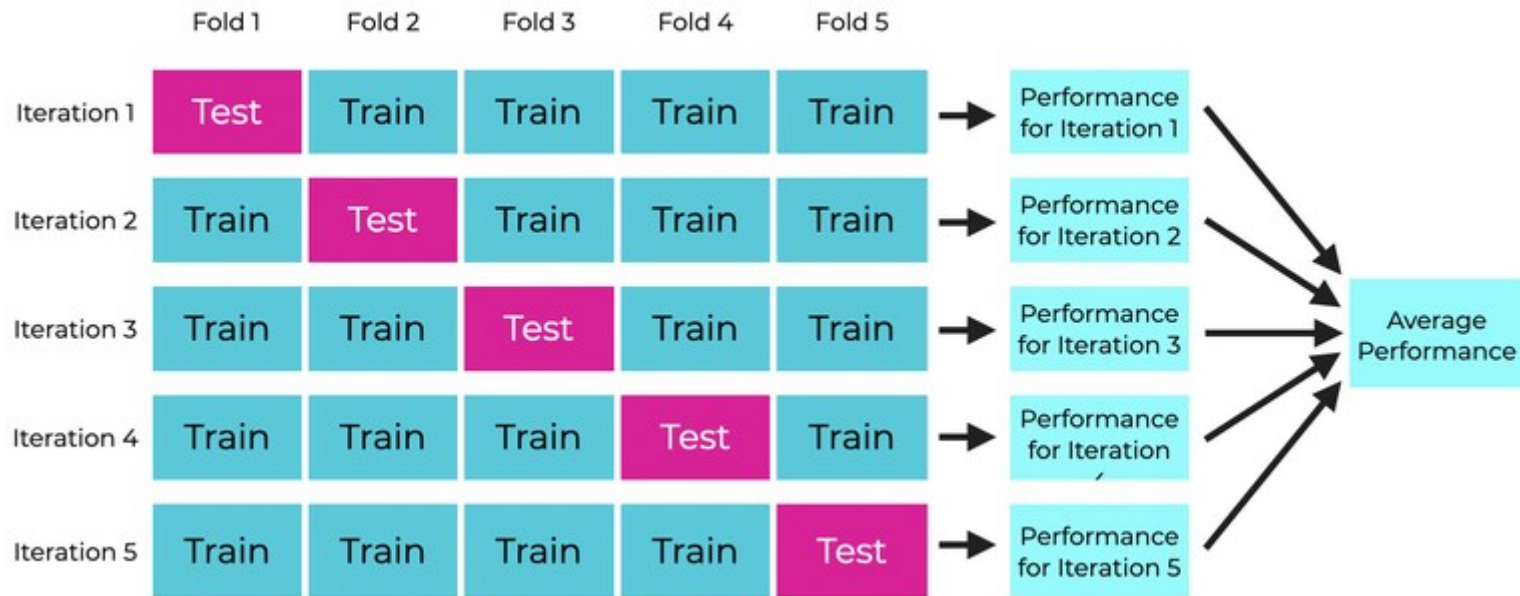
- **Métodos generales para combatir el sobre ajuste:**
 - Aumentar el tamaño del dataset
 - Limpieza de datos: eliminar ruido y outliers que el modelo podría memorizar
 - Reducción de complejidad del modelo
 - Reducir columnas.
 - Validación cruzada
 - Regularización

Validación cruzada

- Definición:
 - Es una técnica estadística utilizada para evaluar la capacidad de generalización de un modelo de Machine Learning. Su objetivo es garantizar que el rendimiento que observamos no sea producto del azar o de una partición "afortunada" de los datos.
 - En lugar de dividir los datos una sola vez (en entrenamiento y prueba), la validación cruzada repite el proceso varias veces utilizando diferentes subdivisiones.

Validación cruzada

- Concepto general:



Validación cruzada

- Implementación:
 - Revisar el archivo ej_RL_cv_pipeline.ipynb

Actividad en clase

- Del dataset escogido o de un nuevo dataset, implemente la Regresión Lineal y analice sus resultados.

Regresión Lineal

- **Regularización:**

- Técnica para evitar evitar el overfitting.
- Su función principal es "frenar" al modelo para que no se vuelva demasiado complejo y no memorice el ruido de los datos de entrenamiento.
- Es una penalización que se le aplica al modelo por tener coeficientes (pesos) demasiado grandes o por usar demasiadas variables.

Regresión Lineal

- Regularización:
 - Qué sucede si los coeficientes son muy grandes?
 - Un coeficiente es básicamente un multiplicador.
 - Si el coeficiente de una variable x es 1,000,000, un cambio diminuto en esa variable (por ejemplo, un error de redondeo o ruido en la medición) provocará un cambio masivo en la predicción final.
 - Modelo sano: Cambios pequeños en los datos → Cambios pequeños en la predicción.

Regresión Lineal

- Regularización:
 - Qué busca la regularización? Rta. Opción B.
 - Opción A (Sin regularización): $y=0.1x_1+10,000x_2$
 - Opción B (Con Ridge/Lasso): $y=0.5x_1+0.8x_2$
 - La Opción B es mucho más preferible porque es más simple y robusta. Si x_2 varía un poquito por error humano, la opción A arruinará la predicción, mientras que la B apenas se inmutará.

Regresión Lineal

- Tipos de regularización:
 - Lasso (L1)
 - Ridge (L2)

Regresión Lineal

- Tipos de regularización:
 - Lasso (L1)
 - La función de costo añade una penalización basada en el valor absoluto de los coeficientes.

$$J(\beta) = \text{MSE} + \alpha \sum_{j=1}^n |\beta_j|$$

- Donde alfa controla la fuerza de la penalización
- $|\beta_j|$: Es el valor absoluto de los coeficientes de la regresión.

Regresión Lineal

- Tipos de regularización:
 - Lasso (L1)
 - La función de costo añade una penalización basada en el valor absoluto de los coeficientes.

$$J(\beta) = \text{MSE} + \alpha \sum_{j=1}^n |\beta_j|$$

- Lasso puede forzar a que algunos coeficientes sean exactamente cero. Esto significa que Lasso decide que algunas variables no son importantes y las elimina por completo del modelo.

Regresión Lineal

- Tipos de regularización:
 - Lasso (L1)
 - Se debe usar cuando se tiene un dataset con muchas columnas, pero que solo unas pocas son realmente útiles.

Regresión Lineal

- Tipos de regularización:
 - Ridge (L2)
 - La función de costo añade una penalización basada en el cuadrado de los coeficientes.

$$J(\beta) = \text{MSE} + \alpha \sum_{j=1}^n \beta_j^2$$

- Donde alfa controla la fuerza de la penalización
- β_j^2 : Es el cuadrado de cada coeficiente (peso) del modelo.

Regresión Lineal

- Tipos de regularización:
 - Ridge (L2)
 - La función de costo añade una penalización basada en el cuadrado de los coeficientes.

$$J(\beta) = \text{MSE} + \alpha \sum_{j=1}^n \beta_j^2$$

- A diferencia de Lasso (L1), Ridge nunca hace que un coeficiente sea exactamente cero. Los hace diminutos, pero mantiene todas las variables en el modelo.

Regresión Lineal

- Tipos de regularización:
 - Ridge (L2)
 - Se debe usar cuando se tiene un dataset con muchas columnas, pero sospechamos que todas inciden de alguna manera en la variable objetivo.

Regresión Lineal

- Regularización:
 - Implementación:
 - Hace uso de los **Pipelines**.
 - Un pipeline en Python es una secuencia de pasos de procesamiento de datos donde la salida de cada paso se convierte automáticamente en la entrada del siguiente.
 - Utilidad:
 - Automatizar flujos de trabajo: Evita ejecutar manualmente cada paso de limpieza, transformación o modelado de datos.
 - Organizar y simplificar el código: Convierte un proceso complejo de decenas de líneas dispersas en una sola estructura limpia y reutilizable.
 - Facilitar el despliegue a producción: Permite aplicar exactamente el mismo pipeline a datos nuevos con un solo comando.

Regresión Lineal

- Regularización:
 - Implementación:
 - Hace uso de los **Pipelines**.
 - Ejemplo:

```
1 from sklearn.pipeline import Pipeline
2 from sklearn.preprocessing import StandardScaler
3 from sklearn.linear_model import LogisticRegression
4
5 # Definición del pipeline con 2 pasos
6 pipeline = Pipeline([
7     ('escalado', StandardScaler()),      # Paso 1: Normaliza los datos
8     ('modelo', LogisticRegression())    # Paso 2: Entrena el modelo
9 ])
10
11 # Ejecuta todos los pasos secuencialmente en una sola línea
12 pipeline.fit(X_train, y_train)
13
14 # Aplica las mismas transformaciones y predice sobre datos nuevos
15 predicciones = pipeline.predict(X_test)
16
```

Regresión Lineal

- Regularización:
 - Implementación:
 - Revisar `ej_RL_regular_pipe.ipynb`

Regresión Lineal

- Conclusiones:
 - En la regresión simple, buscamos la mejor línea.

$$y = \beta_0 + \beta_1 x$$

- En la regresión múltiple con 2 variables, buscamos el mejor plano (3D).
 - Con 3 o más variables, buscamos un hiperplano.

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n$$

Regresión Logística

- Definición:
 - Se utiliza para problemas de clasificación, no de regresión.
 - Su objetivo es predecir la probabilidad de que una observación pertenezca a una categoría específica (como "Sí/No", "Spam/No Spam", "Enfermo/Sano").
 - Predice una probabilidad que siempre está entre 0 y 1.

Regresión Logística

- Definición:

- Formula:

$$f(z) = \frac{1}{1 + e^{-z}}$$

- Donde: $z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3 + \dots + \beta_n x_n$
 - $f(z)$ indica la probabilidad de que una muestra del dataset corresponda a una categoría binaria (SPAM/No SPAM, Sano/Enfermo, etc). A $f(z)$ también se le conoce como $h_\theta(x)$

Regresión Logística

- Definición:

- Formula:

$$f(z) = \frac{1}{1 + e^{-z}}$$

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3 + \cdots + \beta_n x_n$$

- A medida que $z \rightarrow \infty$, el término $e^{-z} \rightarrow 0$, por lo cual $f(z) \rightarrow 1$
 - A medida que $z \rightarrow -\infty$, el término $e^{-z} \rightarrow \infty$, por lo cual $f(z) \rightarrow 0$
 - Cuando $z = 0$, $f(0) = 0,5$. Este valor se utiliza habitualmente como el umbral de decisión (threshold) estándar para la clasificación binaria.

Regresión Logística

- Definición:

- Formula:

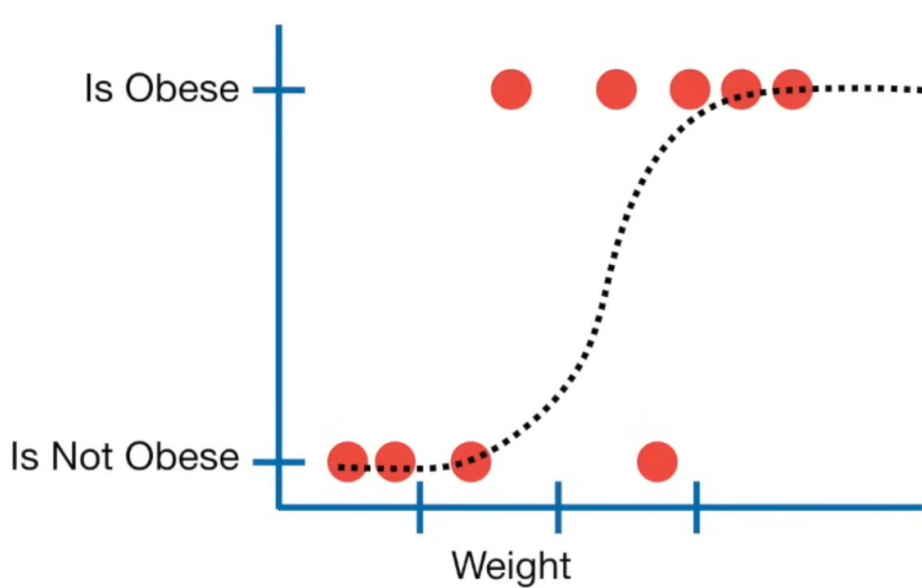
$$f(z) = \frac{1}{1 + e^{-z}}$$

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3 + \cdots + \beta_n x_n$$

- Si $\beta_i > 0$ (Positivo): Al aumentar x_i , el valor de z aumenta. Esto significa que la probabilidad de éxito aumenta. (Ej: Más horas de estudio → Más probabilidad de aprobar).
 - Si $\beta_i < 0$ (Negativo): Al aumentar x_i , el valor de z disminuye. Esto significa que la probabilidad de éxito disminuye. (Ej: Más ausencias a clase → Menos probabilidad de aprobar).
 - Si $\beta_i = 0$: La variable x_i no tiene ningún impacto en la probabilidad.

Regresión Logística

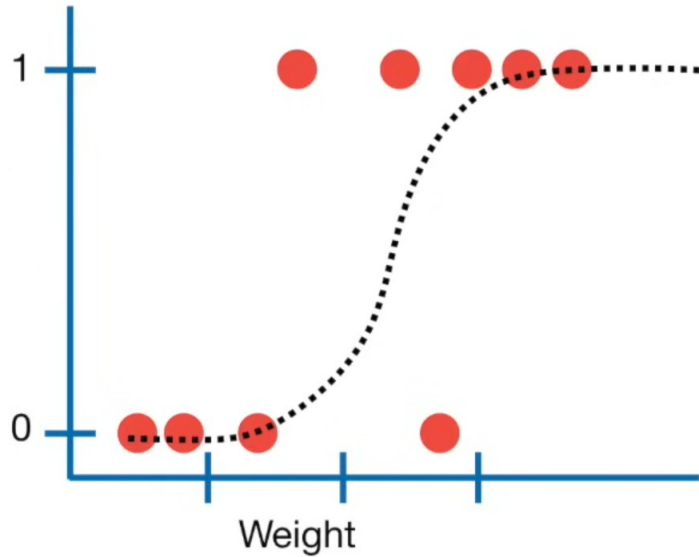
- Concepto intuitivo:



En lugar de encontrar una línea como en la regresión lineal, la idea es encontrar una función logística en forma de “S”.

Regresión Logística

- Concepto intuitivo:



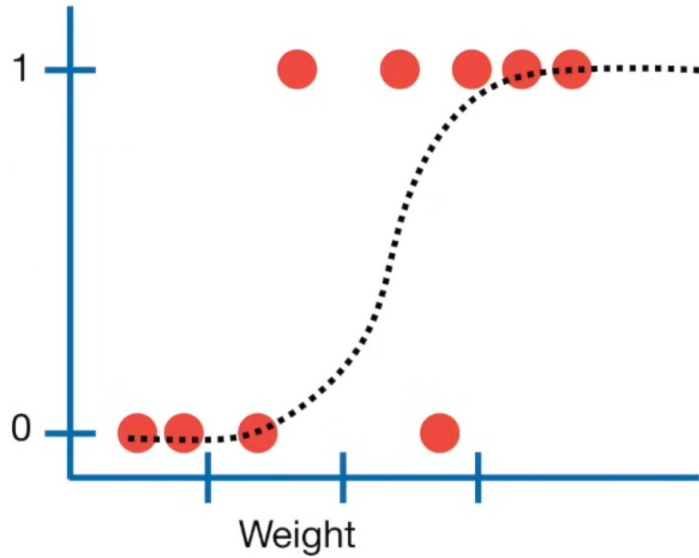
En lugar de encontrar una línea como en la regresión lineal, la idea es encontrar una función logística en forma de “S”.

Esta curva nos dice la probabilidad de que un ratón este obeso basado en su peso.

Si el peso es muy alto, la probabilidad de que esté obeso es muy alta, y por lo tanto, se clasifica como “obeso”, de otro modo, se clasificaría como “no obeso”.

Regresión Logística

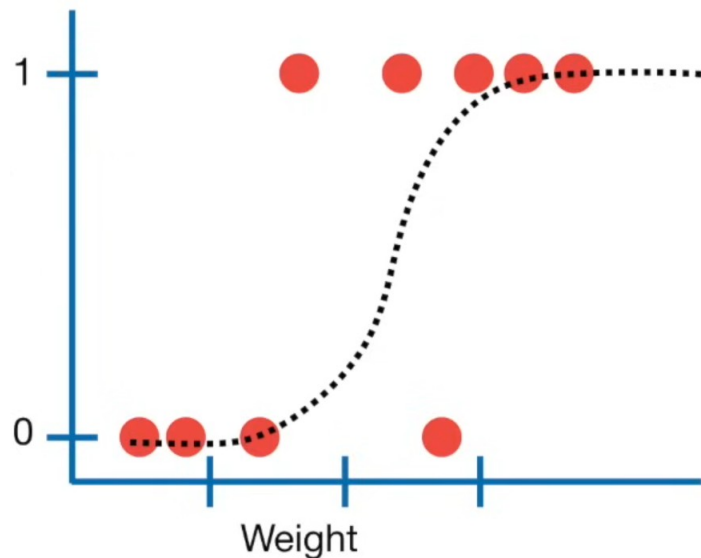
- Concepto intuitivo:



Para este caso, la obesidad depende del peso, pero podría depender de otras variables como genotipo, edad, entre otros. En conclusión, podemos tener varias entradas, continuas o categoricas.

Regresión Logística

- Concepto intuitivo:

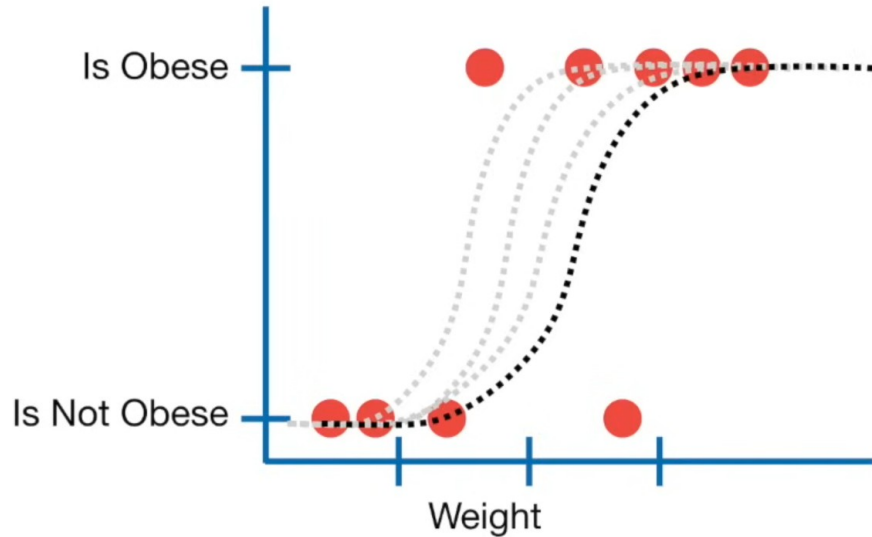


Para este caso, la obesidad depende del peso, pero podría depender de otras variables como genotipo, edad, entre otros. En conclusión, podemos tener varias entradas, continuas o categoricas.

Por otro lado, dado que no tiene sentido calcular residuos como en la regresión lineal, no es útil usar las metricas MAE y R^2 .

Regresión Logística

- Concepto intuitivo:



Para determinar la curva “S” idonea, se hace lo siguiente:

Por cada curva “S” propuesta se calcula la probabilidad de que el punto sea 0 o 1.

La curva que mas logre acertar clasificaciones (0 o 1) se escoge como la curva que mejor predice el fenomenos de estudio.

Regresión Logística

- Función de costo:

- Para una sola muestra: $Cost(h_{\theta}(x), y) = \begin{cases} -\log(h_{\theta}(x)) & \text{si } y = 1 \\ -\log(1 - h_{\theta}(x)) & \text{si } y = 0 \end{cases}$

Regresión Logística

- Función de costo:

- Para una sola muestra: $Cost(h_{\theta}(x), y) = \begin{cases} -\log(h_{\theta}(x)) & \text{si } y = 1 \\ -\log(1 - h_{\theta}(x)) & \text{si } y = 0 \end{cases}$

- Si la respuesta real es $y=1$:
 - Si el modelo predice 1 ($h_{\theta}(x)=1$), el costo es 0.
 - A medida que la predicción baja hacia 0, el costo aumenta exponencialmente hacia el infinito. El modelo recibe un "castigo" enorme por estar tan equivocado.

Regresión Logística

- Función de costo:

- Para una sola muestra: $Cost(h_{\theta}(x), y) = \begin{cases} -\log(h_{\theta}(x)) & \text{si } y = 1 \\ -\log(1 - h_{\theta}(x)) & \text{si } y = 0 \end{cases}$

- Si la respuesta real es $y=1$:
 - Si el modelo predice 1 ($h_{\theta}(x)=1$), el costo es 0.
 - A medida que la predicción baja hacia 0, el costo aumenta exponencialmente hacia el infinito. El modelo recibe un "castigo" enorme por estar tan equivocado.
 - Si la respuesta real es $y=0$:
 - Si el modelo predice 0 ($h_{\theta}(x)=0$), el costo es 0.
 - Si el modelo predice 1, el costo se dispara al infinito.

Regresión Logística

- Función de costo:
 - Para todas las muestras (todo el conjunto de datos):

$$Cost(h_{\theta}(x), y) = \begin{cases} -\log(h_{\theta}(x)) & \text{si } y = 1 \\ -\log(1 - h_{\theta}(x)) & \text{si } y = 0 \end{cases}$$



$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(h_{\theta}(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)}))]$$

Regresión Logística

- Función de costo:
 - Para todas las muestras (todo el conjunto de datos):

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(h_{\theta}(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)}))]$$

Si la respuesta real de la primera muestra es 1 $\rightarrow y^1 = 1$ y el modelo Predice ≈ 1 ($h_{\theta}(x) \approx 1$)

$$1 * \log(0.99) \\ 0.00XX$$

$$+ (1-1) * (...) \\ + 0 * (...) = 0.00XX + \dots$$

Si la respuesta real de la segunda muestra es 0 $\rightarrow y^2 = 0$ y el modelo Predice ≈ 0 ($h_{\theta}(x) \approx 0$)

$$0 * (...) \\ 0$$

$$+ (1-0) * \log(1-0,0XX) \\ + 1 * (0,00XX) = 0.00XX + \dots$$

Regresión Logística

- Minimización de la función de costo:
 - Se realiza mediante el Gradiente Descendente.

$$f(z) = \frac{1}{1 + e^{-z}} \quad z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3 + \cdots + \beta_n x_n$$

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(h_{\theta}(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)}))]$$

- Recuerde que la $f(z)$ es la misma $h_{\theta}(x)$

Regresión Logística

- Métricas de un modelo de clasificación:
 - Ejemplo: Suponga que tiene 100 correos: 10 son Spam (clase positiva) y 90 son correos Deseados (clase negativa).
 - **Matriz de confusión** → supongamos los siguientes resultados:

	Predicho: Spam	Predicho: Deseado
Real: Spam	4 (Verdaderos Positivos)	6 (Falsos Negativos)
Real: Deseado	1 (Falso Positivo)	89 (Verdaderos Negativos)

VP: 4
VN: 89
FP: 1
FN: 6

Regresión Logística

- Métricas de un modelo de clasificación:
 - Ejemplo: Suponga que tiene 100 correos: 10 son Spam (clase positiva) y 90 son correos Deseados (clase negativa).
 - **Accuracy:** proporción de predicciones correctas entre todos los casos.

$$Accuracy = \frac{VP + VN}{VP + VN + FP + FN}$$

$$Accuracy = (4 + 89)/(100) = 93\%$$

Nota: El Accuracy puede ser engañoso. Por ejemplo: se dejaron pasar 6 correos de spam.

	Predicho: Spam	Predicho: Deseado
Real: Spam	4 (Verdaderos Positivos)	6 (Falsos Negativos)
Real: Deseado	1 (Falso Positivo)	89 (Verdaderos Negativos)

VP: 4
VN: 89
FP: 1
FN: 6

Regresión Logística

- Métricas de un modelo de clasificación:
 - Ejemplo: Suponga que tiene 100 correos: 10 son Spam (clase positiva) y 90 son correos Deseados (clase negativa).
 - **Precision:** Mide la calidad de las predicciones positivas.

$$Precision = \frac{VP}{VP + FP}$$

$$Precision = (4)/(4+1) = 80\%$$

De cada 5 correos que el modelo mandó a la carpeta de Spam, uno era un correo importante.

	Predicho: Spam	Predicho: Deseado
Real: Spam	4 (Verdaderos Positivos)	6 (Falsos Negativos)
Real: Deseado	1 (Falso Positivo)	89 (Verdaderos Negativos)

VP: 4

VN: 89

FP: 1

FN: 6

Regresión Logística

- Métricas de un modelo de clasificación:
 - Ejemplo: Suponga que tiene 100 correos: 10 son Spam (clase positiva) y 90 son correos Deseados (clase negativa).
 - **Recall / Sensitivity (Exhaustividad):** Mide la capacidad del modelo para encontrar todos los casos positivos que existen en la realidad.

$$Recall = \frac{VP}{VP + FN}$$

$$Recall = (4)/(4+6) = 40\%$$

¿Qué porcentaje del Spam total logramos capturar?
Rta. Al modelo se le escapó el 60% del Spam real.

	Predicho: Spam	Predicho: Deseado
Real: Spam	4 (Verdaderos Positivos)	6 (Falsos Negativos)
Real: Deseado	1 (Falso Positivo)	89 (Verdaderos Negativos)

VP: 4

VN: 89

FP: 1

FN: 6

Regresión Logística

- Métricas de un modelo de clasificación:
 - Ejemplo: Suponga que tiene 100 correos: 10 son Spam (clase positiva) y 90 son correos Deseados (clase negativa).
 - **Recall / Sensitivity (Exhaustividad):** Es la media armónica entre Precision y Recall. Se usa la media armónica porque penaliza mucho si uno de los dos valores es muy bajo.

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

$$F1 = 2 \cdot (0,8 \cdot 0,4) / (0,8 + 0,4) = 0,53\%$$

El F1-Score (0.53) refleja mucho mejor que el modelo es mediocre, comparado con el Accuracy (0.93).

	Predicho: Spam	Predicho: Deseado
Real: Spam	4 (Verdaderos Positivos)	6 (Falsos Negativos)
Real: Deseado	1 (Falso Positivo)	89 (Verdaderos Negativos)

VP: 4

VN: 89

FP: 1

FN: 6

Regresión Logística

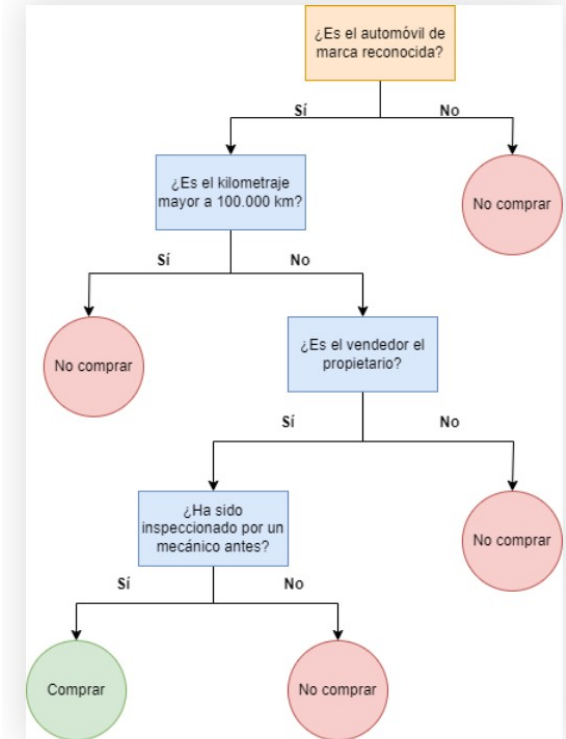
- Implementación:
 - Revisar el archivo `ej_regresiónLogística.ipynb`.

Árboles de Decisión

- Definición: modelo de aprendizaje supervisado que funciona como un mapa de decisiones para predecir el valor de una variable objetivo.
- Son utilizados tanto para problemas de clasificación (predicción de etiquetas categóricas) como de regresión (predicción de valores numéricos continuos).
- Su funcionamiento se basa en la partición binaria recursiva del espacio de características.

Árboles de Decisión

- Estructura y Componentes:
 - **Nodo Raíz (Root Node):** El punto de inicio que contiene la totalidad de la muestra y realiza la primera partición basada en el atributo más discriminante.
 - **Nodos de Decisión (Internal Nodes):** Representan pruebas lógicas sobre una característica específica.
 - **Ramas (Edges):** Representan el resultado de la condición evaluada, dirigiendo el flujo hacia el siguiente nodo.
 - **Nodos Hoja (Leaf Nodes):** Nodos terminales que contienen la predicción final



Árboles de Decisión

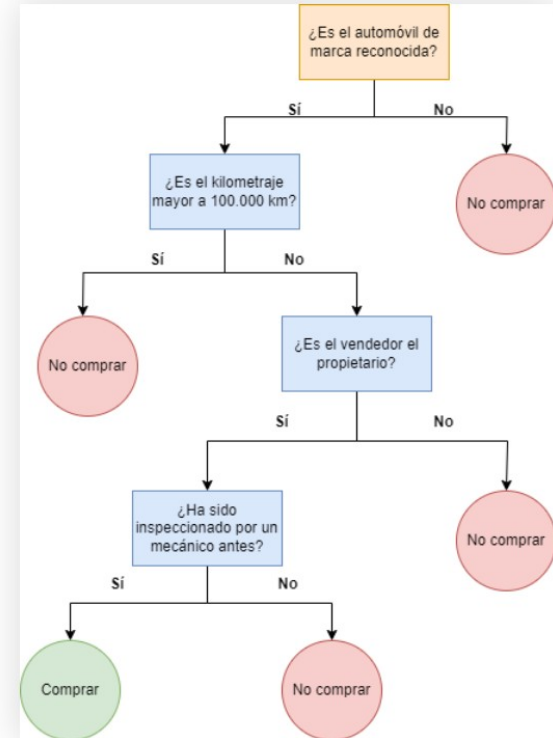
- **Nota importante:** En el contexto de Machine Learning (ML), un árbol de decisión no solo sigue reglas lógicas manuales, sino que las descubre analizando un conjunto de datos histórico mediante procesos estadísticos.

Dataset

Marca	Distancia (km)	Propietario	Revisión previa	Comprar
Mazda	200	Carlos Gomez	No	No
Honda	150	Juan Perez	Si	Si
Subaru	300	Natalia Rodriguez	Si	No
..	..	..	..	..



El modelo descubre el árbol de decisión



Árboles de Decisión

- Usar un solo árbol para predecir es desventajoso debido a que:
 - Un solo árbol es extremadamente sensible a los datos de entrenamiento. Si se cambia apenas un par de filas, la estructura del árbol puede cambiar por completo.
 - Un solo árbol probablemente tendría alta varianza. Se aprendería el ruido de los datos como si fueran reglas reales.
 - Ejemplo: "Si el usuario tiene 22 años, vive en Medellín y su nombre empieza con G, entonces compra").

Árboles de Decisión

- Usar un solo árbol para predecir es desventajoso debido a que:
 - En un solo árbol, si la raíz toma una mala decisión inicial, todo el resto de la predicción está condenada.
 - **Ejemplo:** Supongamos que estamos construyendo un modelo para predecir si un crédito bancario será pagado o no. Tenemos dos variables principales: **Ingresos Mensuales** y **Historial Crediticio**.
 - Supongamos que, por un azar en los datos de entrenamiento (o un sesgo), el algoritmo elige como Nodo Raíz la variable "Tiene Vehículo Propio", simplemente porque en ese conjunto de datos específico coincidió que los que tenían carro pagaron.
 - El árbol pregunta primero: ¿Tiene carro? Entonces puede que el cliente sea excelente pero como no tiene carro, será descartado.

Árboles de Decisión

- La idea es tener un consenso grupal (varios árboles) para evitar el sobreajuste.
 - Solución: usar varios árboles → **Random Forest**
 - No se dependerá de un solo árbol caprichoso.
 - Para cada árbol se seleccionarán características aleatorias.
 - Los nodos raíces pueden variar.
 - Habrá mayoría:
 - El Árbol 3 dirá: "Riesgo" (porque se equivocó en la raíz).
 - El Árbol 1 dirá: "Seguro" (porque vio sus altos ingresos).
 - El Árbol 2 dirá: "Seguro" (porque vio su buen historial).
 - Resultado final: La votación será 2 a 1 a favor de "Seguro". El error del Árbol 3 será diluido por la sabiduría del resto del bosque.

Árboles de Decisión

- **Random Forest:**

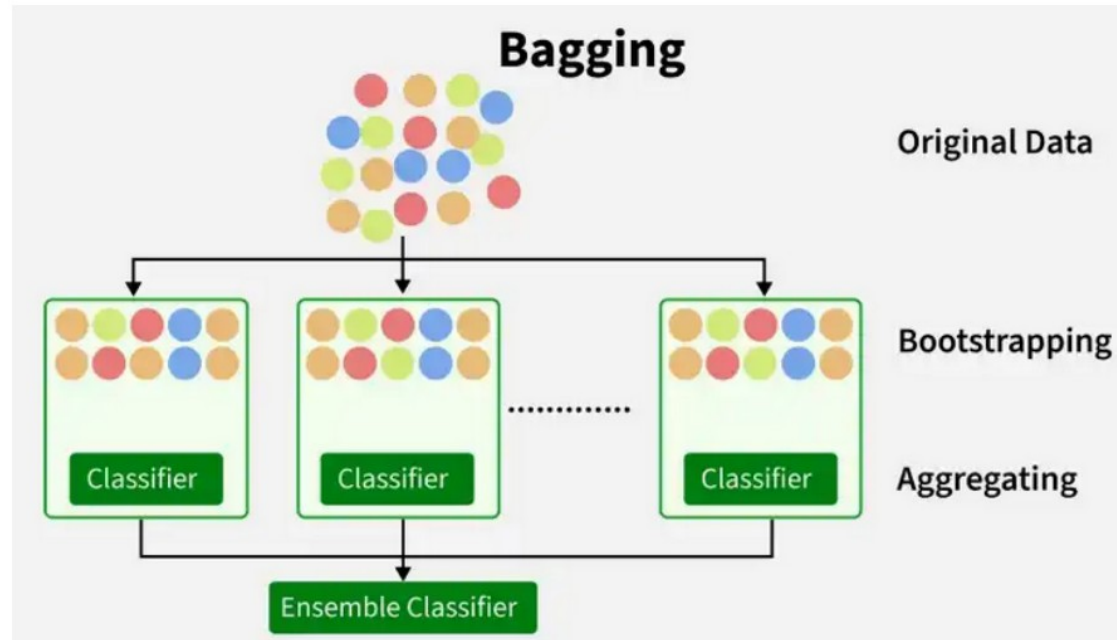
- Usa la sabiduría de muchos árboles para lograr consenso y que así el resultado sea mas robusto.
- Aplica por defecto **Bagging** (Bootstrap Aggregating)

Árboles de Decisión

- **Random Forest:**
 - Aplica por defecto **Bagging** (Bootstrap Aggregating)

Árboles de Decisión

- **Random Forest:**
 - Aplica por defecto **Bagging** (Bootstrap Aggregating)



Árboles de Decisión

- Random Forest:
 - Aplica por defecto Bagging (Bootstrap Aggregating)
 - **Bootstrapping** (Muestreo con reemplazo): En lugar de entrenar todos los árboles con el mismo conjunto de datos exacto, el algoritmo crea múltiples subconjuntos aleatorios.
 - Si tenemos un dataset de 100 filas, para el Árbol #1 el algoritmo elige 100 filas al azar, pero con reemplazo. Esto significa que una misma fila puede aparecer 2 o 3 veces en la muestra del Árbol #1, mientras que otras filas no aparecerán en absoluto.
 - Analogía: es como tener una bolsa de 100 filas y sacamos de a una fila, volviendola a incluir para sacar la siguiente hasta completar las 100 extracciones.

Árboles de Decisión

- Random Forest:
 - Aplica por defecto Bagging (Bootstrap Aggregating)
 - **Bootstrapping** (Muestreo con reemplazo): En lugar de entrenar todos los árboles con el mismo conjunto de datos exacto, el algoritmo crea múltiples subconjuntos aleatorios.
 - Si tenemos un dataset de 100 filas, para el Árbol #1 el algoritmo elige 100 filas al azar, pero con reemplazo. Esto significa que una misma fila puede aparecer 2 o 3 veces en la muestra del Árbol #1, mientras que otras filas no aparecerán en absoluto.
 - En promedio, cada árbol deja fuera un 36.8% de los datos originales (llamados datos Out-of-Bag o OOB), los cuales sirven después para validar el modelo sin necesidad de un set de pruebas externo.

Árboles de Decisión

- Random Forest:
 - Aplica por defecto Bagging (Bootstrap Aggregating)
 - Ejemplo:

Fila	Árbol 1	Árbol 2	Árbol 3	Árbol 4	Árbol 5	¿Quiénes evalúan esta fila?
Fila 1	In-Bag	OOB	In-Bag	OOB	OOB	Árboles 2, 4 y 5
Fila 2	OOB	In-Bag	OOB	In-Bag	In-Bag	Árboles 1 y 3
Fila 3	In-Bag	In-Bag	OOB	OOB	In-Bag	Árboles 3 y 4
Fila 4	OOB	OOB	In-Bag	In-Bag	OOB	Árboles 1, 2 y 5

Árboles de Decisión

- Random Forest:
 - Aplica por defecto Bagging (Bootstrap Aggregating)
 - **Aggregating** (Agregación de resultados)
 - Una vez que tenemos, por ejemplo, 100 árboles entrenados con sus respectivas muestras Bootstrap, llega el momento de predecir. El Bagging combina sus respuestas para obtener un resultado final más robusto:

Árboles de Decisión

- Random Forest:
 - Aplica por defecto Bagging (Bootstrap Aggregating)
 - **Aggregating** (Agregación de resultados)
 - Una vez que tenemos, por ejemplo, 100 árboles entrenados con sus respectivas muestras Bootstrap, llega el momento de predecir. El Bagging combina sus respuestas para obtener un resultado final más robusto:
 - **Para Clasificación:** Se utiliza la **Votación Mayoritaria**. Si 70 árboles dicen "Clase A" y 30 dicen "Clase B", el resultado final es "Clase A".

Árboles de Decisión

- Random Forest:
 - Aplica por defecto Bagging (Bootstrap Aggregating)
 - **Aggregating** (Agregación de resultados)
 - Una vez que tenemos, por ejemplo, 100 árboles entrenados con sus respectivas muestras Bootstrap, llega el momento de predecir. El Bagging combina sus respuestas para obtener un resultado final más robusto:
 - **Para Clasificación:** Se utiliza la Votación Mayoritaria. Si 70 árboles dicen "Clase A" y 30 dicen "Clase B", el resultado final es "Clase A".
 - **Para Regresión:** Se calcula el Promedio de las predicciones de todos los árboles.

Árboles de Decisión

- Random Forest:
 - Aplica por defecto Bagging (Bootstrap Aggregating)
 - **Beneficios:**
 - **Promediamos el error:** Los errores aleatorios de un árbol individual (que quizás se enfocó demasiado en un outlier que le tocó en su muestra Bootstrap) se cancelan con los aciertos de los demás árboles.
 - **Estabilidad:** El modelo final es mucho menos sensible a cambios en los datos de entrenamiento que un solo árbol.

Árboles de Decisión

- Construcción de un árbol de decisión:

Supongamos que tenemos este dataset...

Loves Popcorn	Loves Soda	Age	Loves Cool As Ice
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No

Árboles de Decisión

- Construcción de un árbol de decisión:

Loves Popcorn	Loves Soda	Age	Loves Cool As Ice
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No

Supongamos que tenemos este dataset...

“Loves popcorn”, “Loves soda” y “Age” → características del dataset

“Loves Cool as Ice” → variable objetivo

Árboles de Decisión

- Construcción de un árbol de decisión:

Loves Popcorn	Loves Soda	Age	Loves Cool As Ice
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No

Supongamos que tenemos este dataset...

“Loves popcorn”, “Loves soda” y “Age” → características del dataset

“Loves Cool as Ice” → variable objetivo

Objetivo: encontrar un árbol que prediga si a una persona le gustará la película “Cool as Ice” dadas las anteriores características

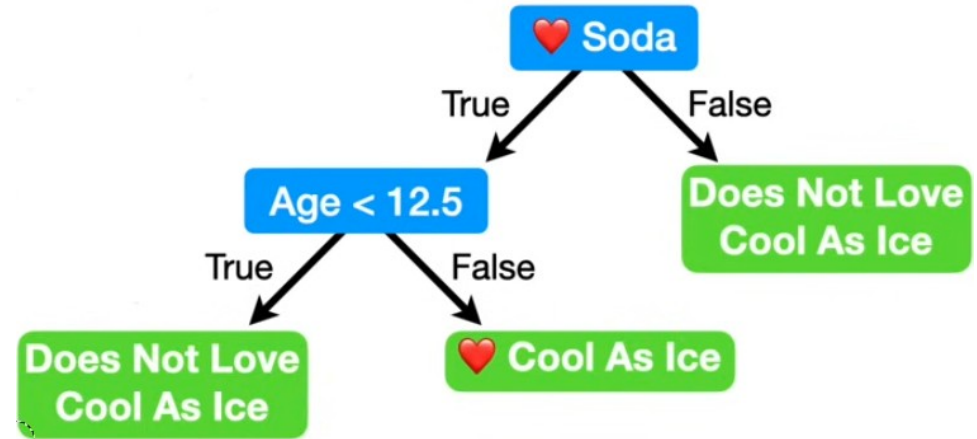
Árboles de Decisión

- Construcción de un árbol de decisión:

Loves Popcorn	Loves Soda	Age	Loves Cool As Ice
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No

Objetivo: encontrar un árbol que prediga si a una persona le gustará la película “Cool as Ice” dadas las anteriores características

Por ejemplo:



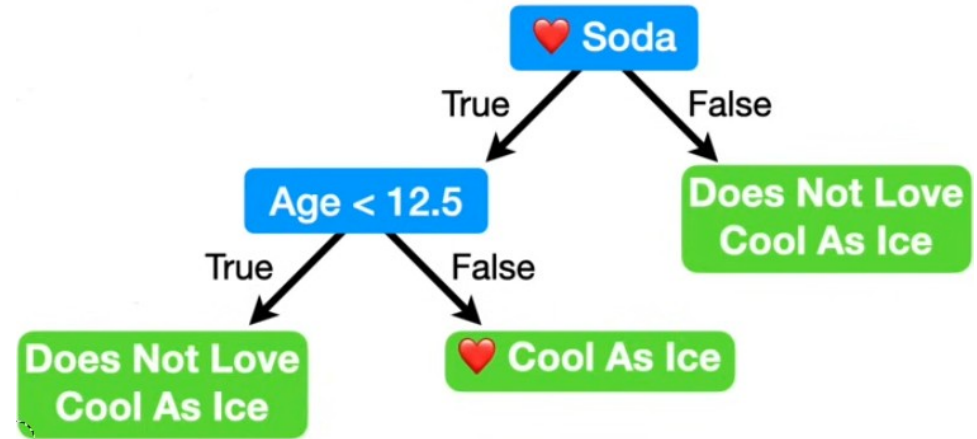
Árboles de Decisión

- Construcción de un árbol de decisión:

Loves Popcorn	Loves Soda	Age	Loves Cool As Ice
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No

Objetivo: encontrar un árbol que prediga si a una persona le gustará la película “Cool as Ice” dadas las anteriores características

Por ejemplo:

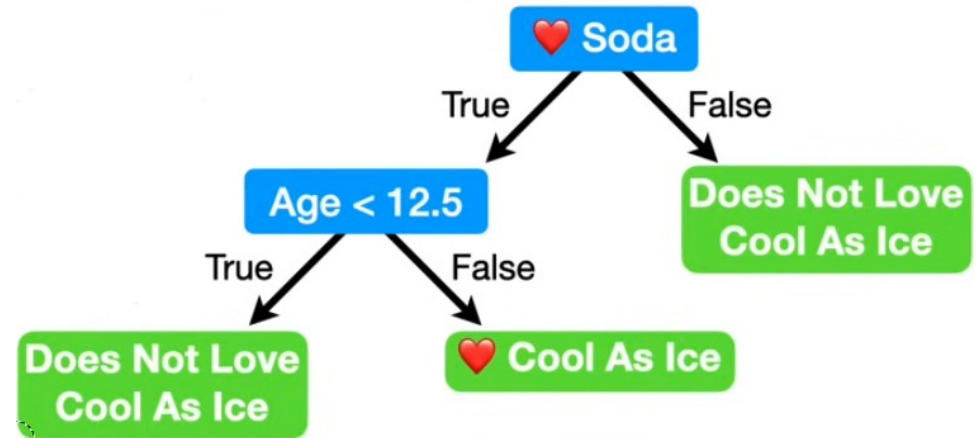


Nodos azules → características del dataset
Nodos verdes → variable objetivo

Árboles de Decisión

- Construcción de un árbol de decisión:

Loves Popcorn	Loves Soda	Age	Loves Cool As Ice
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No



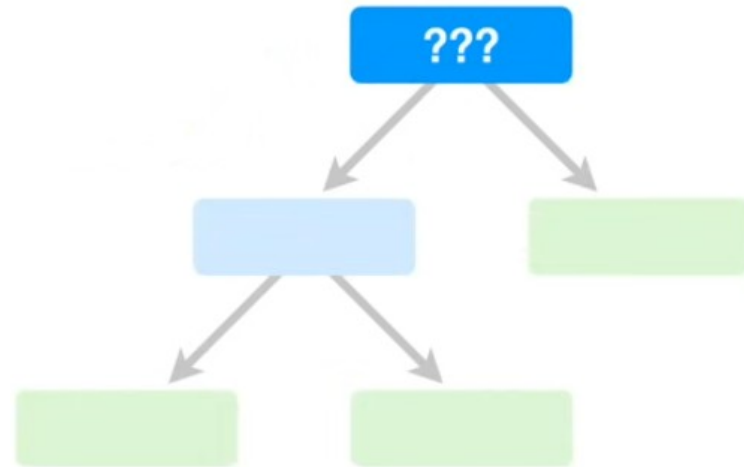
Recordemos: este árbol NO lo tenemos...la idea es encontrarlo a partir del dataset!

Árboles de Decisión

- Construcción de un árbol de decisión:

Loves Popcorn	Loves Soda	Age	Loves Cool As Ice
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No

Paso 1: determinar cual característica será nuestro nodo raíz



Árboles de Decisión

- Construcción de un árbol de decisión:

Loves Popcorn	Loves Soda	Age	Loves Cool As Ice
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No

Paso 1: determinar cual característica será nuestro nodo raíz

Evaluemos la primera característica → “Loves popcorn”

Es decir, que tanto predice “Loves popcorn” que a alguien le guste “Cool as Ice”

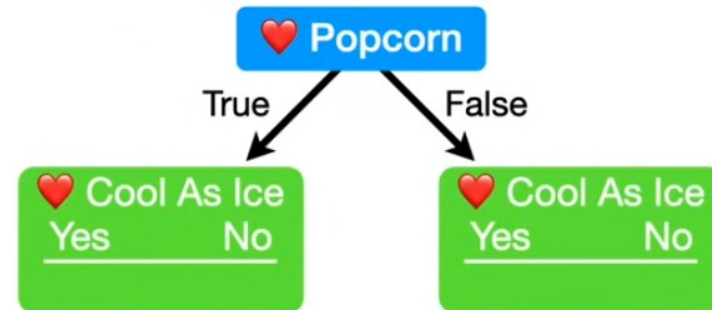
Árboles de Decisión

- Construcción de un árbol de decisión:

Loves Popcorn	Loves Soda	Age	Loves Cool As Ice
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No

Paso 1: determinar cual característica será nuestro nodo raíz

Evaluemos la primera característica → “Loves popcorn”



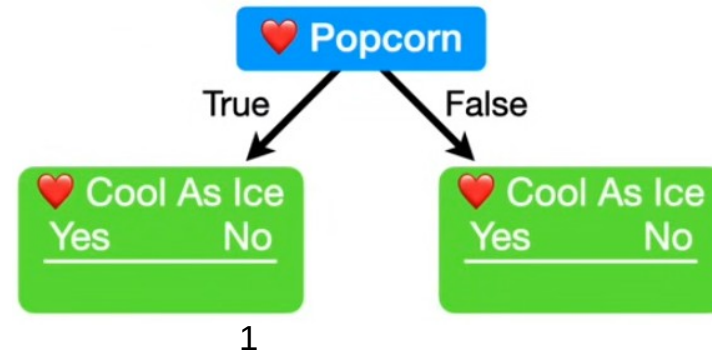
Árboles de Decisión

- Construcción de un árbol de decisión:

Loves Popcorn	Loves Soda	Age	Loves Cool As Ice
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No

Paso 1: determinar cual característica será nuestro nodo raíz

Evaluemos la primera característica → “Loves popcorn”



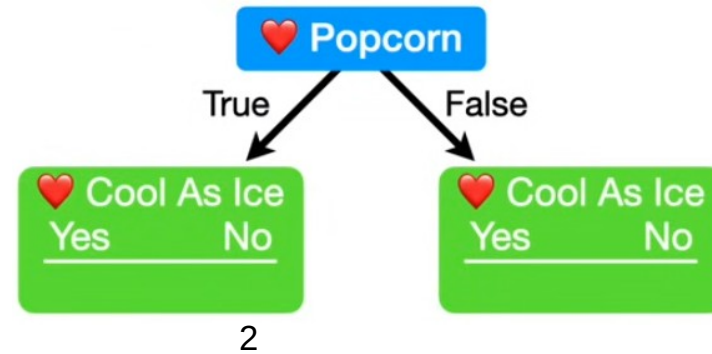
Árboles de Decisión

- Construcción de un árbol de decisión:

Loves Popcorn	Loves Soda	Age	Loves Cool As Ice
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No

Paso 1: determinar cual característica será nuestro nodo raíz

Evaluemos la primera característica → “Loves popcorn”



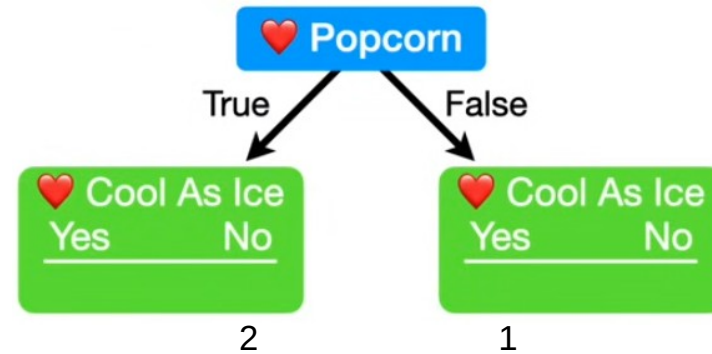
Árboles de Decisión

- Construcción de un árbol de decisión:

Loves Popcorn	Loves Soda	Age	Loves Cool As Ice
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No

Paso 1: determinar cual característica será nuestro nodo raíz

Evaluemos la primera característica → “Loves popcorn”



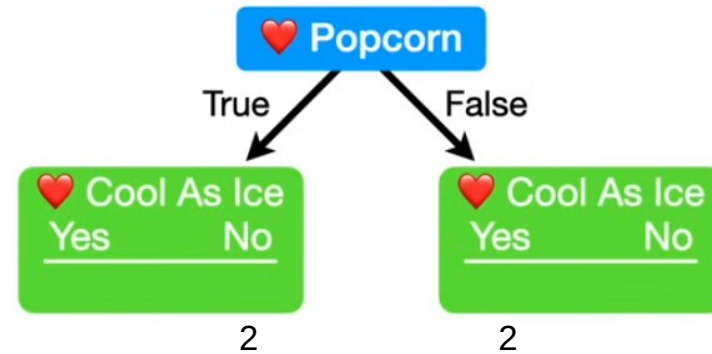
Árboles de Decisión

- Construcción de un árbol de decisión:

Loves Popcorn	Loves Soda	Age	Loves Cool As Ice
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No

Paso 1: determinar cual característica será nuestro nodo raíz

Evaluemos la primera característica → “Loves popcorn”



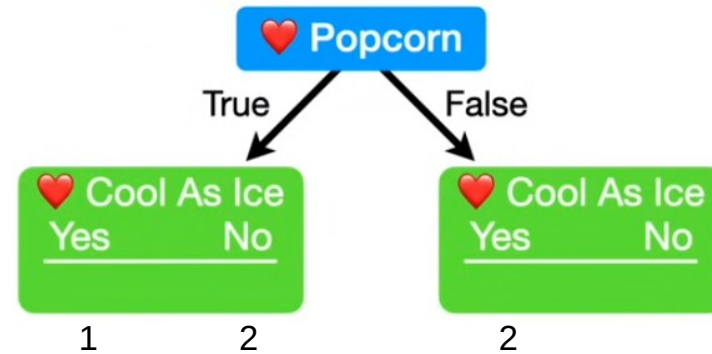
Árboles de Decisión

- Construcción de un árbol de decisión:

Loves Popcorn	Loves Soda	Age	Loves Cool As Ice
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No

Paso 1: determinar cual característica será nuestro nodo raíz

Evaluemos la primera característica → “Loves popcorn”



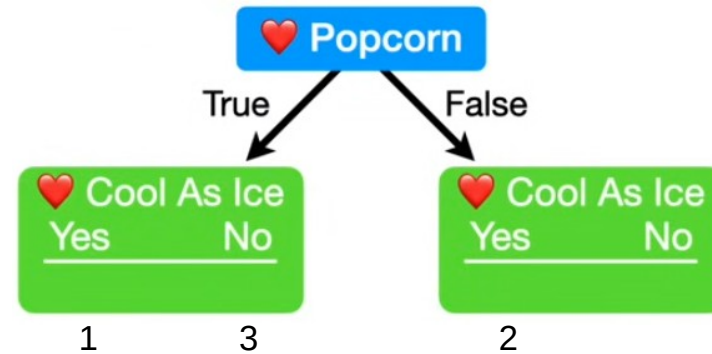
Árboles de Decisión

- Construcción de un árbol de decisión:

Loves Popcorn	Loves Soda	Age	Loves Cool As Ice
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No

Paso 1: determinar cual característica será nuestro nodo raíz

Evaluemos la primera característica → “Loves popcorn”



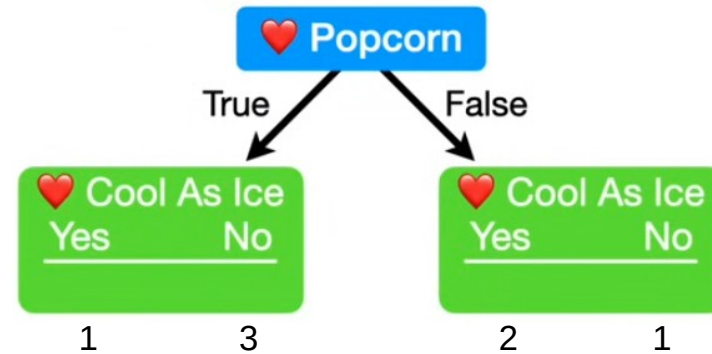
Árboles de Decisión

- Construcción de un árbol de decisión:

Loves Popcorn	Loves Soda	Age	Loves Cool As Ice
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No

Paso 1: determinar cual característica será nuestro nodo raíz

Evaluemos la primera característica → “Loves popcorn”



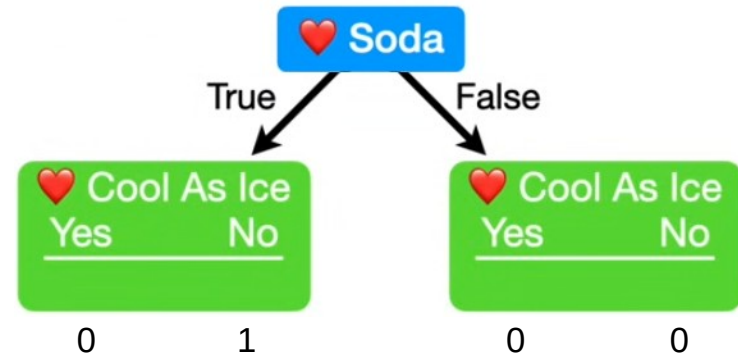
Árboles de Decisión

- Construcción de un árbol de decisión:

Loves Popcorn	Loves Soda	Age	Loves Cool As Ice
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No

Paso 1: determinar cual característica será nuestro nodo raíz

Evaluemos la segunda característica → “Loves popcorn”



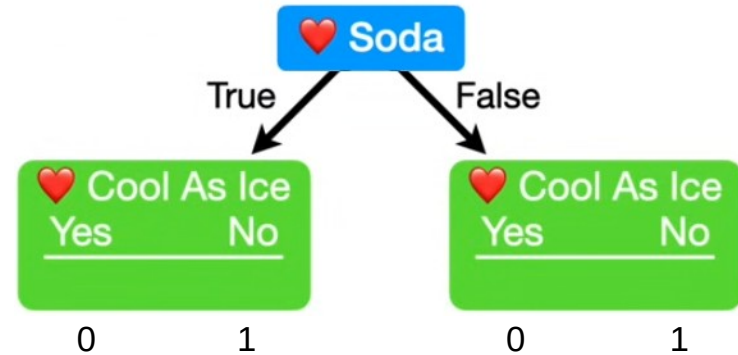
Árboles de Decisión

- Construcción de un árbol de decisión:

Loves Popcorn	Loves Soda	Age	Loves Cool As Ice
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No

Paso 1: determinar cual característica será nuestro nodo raíz

Evaluemos la segunda característica → “Loves popcorn”



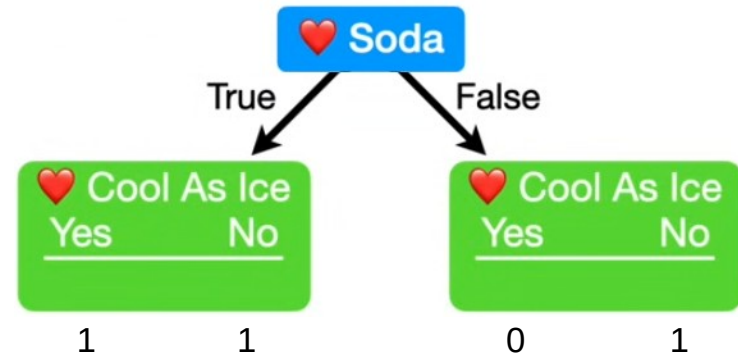
Árboles de Decisión

- Construcción de un árbol de decisión:

Loves Popcorn	Loves Soda	Age	Loves Cool As Ice
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No

Paso 1: determinar cual característica será nuestro nodo raíz

Evaluemos la segunda característica → “Loves popcorn”



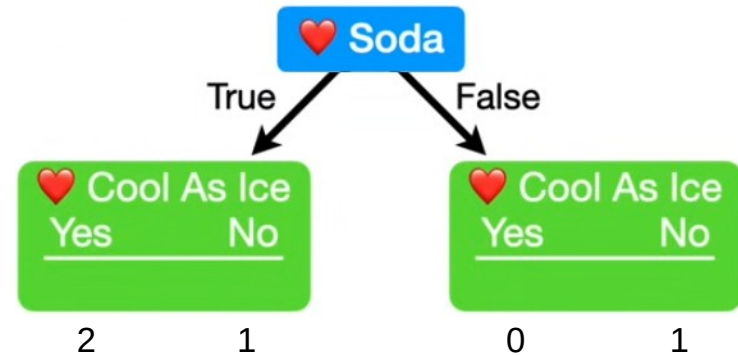
Árboles de Decisión

- Construcción de un árbol de decisión:

Loves Popcorn	Loves Soda	Age	Loves Cool As Ice
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No

Paso 1: determinar cual característica será nuestro nodo raíz

Evaluemos la segunda característica → “Loves popcorn”



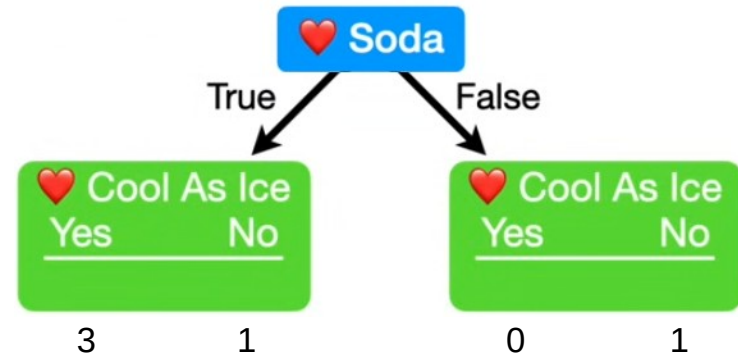
Árboles de Decisión

- Construcción de un árbol de decisión:

Loves Popcorn	Loves Soda	Age	Loves Cool As Ice
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No

Paso 1: determinar cual característica será nuestro nodo raíz

Evaluemos la segunda característica → “Loves popcorn”



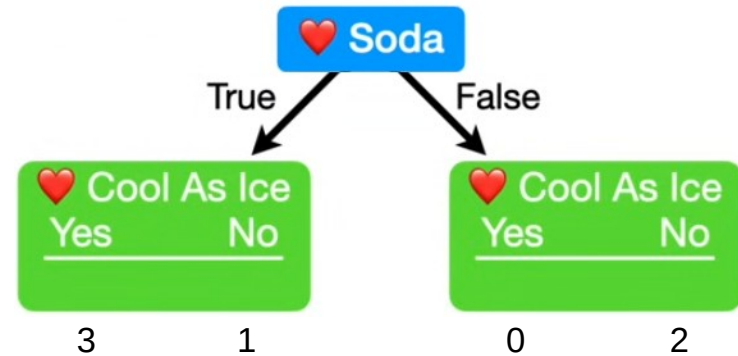
Árboles de Decisión

- Construcción de un árbol de decisión:

Loves Popcorn	Loves Soda	Age	Loves Cool As Ice
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No

Paso 1: determinar cual característica será nuestro nodo raíz

Evaluemos la segunda característica → “Loves popcorn”



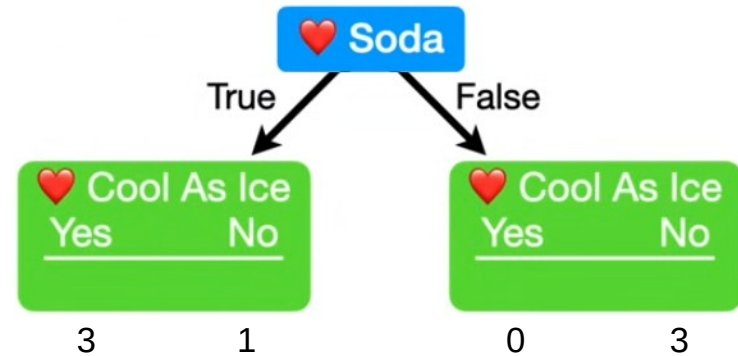
Árboles de Decisión

- Construcción de un árbol de decisión:

Loves Popcorn	Loves Soda	Age	Loves Cool As Ice
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No

Paso 1: determinar cual característica será nuestro nodo raíz

Evaluemos la segunda característica → “Loves popcorn”



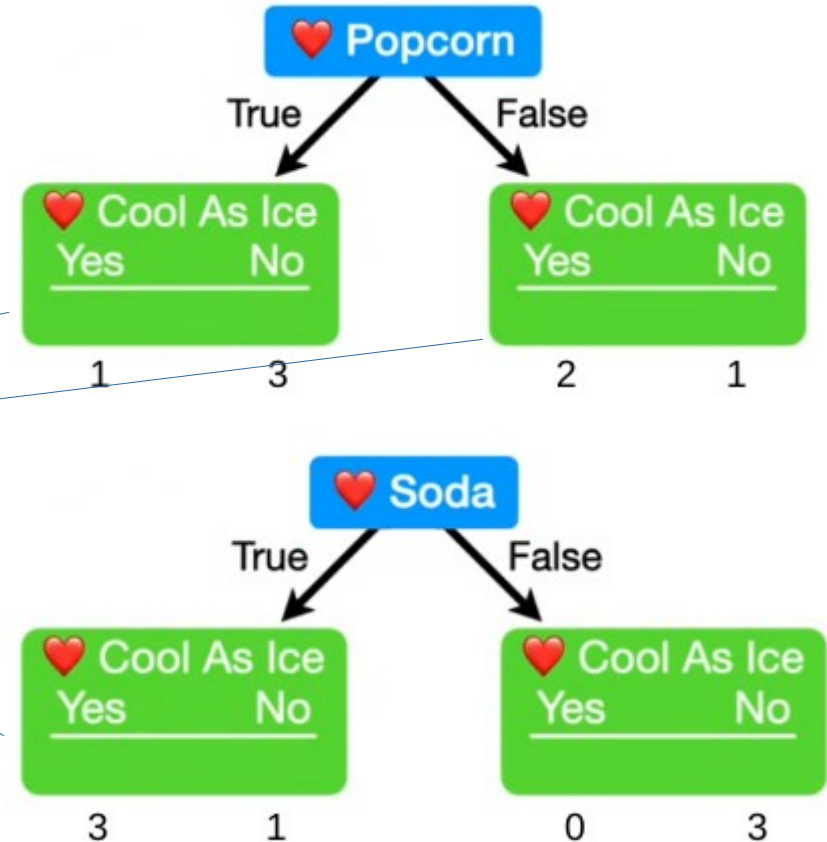
Árboles de Decisión

- Construcción de un árbol de decisión:

Paso 1: determinar cual característica será nuestro nodo raíz

Evaluemos la segunda característica → “Loves soda”

Estas tres hojas poseen mezclas de personas que les gusta y no les gusta “Cool as Ice”



Árboles de Decisión

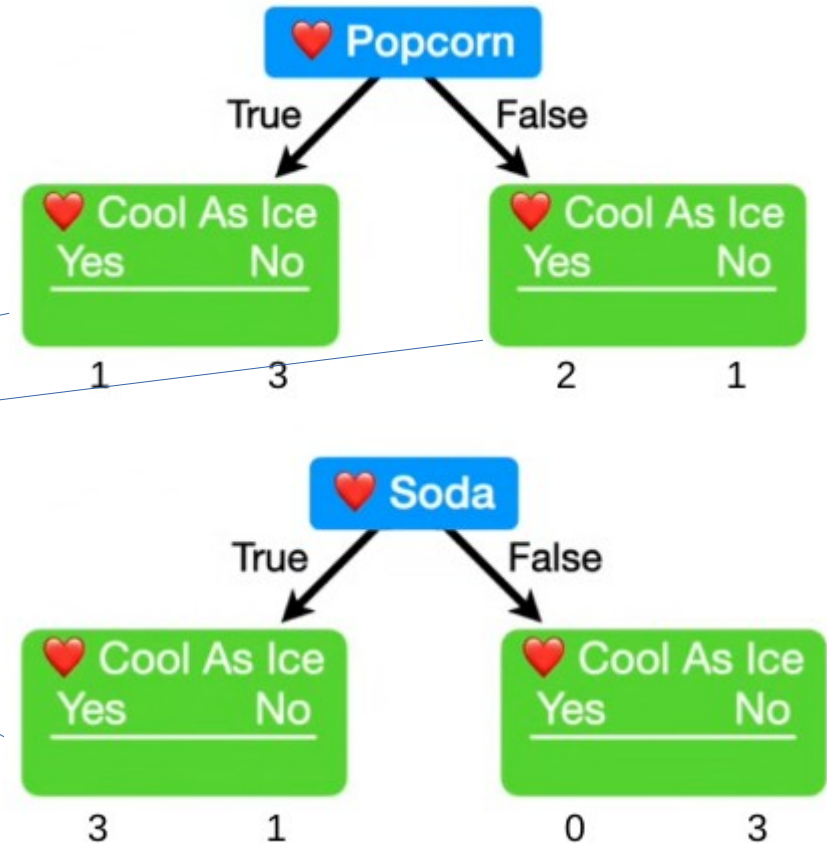
- Construcción de un árbol de decisión:

Paso 1: determinar cual característica será nuestro nodo raíz

Evaluemos la segunda característica → “Loves soda”

Estas tres hojas poseen mezclas de personas que les gusta y no les gusta “Cool as Ice”

↓
Hojas Impuras



Árboles de Decisión

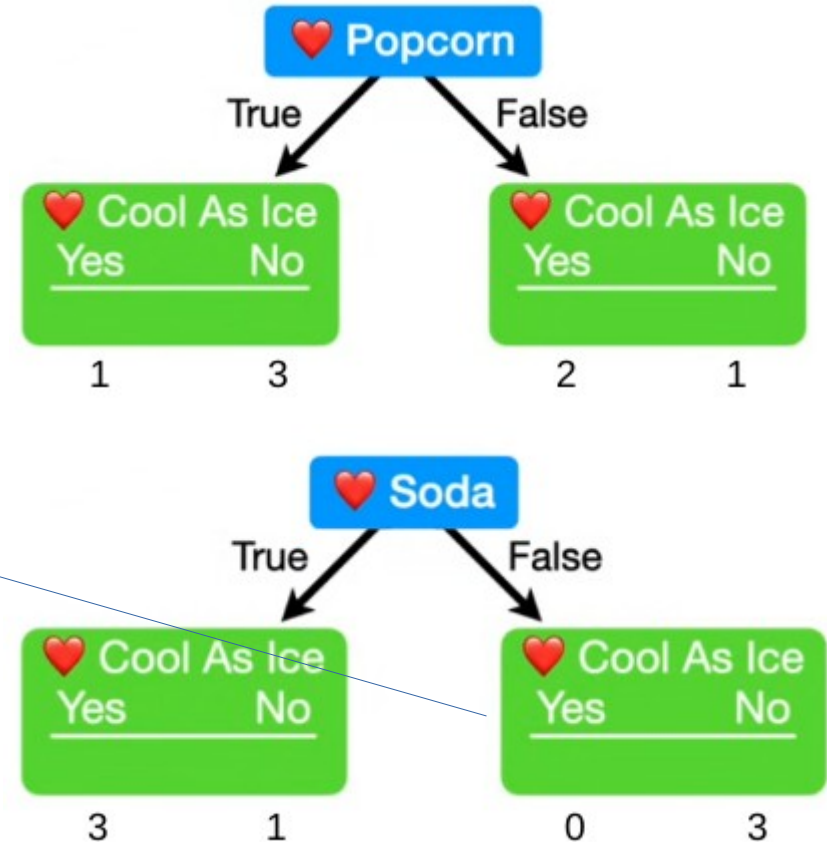
- Construcción de un árbol de decisión:

Paso 1: determinar cual característica será nuestro nodo raíz

Evaluemos la segunda característica → “Loves soda”

Esta hoja solo contiene personas que no les gusta “Cool as Ice”

Hoja Pura



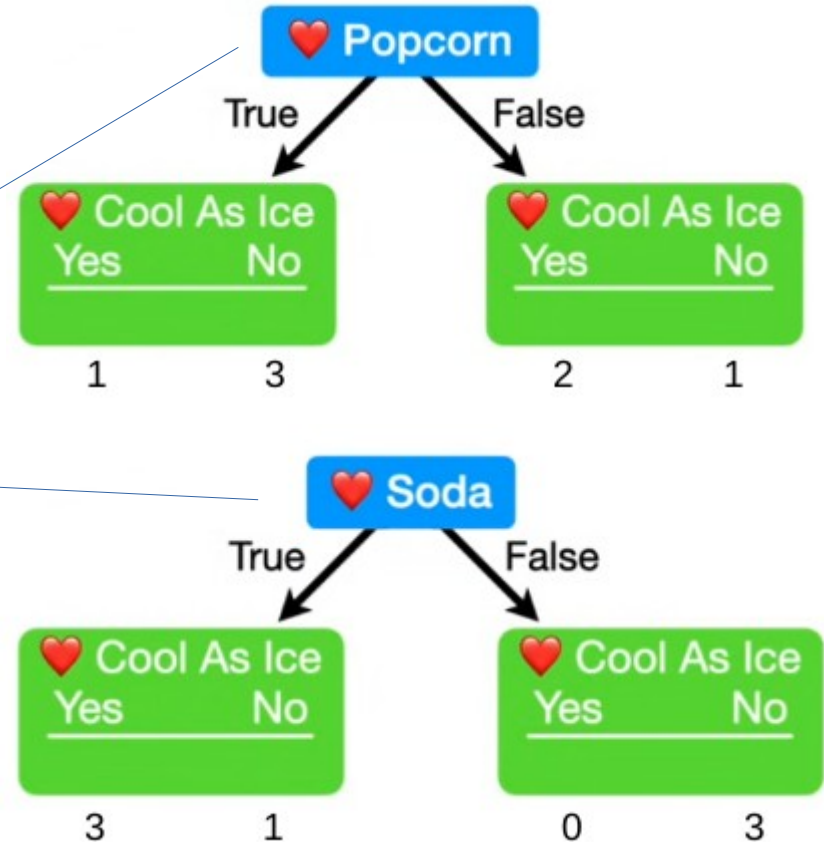
Árboles de Decisión

- Construcción de un árbol de decisión:

Paso 1: determinar cual característica será nuestro nodo raíz

Evaluemos la segunda característica → “Loves soda”

Entre estas dos raíces candidatas, “Loves Soda” es menos impura que “Loves popcorn”



Árboles de Decisión

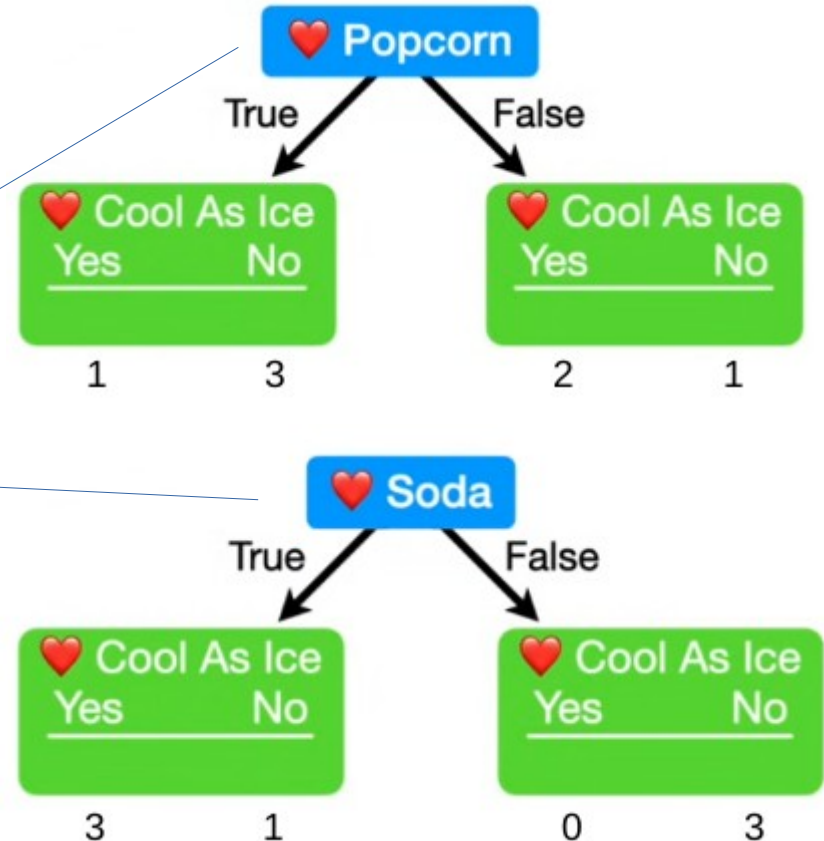
- Construcción de un árbol de decisión:

Paso 1: determinar cual característica será nuestro nodo raíz

Evaluemos la segunda característica → “Loves soda”

Entre estas dos raíces candidatas, “Loves Soda” es menos impura que “Loves popcorn”

“Loves soda” debería ser el nodo raíz



Árboles de Decisión

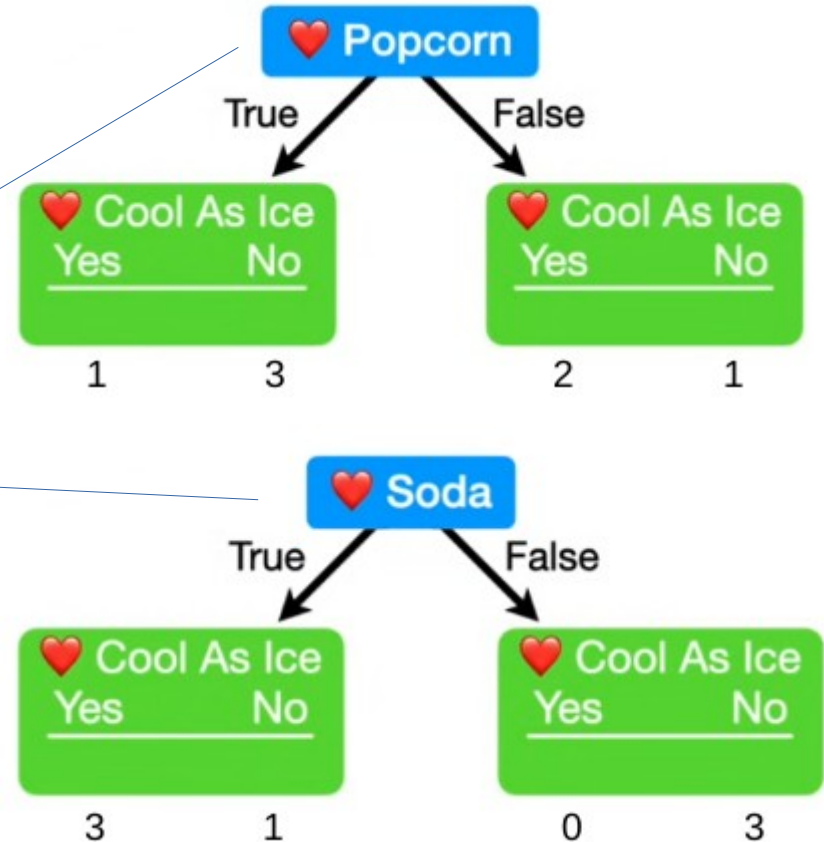
- Construcción de un árbol de decisión:

Paso 1: determinar cual característica será nuestro nodo raíz

Evaluemos la segunda característica → “Loves soda”

Entre estas dos raíces candidatas, “Loves Soda” es menos impura que “Loves popcorn”

Conclusión: entre menos impuro sea un nodo, mas chance tiene de ser nodo raíz.



Árboles de Decisión

- Construcción de un árbol de decisión:

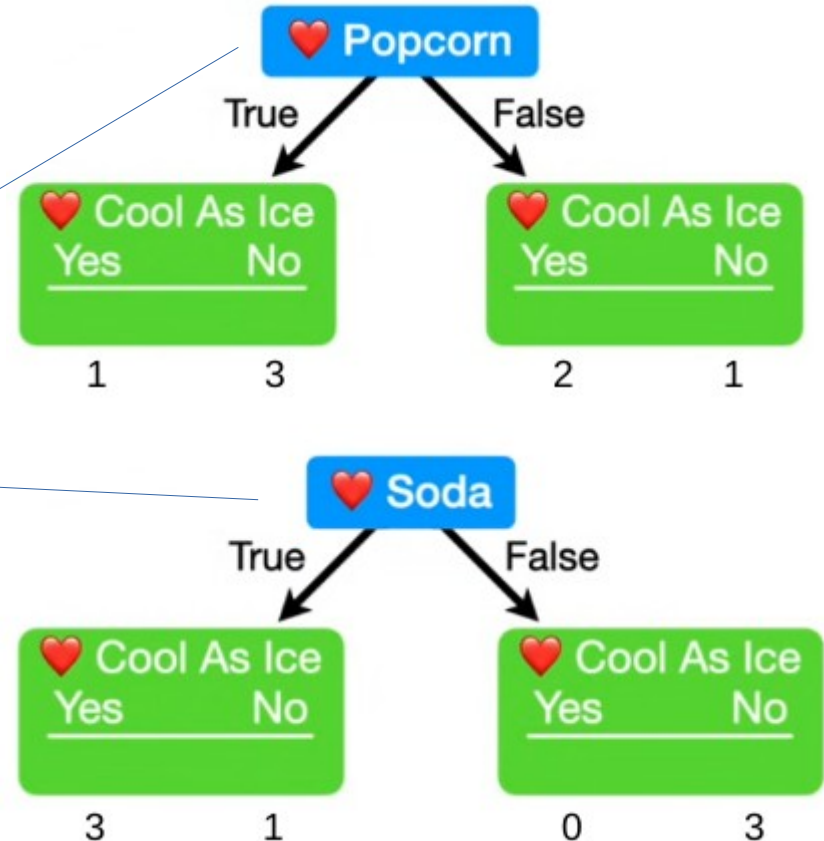
Paso 1: determinar cual característica será nuestro nodo raíz

Evaluemos la segunda característica → “Loves soda”

Entre estas dos raíces candidatas, “Loves Soda” es menos impura que “Loves popcorn”

Conclusión: entre menos impuro sea un nodo, mas chance tiene de ser nodo raíz.

Cómo cuantificamos la impureza?



Árboles de Decisión

- Construcción de un árbol de decisión:

Cuantificación de la impureza → **Impureza de Gini**

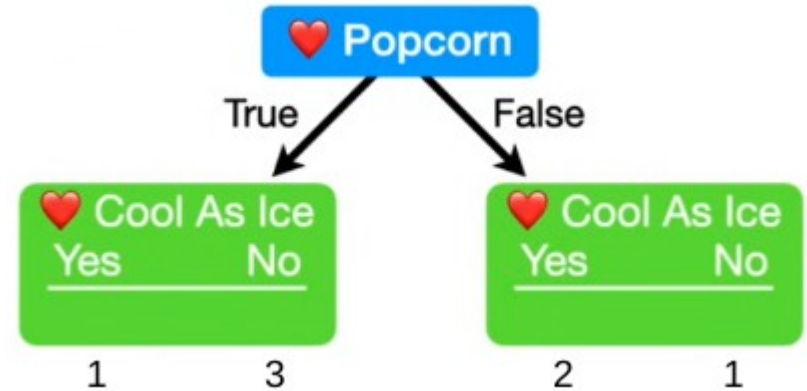
Para un nodo o una hoja m, la impureza de Gini (G) se calcula como:

$$G = 1 - \sum_{i=1}^J p_i^2$$

Donde:

J: Es el número total de clases.

p_i : Es la proporción (probabilidad) de elementos que pertenecen a la clase i en esa hoja específica.



Árboles de Decisión

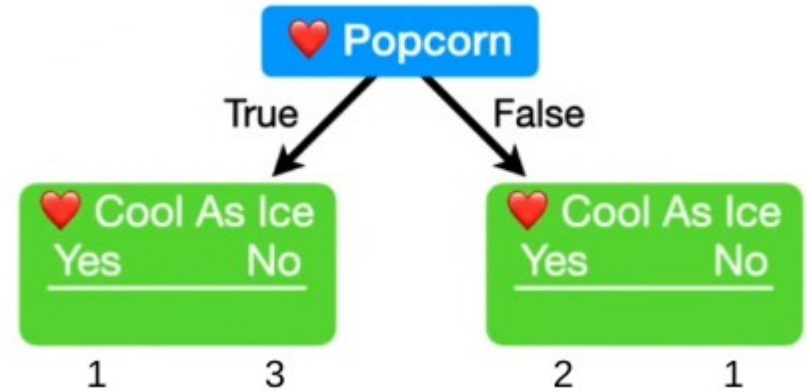
- Construcción de un árbol de decisión:

Cuantificación de la impureza → **Impureza de Gini**

Para un nodo o una hoja m, la impureza de Gini (G) se calcula como:

$$G = 1 - \sum_{i=1}^J p_i^2$$

Gini = 0 (Pureza Total): Todos los elementos de la hoja pertenecen a la misma clase. El modelo tiene una certeza total en ese punto.



Árboles de Decisión

- Construcción de un árbol de decisión:

Cuantificación de la impureza → **Impureza de Gini**

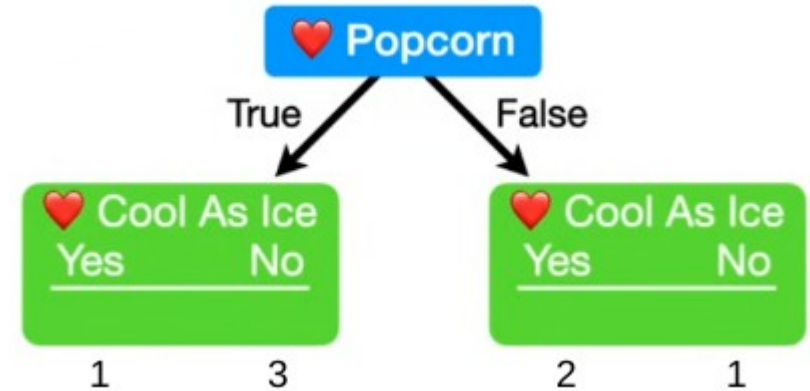
Para un nodo o una hoja m, la impureza de Gini (G) se calcula como:

$$G = 1 - \sum_{i=1}^J p_i^2$$

Gini = 0 (Pureza Total): Todos los elementos de la hoja pertenecen a la misma clase. El modelo tiene una certeza total en ese punto.

Gini máximo (Impureza Máxima): Los elementos están distribuidos uniformemente entre todas las clases. Por ejemplo, en un problema binario (clase A y B), si hay 50% de cada una, el Gini será:

$$1 - (0.5^2 + 0.5^2) = 1 - (0.25 + 0.25) = \mathbf{0.5}$$



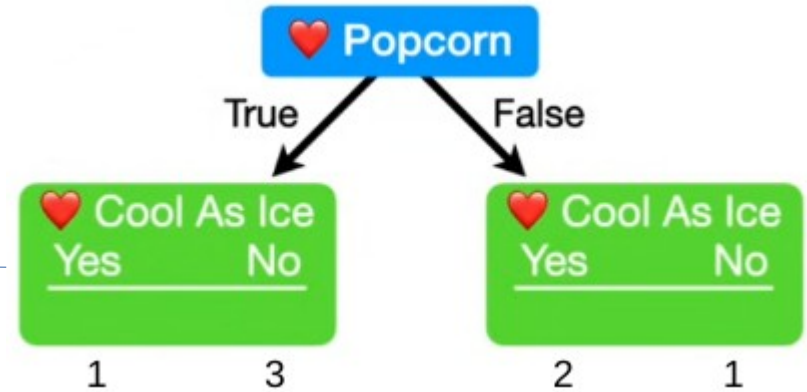
Árboles de Decisión

- Construcción de un árbol de decisión:

Cuantificación de la impureza → **Impureza de Gini**

Impureza de Gini = $1 - (\text{prob. Yes})^2 - (\text{prob. No})^2$

$$= 1 - (1/(1+3))^2 - (3/(1+3))^2$$



Árboles de Decisión

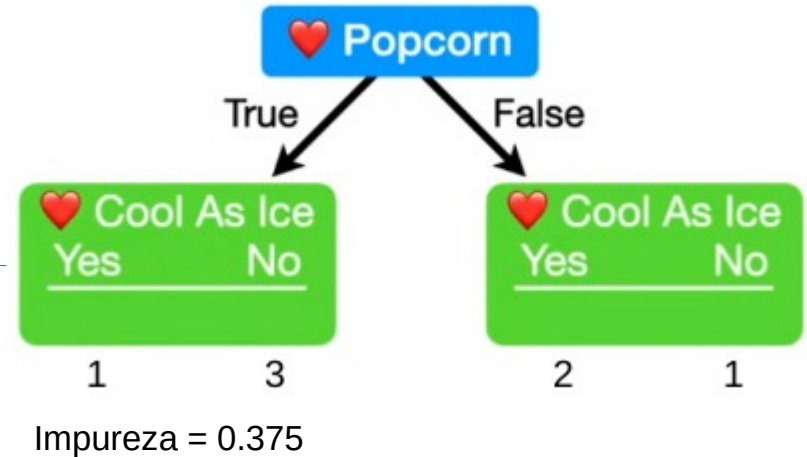
- Construcción de un árbol de decisión:

Cuantificación de la impureza → **Impureza de Gini**

Impureza de Gini = $1 - (\text{prob. Yes})^2 - (\text{prob. No})^2$

$$= 1 - (1/(1+3))^2 - (3/(1+3))^2$$

$$= 0.375$$



Árboles de Decisión

- Construcción de un árbol de decisión:

Cuantificación de la impureza → **Impureza de Gini**

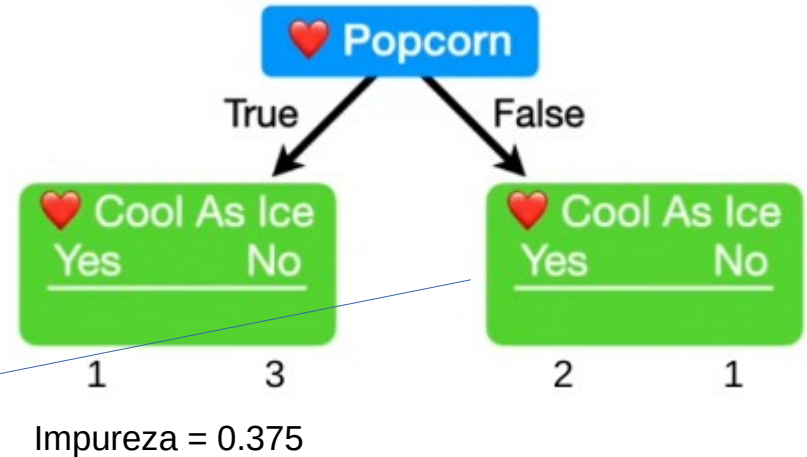
Impureza de Gini = $1 - (\text{prob. Yes})^2 - (\text{prob. No})^2$

$$= 1 - (1/(1+3))^2 - (3/(1+3))^2$$

$$= 0.375$$

Impureza de Gini = $1 - (\text{prob. Yes})^2 - (\text{prob. No})^2$

$$= 1 - (2/(2+1))^2 - (1/(2+1))^2$$



Árboles de Decisión

- Construcción de un árbol de decisión:

Cuantificación de la impureza → **Impureza de Gini**

Impureza de Gini = $1 - (\text{prob. Yes})^2 - (\text{prob. No})^2$

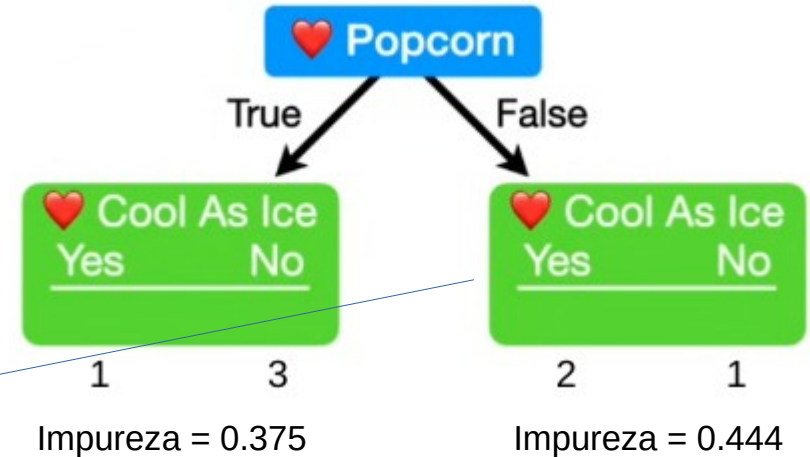
$$= 1 - (1/(1+3))^2 - (3/(1+3))^2$$

$$= 0.375$$

Impureza de Gini = $1 - (\text{prob. Yes})^2 - (\text{prob. No})^2$

$$= 1 - (2/(2+1))^2 - (1/(2+1))^2$$

$$= 0.444$$



Árboles de Decisión

- Construcción de un árbol de decisión:

Cuantificación de la impureza → **Impureza de Gini**

Impureza de Gini = $1 - (\text{prob. Yes})^2 - (\text{prob. No})^2$

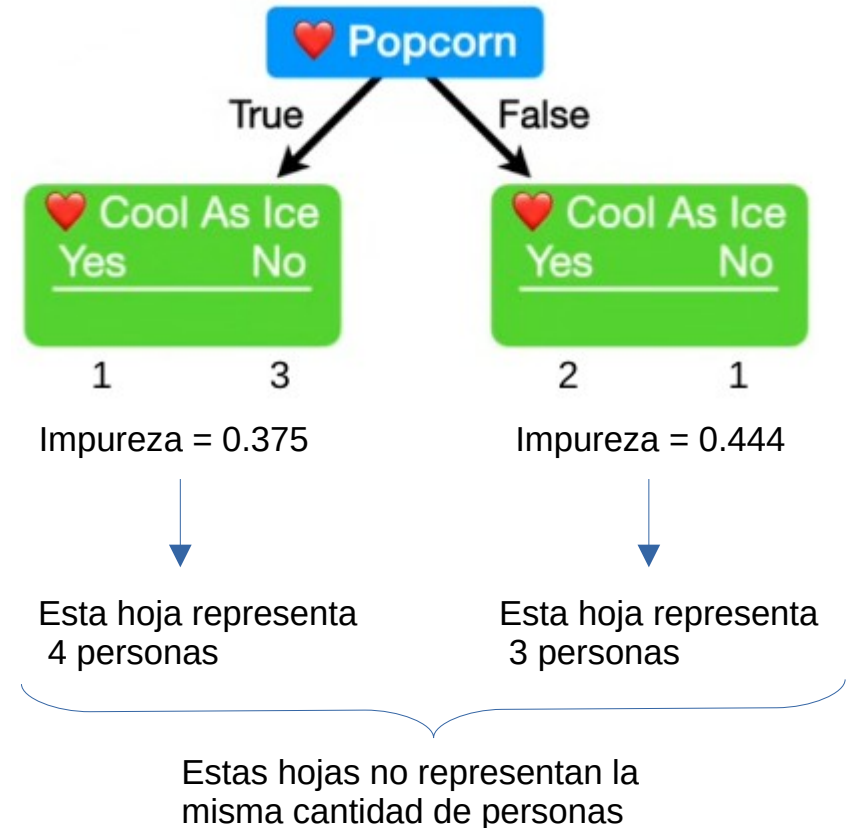
$$= 1 - (1/(1+3))^2 - (3/(1+3))^2$$

$$= 0.375$$

Impureza de Gini = $1 - (\text{prob. Yes})^2 - (\text{prob. No})^2$

$$= 1 - (2/(2+1))^2 - (1/(2+1))^2$$

$$= 0.444$$



Árboles de Decisión

- Construcción de un árbol de decisión:

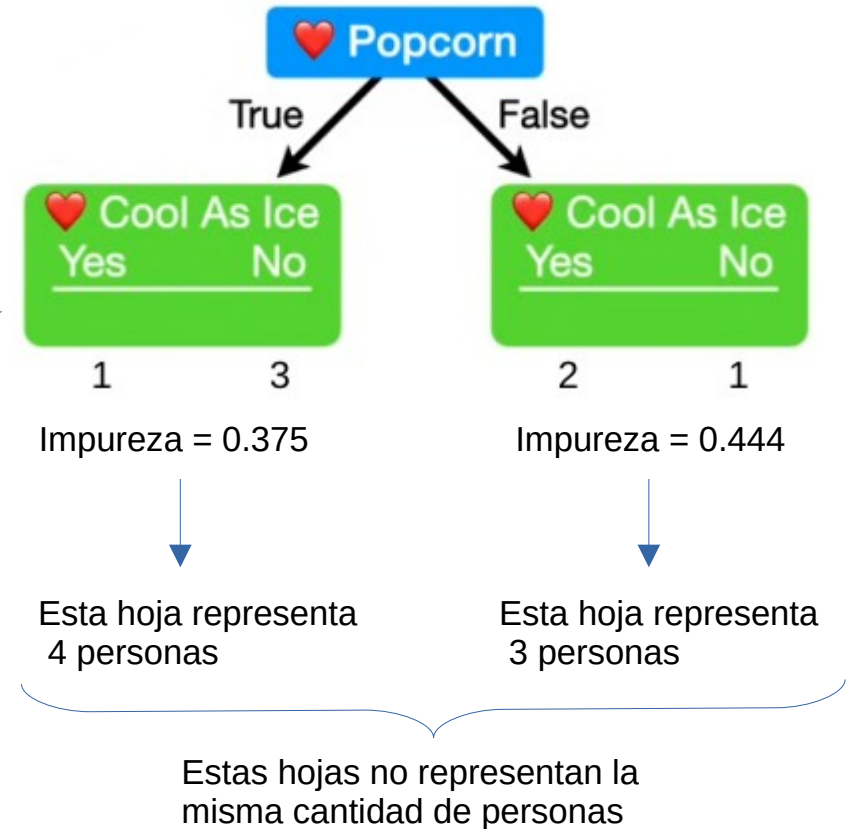
Cuantificación de la impureza → **Impureza de Gini**

Impureza total de Gini = peso ponderado de las impurezas de las hojas

$$= (4/(4+3))0.375 + \dots$$



Número de personas en la hoja de todas las personas en ambas hojas, por su impureza



Árboles de Decisión

- Construcción de un árbol de decisión:

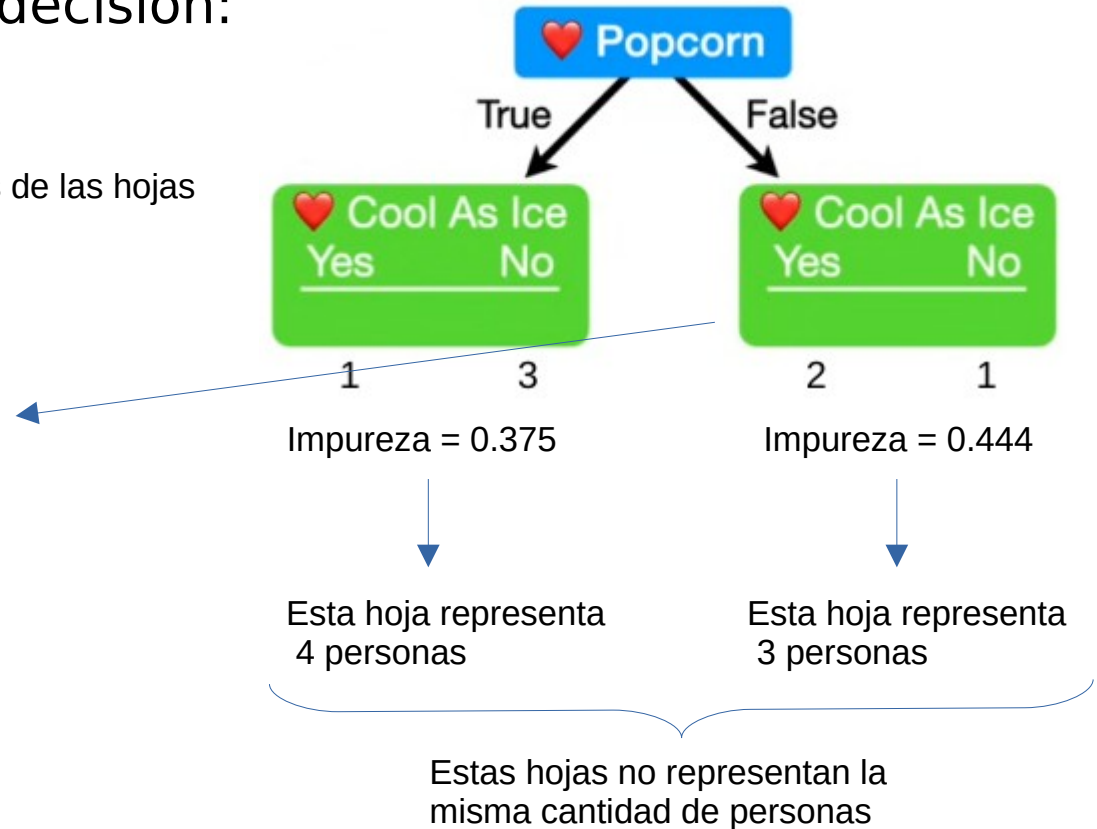
Cuantificación de la impureza → **Impureza de Gini**

Impureza total de Gini = peso ponderado de las impurezas de las hojas

$$= (4/(4+3))0.375 + (3/(4+3))0.444$$



Número de personas en la hoja de todas las personas en ambas hojas, por su impureza



Árboles de Decisión

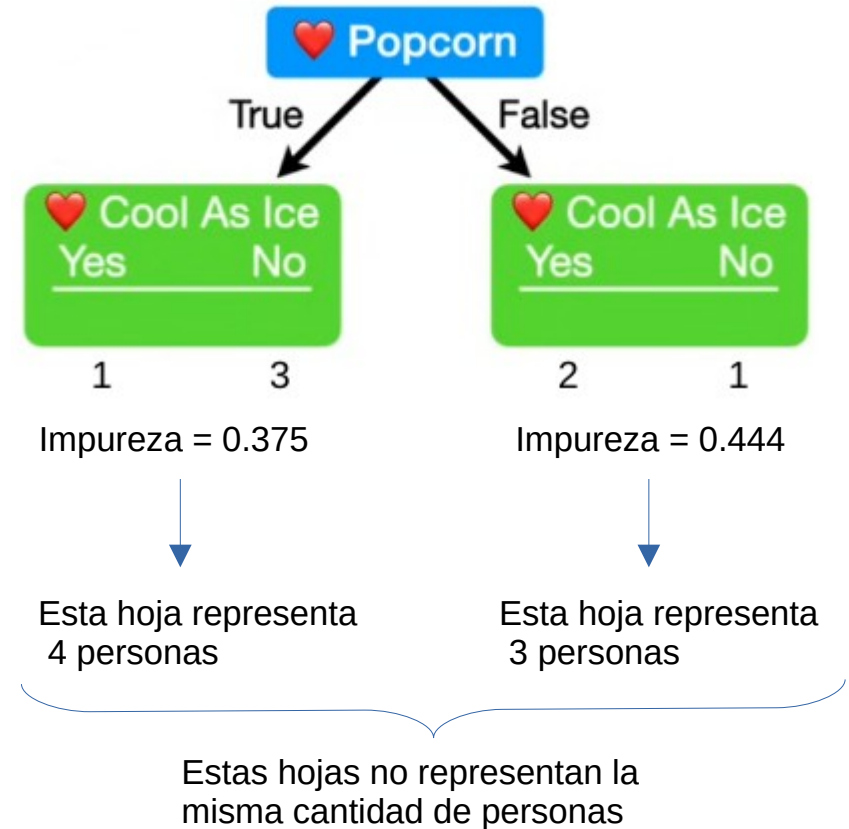
- Construcción de un árbol de decisión:

Cuantificación de la impureza → **Impureza de Gini**

Impureza total de Gini = peso ponderado de las impurezas de las hojas

$$= (4/(4+3))0.375 + (3/(4+3))0.444$$

$$= 0.405$$



Árboles de Decisión

- Construcción de un árbol de decisión:

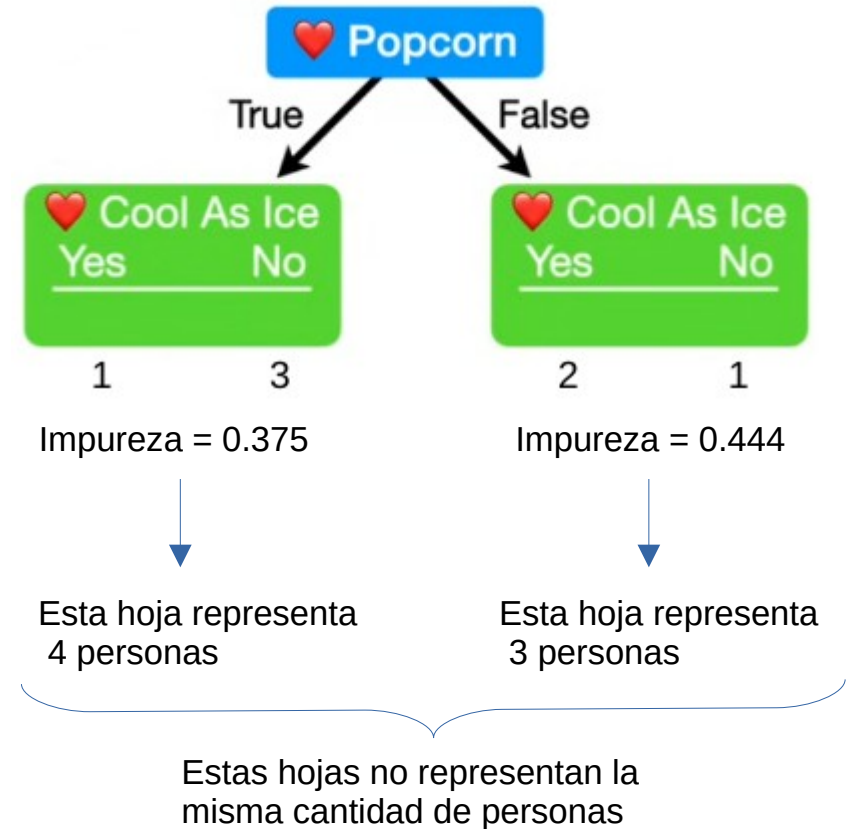
Cuantificación de la impureza → **Impureza de Gini**

Impureza total de Gini = peso ponderado de las impurezas de las hojas

$$= (4/(4+3))0.375 + (3/(4+3))0.444$$

$$= 0.405$$

Impureza Gini de "Loves popcorn" = 0.405



Árboles de Decisión

- Construcción de un árbol de decisión:

Cuantificación de la impureza → **Impureza de Gini**

Impureza de Gini = $1 - (\text{prob. Yes})^2 - (\text{prob. No})^2$

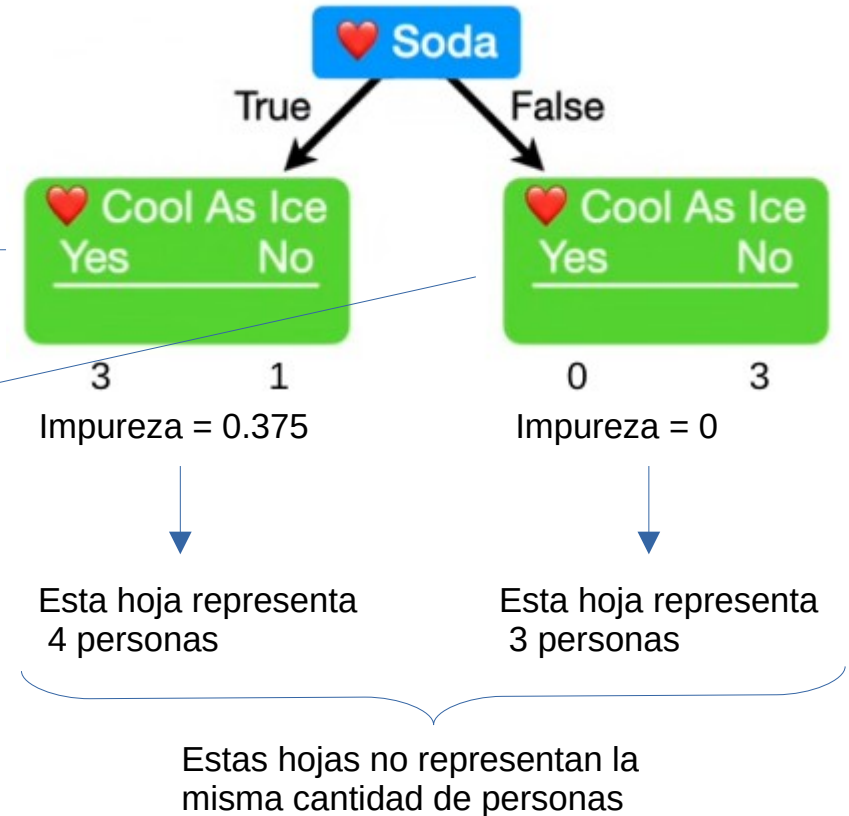
$$= 1 - (3/(3+1))^2 - (1/(3+1))^2$$

$$= 0.375$$

Impureza de Gini = $1 - (\text{prob. Yes})^2 - (\text{prob. No})^2$

$$= 1 - (0/(0+3))^2 - (3/(0+3))^2$$

$$= 0$$



Árboles de Decisión

- Construcción de un árbol de decisión:

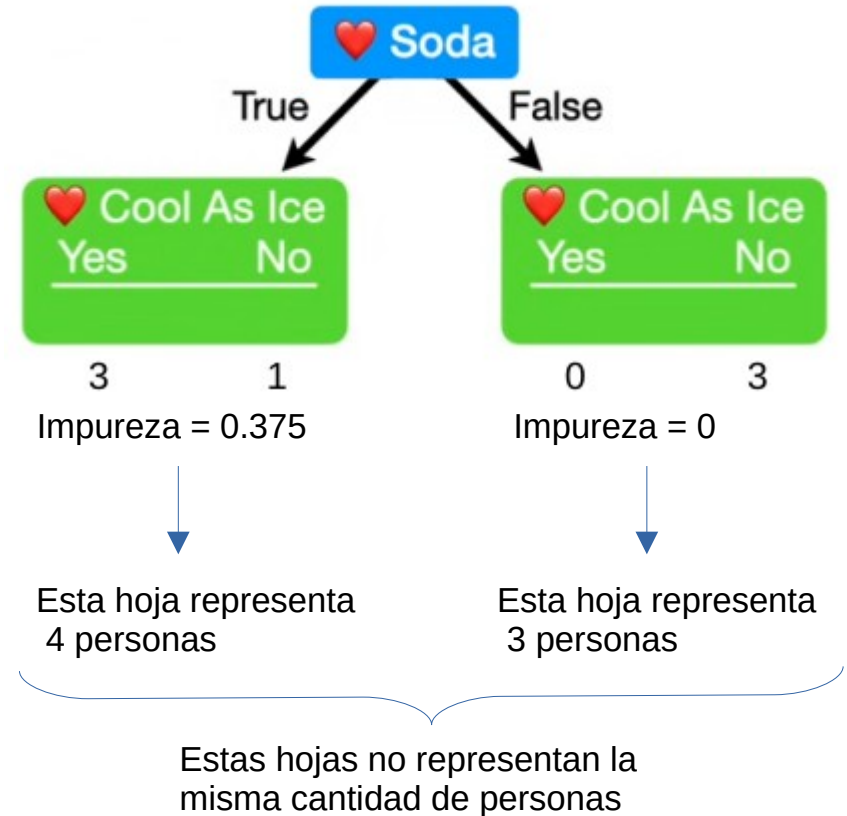
Cuantificación de la impureza → **Impureza de Gini**

Impureza total de Gini = peso ponderado de las impurezas de las hojas

$$= (4/(4+3))0.375 + (3/(4+3))*0$$

$$= 0.214$$

Impureza Gini de “Loves soda” = 0.214



Árboles de Decisión

- Construcción de un árbol de decisión:

Cuantificación de la impureza → **Impureza de Gini**

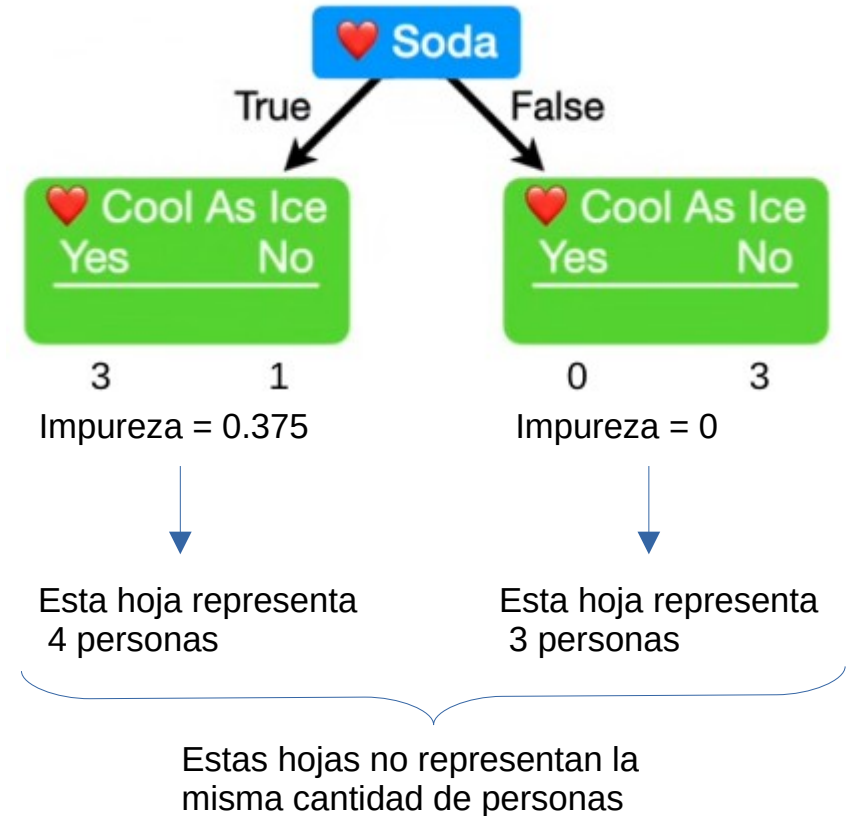
Impureza total de Gini = peso ponderado de las impurezas de las hojas

$$= (4/(4+3))0.375 + (3/(4+3))*0$$

$$= 0.214$$

Impureza Gini de “Loves soda” = 0.214

Conclusión: “Loves soda” es menos impura que “Loves popcorn”



Árboles de Decisión

- Construcción de un árbol de decisión:

Cuantificación de la impureza → **Impureza de Gini**

Impureza total de Gini = peso ponderado de las impurezas de las hojas

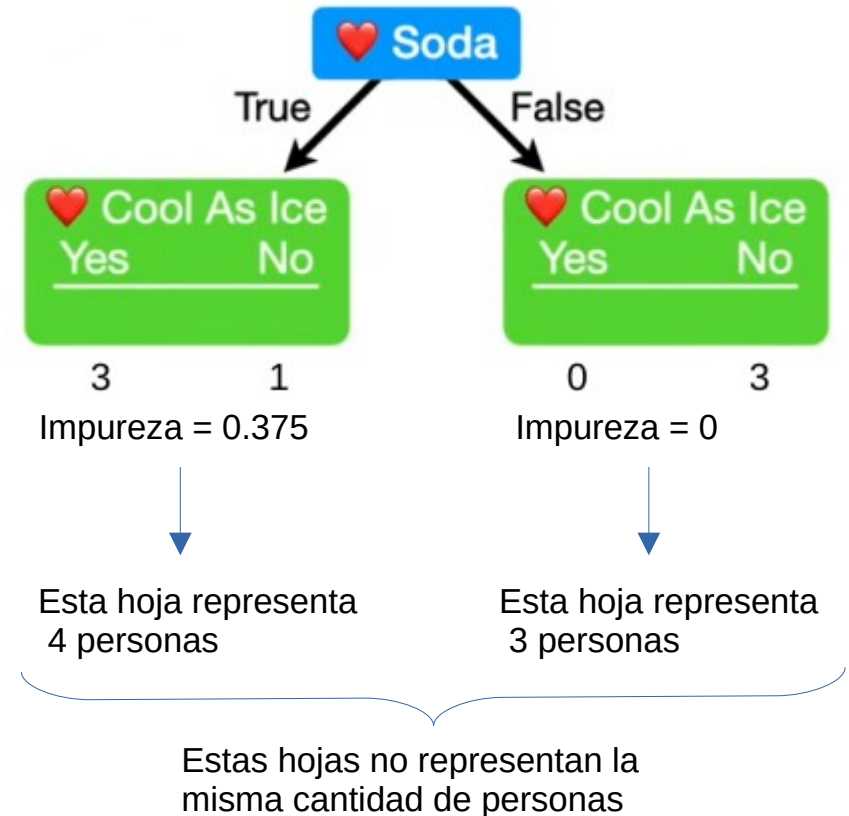
$$= (4/(4+3))0.375 + (3/(4+3))*0$$

$$= 0.214$$

Impureza Gini de “Loves soda” = 0.214

Conclusión: “Loves soda” es menos impura que “Loves popcorn”

=> **“Loves soda” es mas candidata a ser raíz que “Loves popcorn”**



Árboles de Decisión

- Construcción de un árbol de decisión:

Loves Popcorn	Loves Soda	Age	Loves Cool As Ice
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No

Evaluemos la tercera característica → “Age”

Note que “Age” es numérica...

La ordenamos de menor a mayor...

Árboles de Decisión

- Construcción de un árbol de decisión:

Calculamos la edad promedio entre cada par de personas adyacentes...

	Age	Loves Cool As Ice
9.5	7	No
15	12	No
26.5	18	Yes
36.5	35	Yes
44	38	Yes
66.5	50	No
	83	No

Árboles de Decisión

- Construcción de un árbol de decisión:

Calculamos la edad promedio entre cada par de personas adyacentes...

Calculamos la impureza de Gini para cada edad promedio:

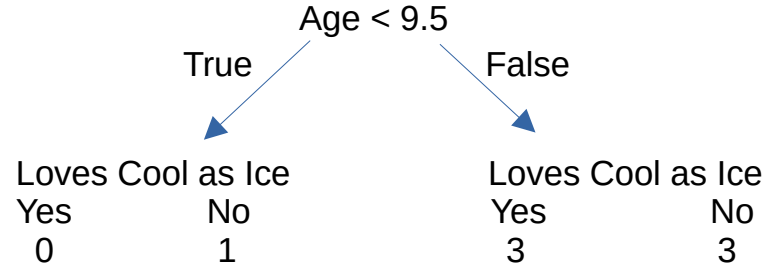
	Age	Loves Cool As Ice	
9.5	7	No	Impureza=0.429
15	12	No	Impureza=0.343
26.5	18	Yes	.
36.5	35	Yes	.
44	38	Yes	.
66.5	50	No	.
	83	No	.

Árboles de Decisión

- Construcción de un árbol de decisión:

	Age	Loves Cool As Ice
	7	No
9.5	12	No
15	18	Yes
26.5	35	Yes
36.5	38	Yes
44	50	No
66.5	83	No

Calculemos la impureza de Gini para 9.5:



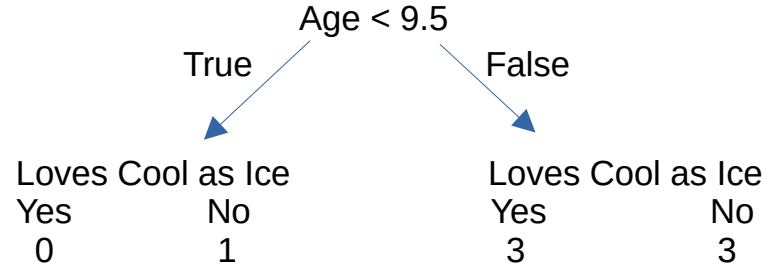
$$\begin{aligned}\text{Impureza de Gini (True)} &= 1 - (\text{prob. Yes})^2 - (\text{prob. No})^2 \\ &= 1 - (0/(0+1))^2 - (1/(0+1))^2 \\ &= 0\end{aligned}$$

Árboles de Decisión

- Construcción de un árbol de decisión:

	Age	Loves Cool As Ice
	7	No
9.5	12	No
15	18	Yes
26.5	35	Yes
36.5	38	Yes
44	50	No
66.5	83	No

Calculemos la impureza de Gini para 9.5:



$$\begin{aligned}\text{Impureza de Gini (False)} &= 1 - (\text{prob. Yes})^2 - (\text{prob. No})^2 \\ &= 1 - (3/(3+3))^2 - (3/(3+3))^2 \\ &= 0.5\end{aligned}$$

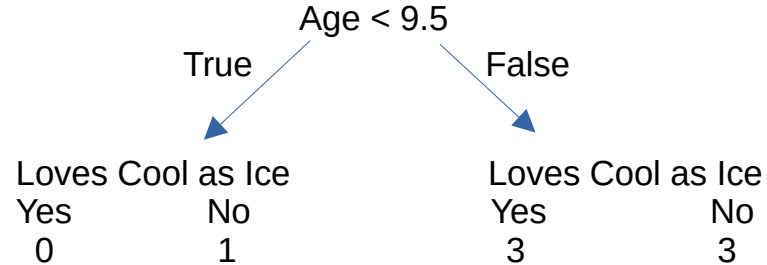
Árboles de Decisión

- Construcción de un árbol de decisión:

	Age	Loves Cool As Ice
9.5	7	No
15	12	No
26.5	18	Yes
36.5	35	Yes
44	38	Yes
66.5	50	No
	83	No

Impureza=0.429

Calculemos la impureza de Gini para 9.5:



Impureza total de Gini para 9.5= peso ponderado de las impurezas de las hojas

$$= (1/(1+6))0 + (6/(1+6))*0.5 = 0.429$$

Árboles de Decisión

- Construcción de un árbol de decisión:

Calculemos la impureza de Gini para el resto de edades promedio...

	Age	Loves Cool As Ice	
	7	No	
9.5	12	No	Impureza=0.429
15	18	Yes	Impureza=0.343
26.5	35	Yes	
36.5	38	Yes	
44	50	No	
66.5	83	No	

Árboles de Decisión

- Construcción de un árbol de decisión:

Calculemos la impureza de Gini para el resto de edades promedio...

	Age	Loves Cool As Ice	
	7	No	
9.5			Impureza=0.429
	12	No	
15			Impureza=0.343
	18	Yes	
26.5			Impureza=0.476
	35	Yes	
36.5			Impureza=0.476
	38	Yes	
44			Impureza=0.343
	50	No	
66.5			Impureza=0.429
	83	No	

Árboles de Decisión

- Construcción de un árbol de decisión:

Calculemos la impureza de Gini para el resto de edades promedio...

	Age	Loves Cool As Ice	
9.5	7	No	Impureza=0.429
15	12	No	Impureza=0.343
26.5	18	Yes	Impureza=0.476
36.5	35	Yes	Impureza=0.476
44	38	Yes	Impureza=0.343
66.5	50	No	Impureza=0.429
	83	No	

Impureza mas baja → escogemos cualquier de estos dos umbrales como nodo de decisión → escojamos a 15

Árboles de Decisión

- Construcción de un árbol de decisión:

Calculemos la impureza de Gini para el resto de edades promedio...

	Age	Loves Cool As Ice	
9.5	7	No	Impureza=0.429
15	12	No	Impureza=0.343
26.5	18	Yes	Impureza=0.476
36.5	35	Yes	Impureza=0.476
44	38	Yes	Impureza=0.343
66.5	50	No	Impureza=0.429
	83	No	

Impureza mas baja → escogemos cualquier de estos dos umbrales como nodo de decisión → escojamos a 15

Recordemos que estamos hallando las impurezas de cada columna para decidir cual debe ser la raíz...

Árboles de Decisión

- Construcción de un árbol de decisión:

Calculemos la impureza de Gini para el resto de edades promedio...

	Age	Loves Cool As Ice	
9.5	7	No	Impureza=0.429
15	12	No	Impureza=0.343
26.5	18	Yes	Impureza=0.476
36.5	35	Yes	Impureza=0.476
44	38	Yes	Impureza=0.343
66.5	50	No	Impureza=0.429
	83	No	

Impureza mas baja → escogemos cualquier de estos dos umbrales como nodo de decisión → escojamos a 15

Recordemos que estamos hallando las impurezas de cada columna para decidir cual debe ser la raíz:

Impureza de "Loves popcorn" = 0.405

Impureza de "Loves soda" = 0.215

Impureza para "Age < 15" = 0.343

Árboles de Decisión

- Construcción de un árbol de decisión:

Calculemos la impureza de Gini para el resto de edades promedio...

	Age	Loves Cool As Ice	
9.5	7	No	Impureza=0.429
15	12	No	Impureza=0.343
26.5	18	Yes	Impureza=0.476
36.5	35	Yes	Impureza=0.476
44	38	Yes	Impureza=0.343
66.5	50	No	Impureza=0.429
	83	No	

Impureza mas baja → escogemos cualquier de estos dos umbrales como nodo de decisión → escojamos a 15

Recordemos que estamos hallando las impurezas de cada columna para decidir cual debe ser la raíz:

Impureza de "Loves popcorn" = 0.405

Impureza de "Loves soda" = 0.215 Esta debería ser la raíz!

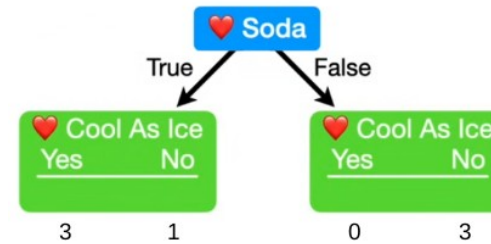
Impureza para "Age < 15" = 0.343

Árboles de Decisión

- Construcción de un árbol de decisión:

Impureza de “Loves soda” = 0.215 Esta debería ser la raíz!

	Age	Loves Cool As Ice	
9.5	7	No	Impureza=0.429
15	12	No	Impureza=0.343
	18	Yes	Impureza=0.476
26.5	35	Yes	Impureza=0.476
36.5	38	Yes	Impureza=0.343
44	50	No	Impureza=0.429
66.5	83	No	

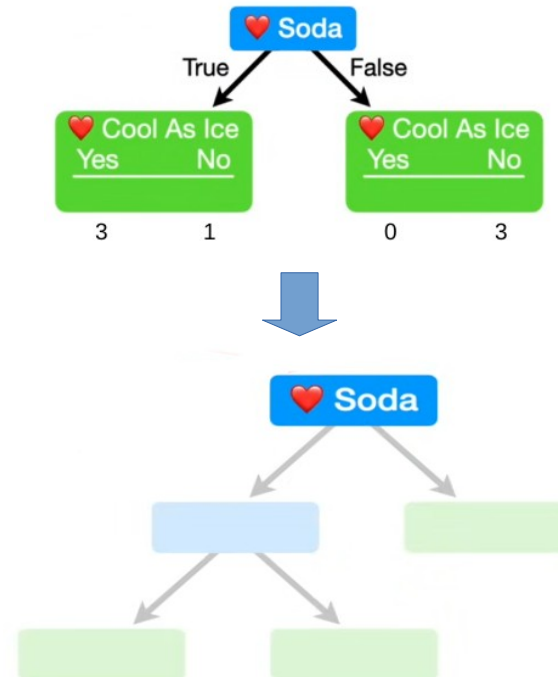


Árboles de Decisión

- Construcción de un árbol de decisión:

Impureza de “Loves soda” = 0.215 Esta debería ser la raíz!

	Age	Loves Cool As Ice	
9.5	7	No	Impureza=0.429
15	12	No	Impureza=0.343
26.5	18	Yes	Impureza=0.476
36.5	35	Yes	Impureza=0.476
44	38	Yes	Impureza=0.343
66.5	50	No	Impureza=0.429
	83	No	

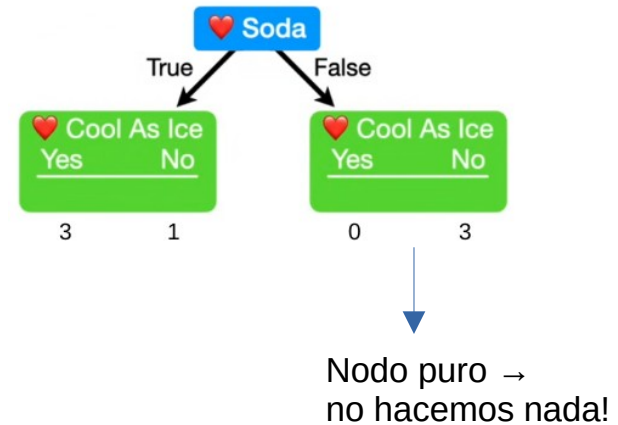


Árboles de Decisión

- Construcción de un árbol de decisión:

	Age	Loves Cool As Ice	
9.5	7	No	Impureza=0.429
15	12	No	Impureza=0.343
26.5	18	Yes	Impureza=0.476
36.5	35	Yes	Impureza=0.476
44	38	Yes	Impureza=0.343
66.5	50	No	Impureza=0.429
	83	No	

Impureza de “Loves soda” = 0.215 Esta debería ser la raíz!

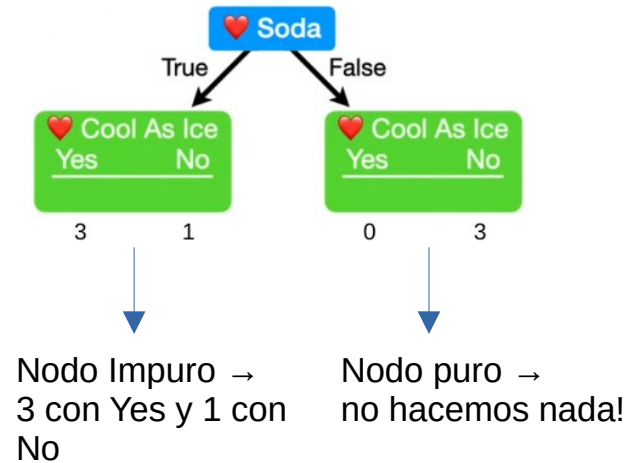


Árboles de Decisión

- Construcción de un árbol de decisión:

	Age	Loves Cool As Ice	
9.5	7	No	Impureza=0.429
15	12	No	Impureza=0.343
	18	Yes	Impureza=0.476
26.5	35	Yes	Impureza=0.476
36.5	38	Yes	Impureza=0.343
44	50	No	Impureza=0.429
66.5	83	No	

Impureza de "Loves soda" = 0.215 Esta debería ser la raíz!

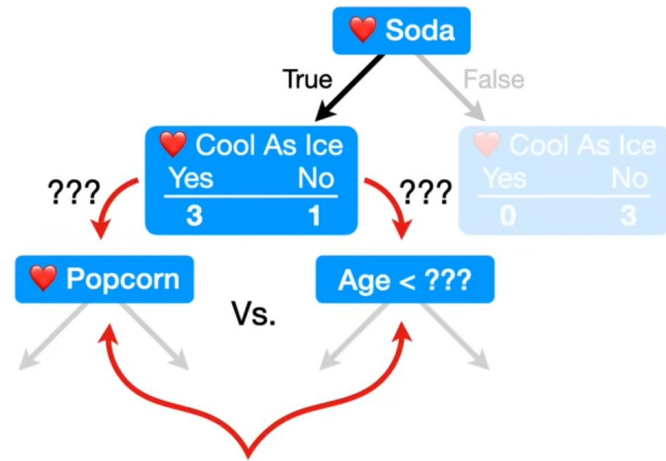


Árboles de Decisión

- Construcción de un árbol de decisión:

	Age	Loves Cool As Ice	
9.5	7	No	Impureza=0.429
15	12	No	Impureza=0.343
	18	Yes	Impureza=0.476
26.5	35	Yes	Impureza=0.476
36.5	38	Yes	Impureza=0.343
44	50	No	Impureza=0.429
66.5	83	No	

Impureza de "Loves soda" = 0.215 Esta debería ser la raíz!



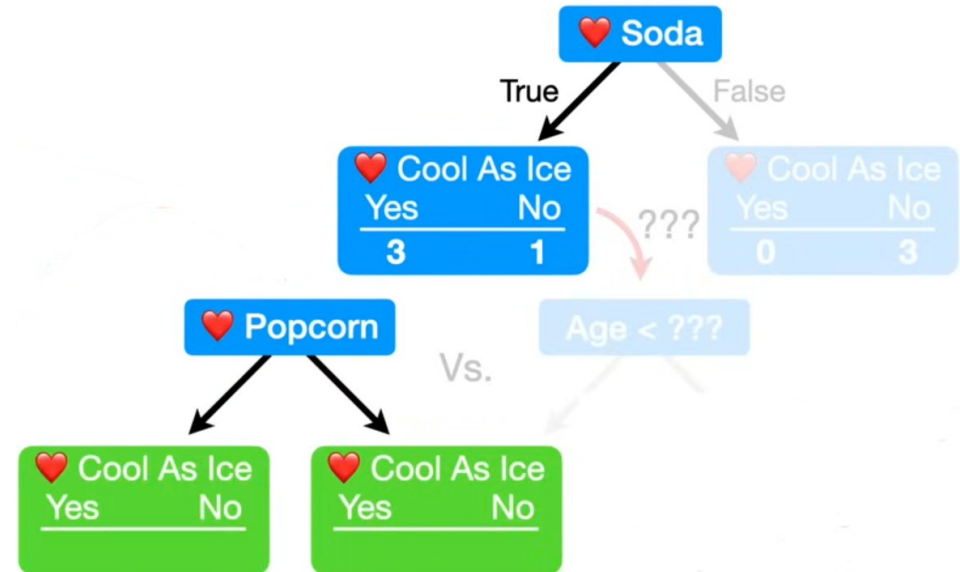
Debemos analizar si se puede reducir la impureza dividiendo las personas que les gusta la Soda basándonos en "Loves Popcorn" o "Age".

Árboles de Decisión

- Construcción de un árbol de decisión:

Loves Popcorn	Loves Soda	Age	Loves Cool As Ice
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No

Impureza de “Loves soda” = 0.215 Esta debería ser la raíz!



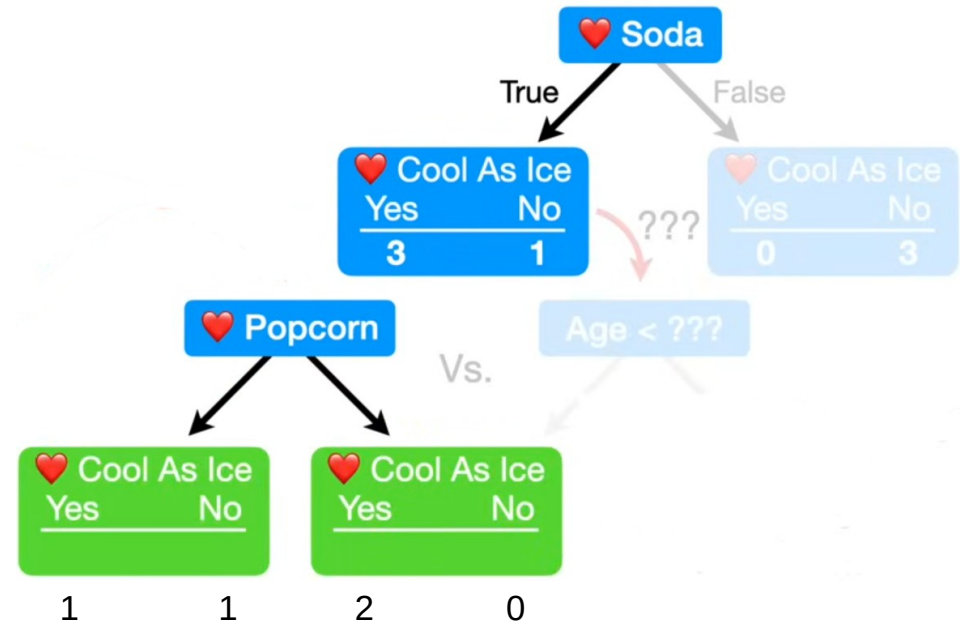
De acuerdo a los que les guste la Soda, debemos contabilizar los “Yes” y los “No” de si nos gusta o no el Popcorn.

Árboles de Decisión

- Construcción de un árbol de decisión:

Loves Popcorn	Loves Soda	Age	Loves Cool As Ice
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No

Impureza de “Loves soda” = 0.215 Esta debería ser la raíz!

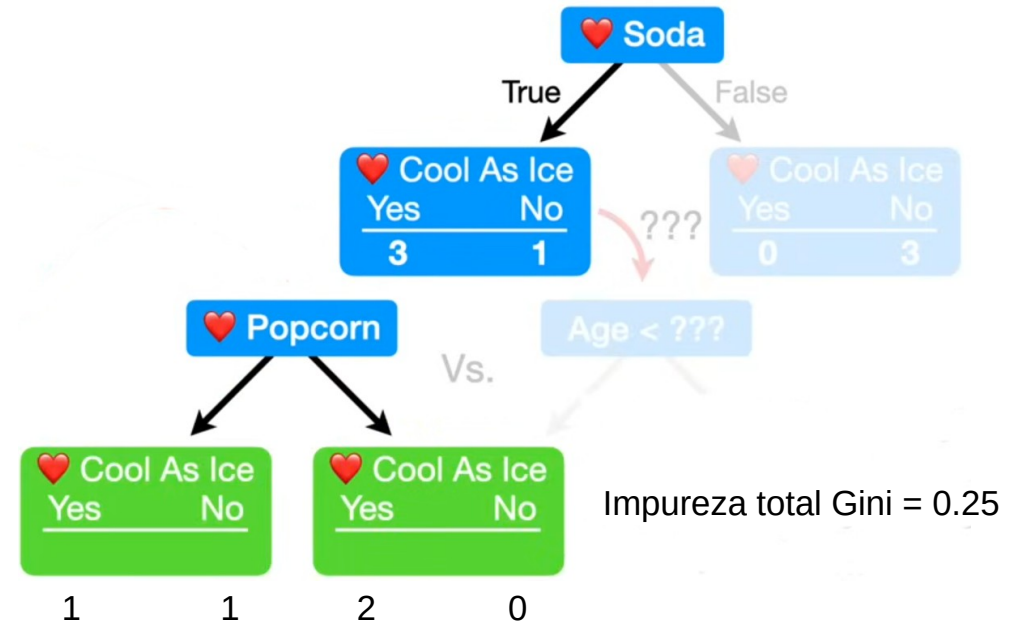


Árboles de Decisión

- Construcción de un árbol de decisión:

Loves Popcorn	Loves Soda	Age	Loves Cool As Ice
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No

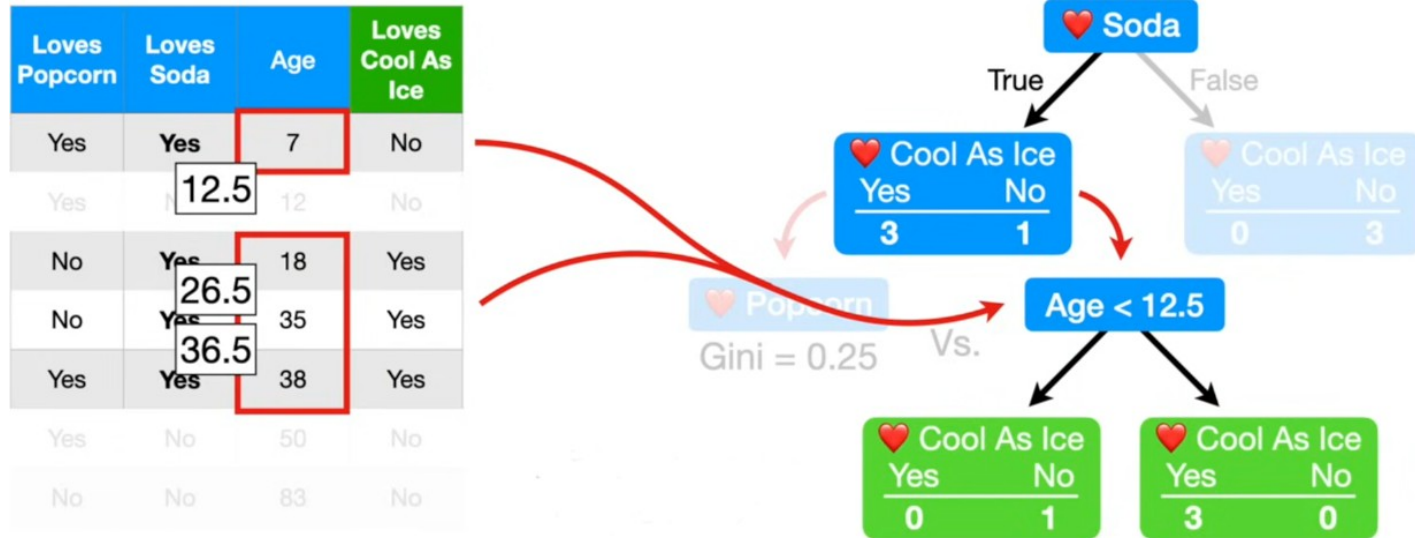
Impureza de “Loves soda” = 0.215 Esta debería ser la raíz!



Árboles de Decisión

- Construcción de un árbol de decisión:

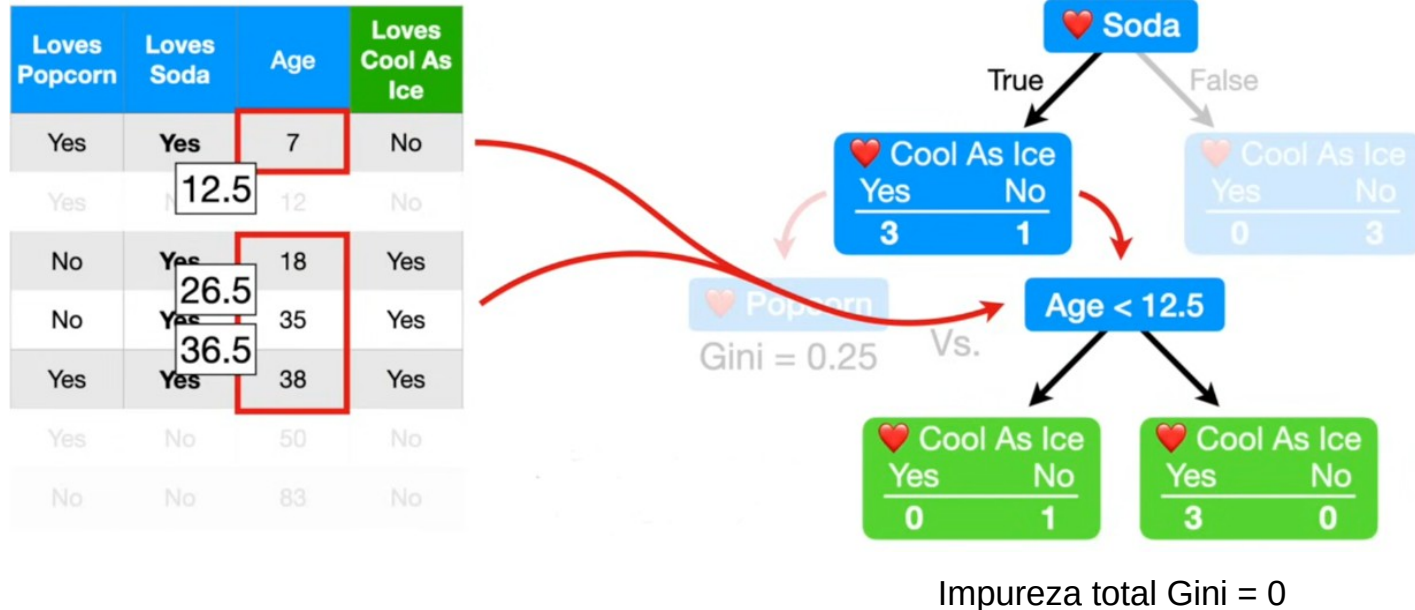
Impureza de “Loves soda” = 0.215 Esta debería ser la raíz!



Árboles de Decisión

- Construcción de un árbol de decisión:

Impureza de “Loves soda” = 0.215 Esta debería ser la raíz!

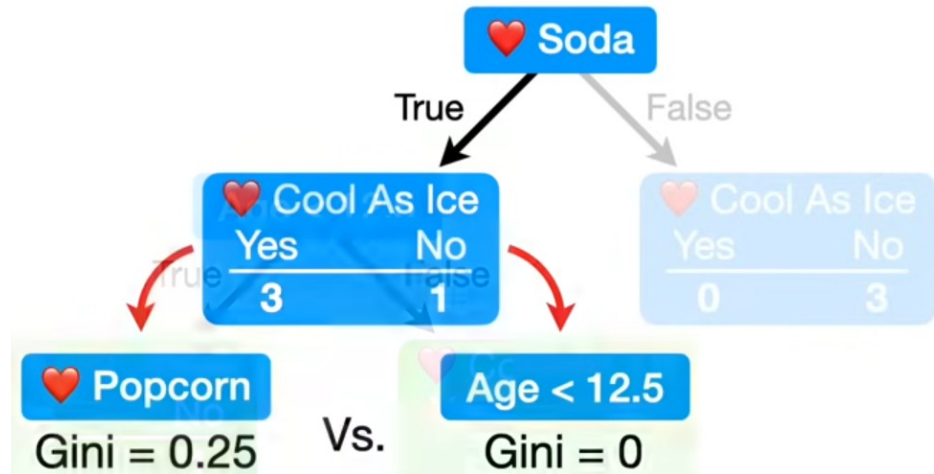


Árboles de Decisión

- Construcción de un árbol de decisión:

Loves Popcorn	Loves Soda	Age	Loves Cool As Ice
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No

Impureza de “Loves soda” = 0.215 Esta debería ser la raíz!



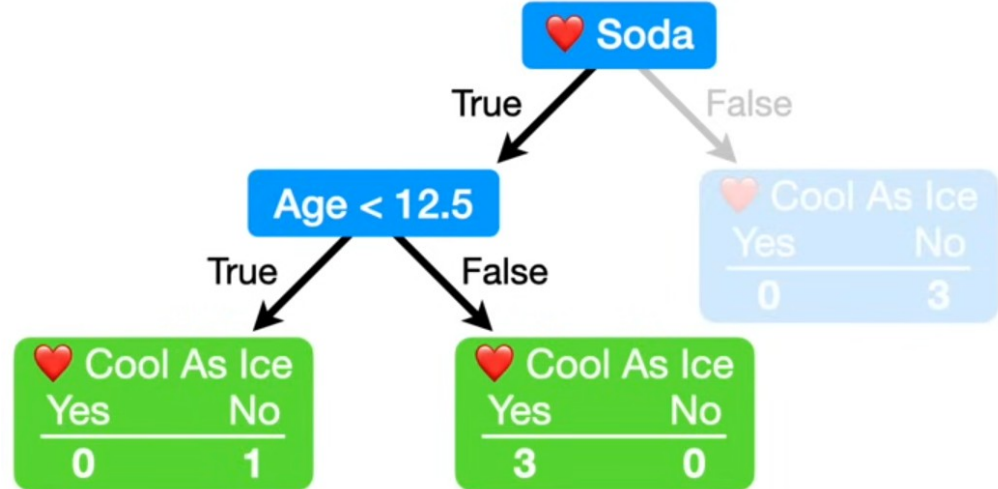
Dado que “Age < 12.5” nos da un Gini menor que el de “Loves popcorn”, el nodo “Cool as Ice” lo dividimos en hojas.

Árboles de Decisión

- Construcción de un árbol de decisión:

Loves Popcorn	Loves Soda	Age	Loves Cool As Ice
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No

Impureza de “Loves soda” = 0.215 Esta debería ser la raíz!

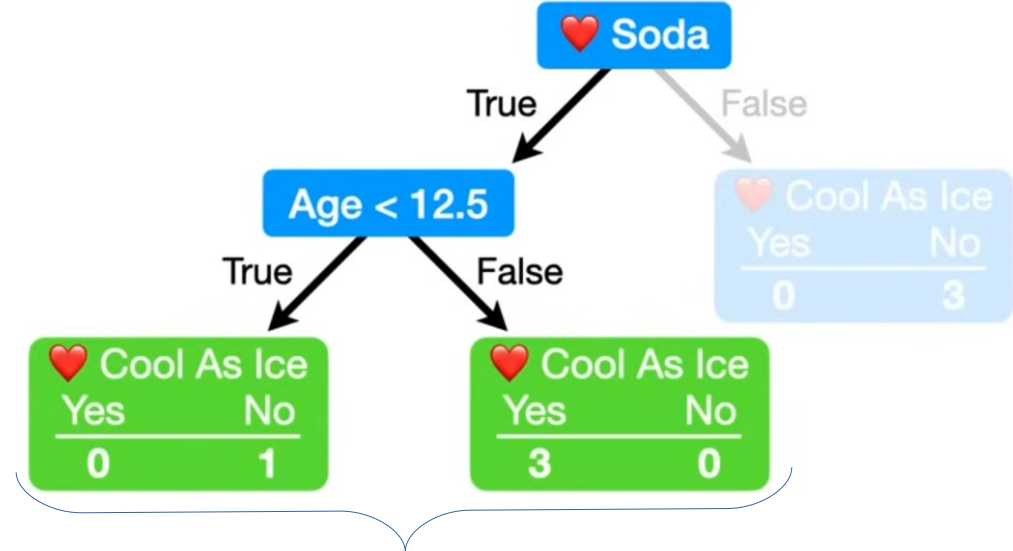


Árboles de Decisión

- Construcción de un árbol de decisión:

Loves Popcorn	Loves Soda	Age	Loves Cool As Ice
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No

Impureza de “Loves soda” = 0.215 Esta debería ser la raíz!



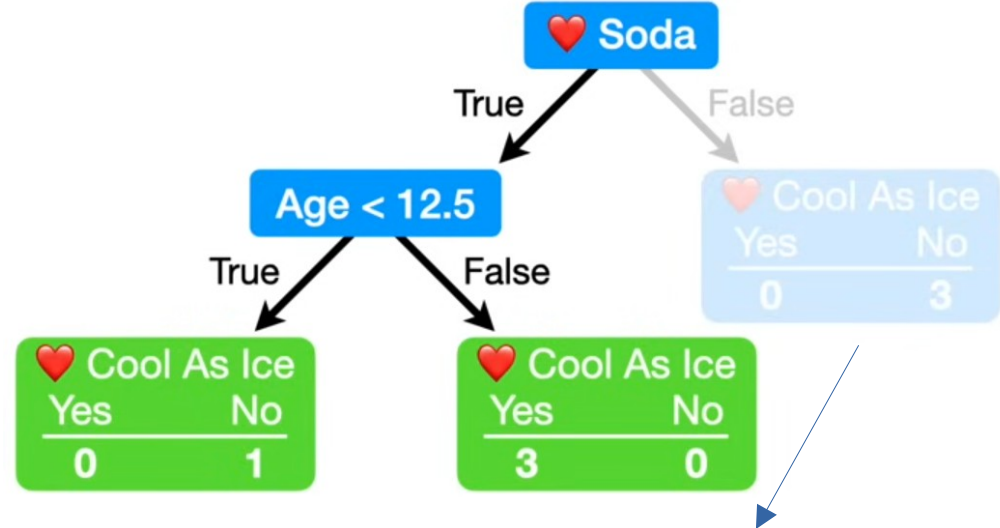
Estos nodos serían hojas porque no tendría sentido dividirlos mas porque su impureza es 0.

Árboles de Decisión

- Construcción de un árbol de decisión:

Loves Popcorn	Loves Soda	Age	Loves Cool As Ice
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No

Impureza de “Loves soda” = 0.215 Esta debería ser la raíz!



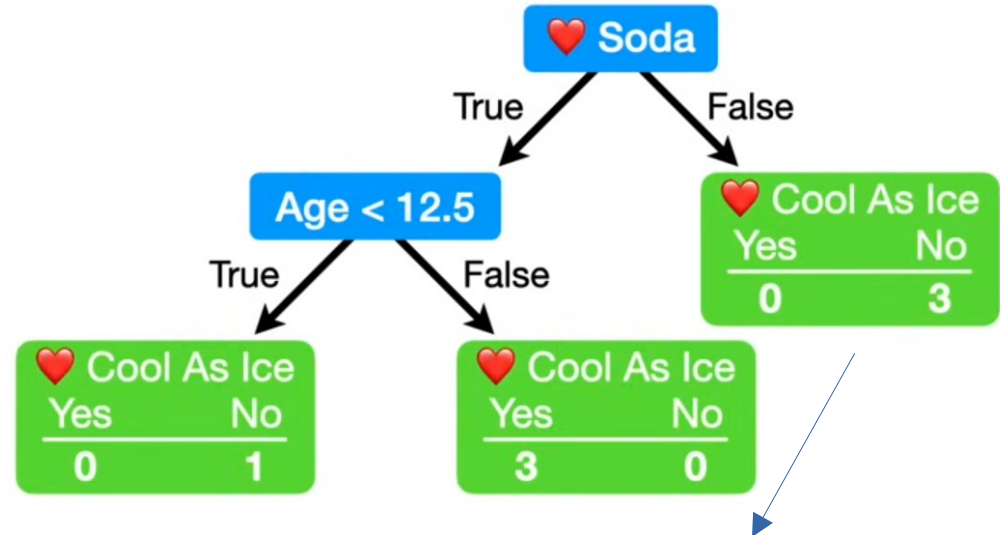
Dado que en este nodo su impureza es 0, entonces también sería un nodo hoja.

Árboles de Decisión

- Construcción de un árbol de decisión:

Loves Popcorn	Loves Soda	Age	Loves Cool As Ice
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No

Impureza de “Loves soda” = 0.215 Esta debería ser la raíz!



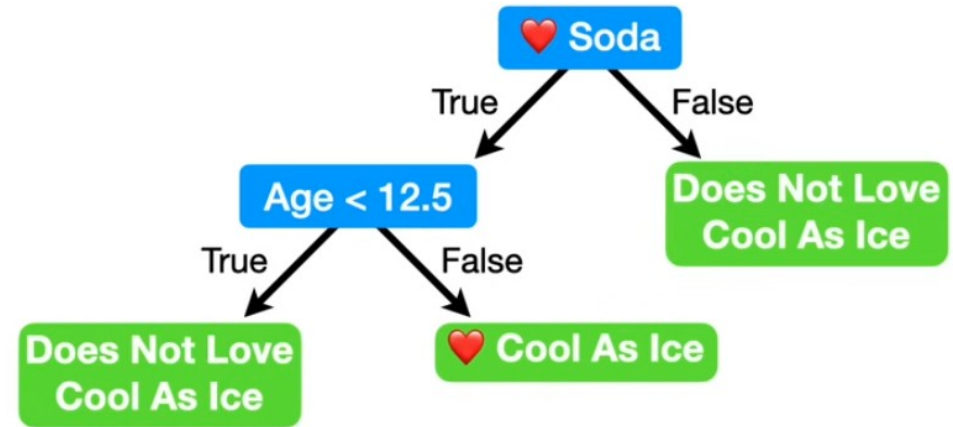
Dado que en este nodo su impureza es 0, entonces también sería un nodo hoja.

Árboles de Decisión

- Construcción de un árbol de decisión:

Loves Popcorn	Loves Soda	Age	Loves Cool As Ice
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No

Impureza de “Loves soda” = 0.215 Esta debería ser la raíz!



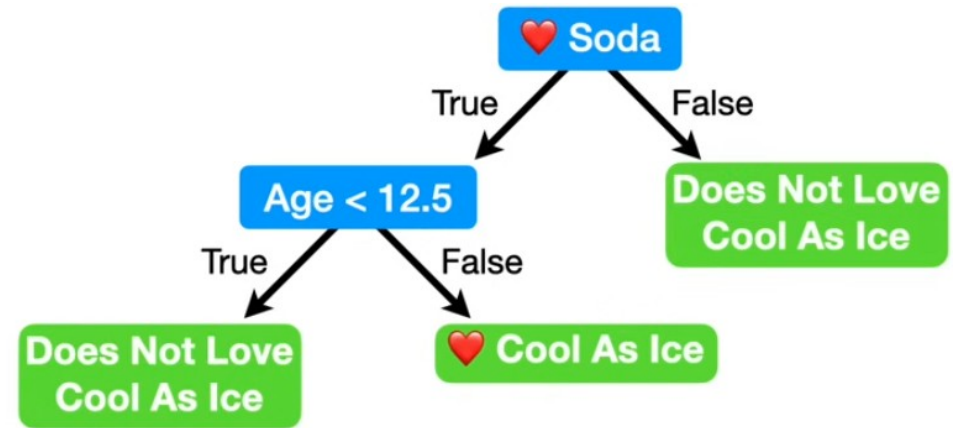
A cada hoja se le asigna como salida, la clase mayoritaria.

Árboles de Decisión

- Construcción de un árbol de decisión:

Loves Popcorn	Loves Soda	Age	Loves Cool As Ice
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No

Impureza de “Loves soda” = 0.215 Esta debería ser la raíz!

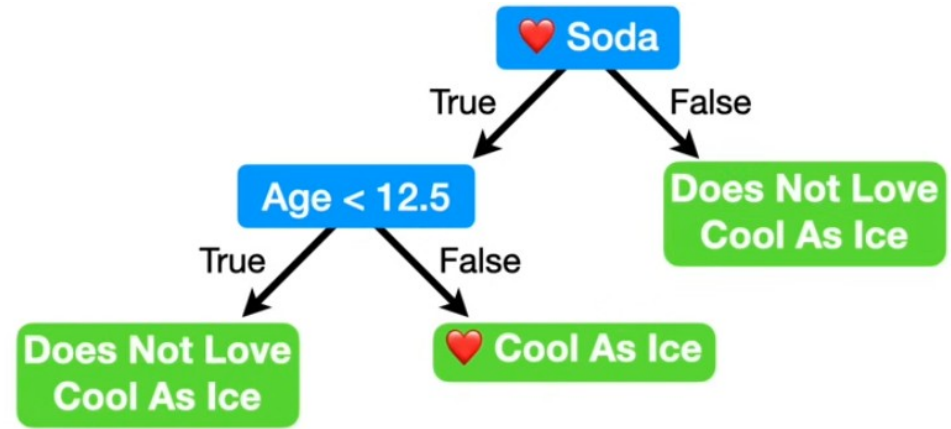


Dado que llegamos a hojas con impureza 0, el árbol acertará la variable objetivo (Loves Cool as Ice) para todas las muestras (personas) del dataset.

Árboles de Decisión

- Construcción de un árbol de decisión:

Loves Popcorn	Loves Soda	Age	Loves Cool As Ice
Yes	Yes	15	???



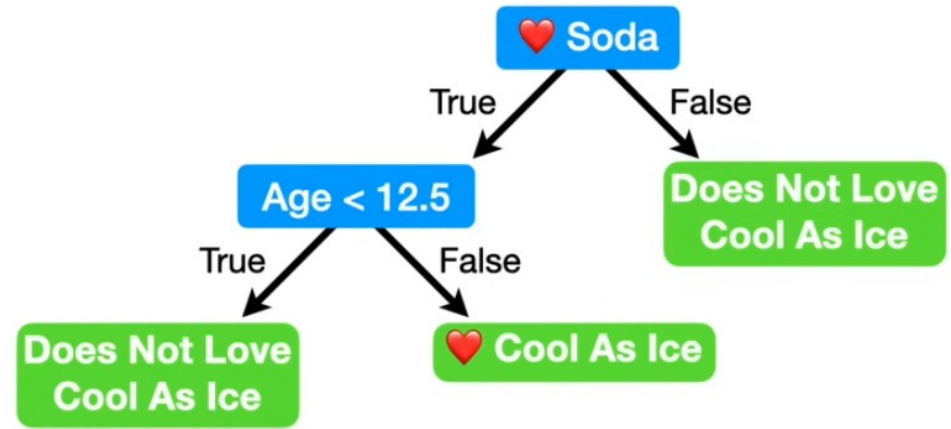
Ya podríamos predecir una nueva muestra que llegue a nuestro dataset!

Árboles de Decisión

- Construcción de un árbol de decisión:

Loves Popcorn	Loves Soda	Age	Loves Cool As Ice
Yes	Yes	15	???

Yes



Ya podríamos predecir una nueva muestra que llegue a nuestro dataset!

Árboles de Decisión

- Construcción de un árbol de decisión:
 - El procedimiento que acabamos de ver corresponde al algoritmo **CART** (Classification and Regression Trees).
 - Es la piedra angular de los árboles de decisión modernos.
 - Su funcionamiento se basa en una estrategia denominada partición binaria recursiva.
 - A diferencia de otros algoritmos como ID3 o C4.5, CART siempre divide un nodo en exactamente dos hijos (Izquierda/Derecha).

Árboles de Decisión

- Estrategia general de CART:
 - **Evaluación de características:** El algoritmo analiza todas las variables disponibles (X_1 , X_2 , . . . , X_n) y todos los posibles puntos de corte (umbrales) para cada una.
 - **Cálculo de Impureza:** Para cada posible división, se calcula la reducción del desorden.
 - **Selección de la mejor división:** Elige la combinación de variable y umbral que maximiza la "pureza" de los nodos resultantes.
 - **Recursión:** El proceso se repite para cada uno de los nodos hijos creados, convirtiéndolos en nuevos "nodos raíz" de sus propios subárboles.
 - **Criterio de parada:** El algoritmo se detiene cuando se cumple una condición (ej. el nodo es puro, se alcanzó la profundidad máxima o hay muy pocos datos para continuar).

Árboles de Decisión

- Implementación:
 - Revisar el archivo ej_RF_clasificación.ipynb.

K-Nearest Neighbors (KNN)

- Características:
 - Es uno de los métodos más intuitivos y sencillos dentro del aprendizaje supervisado.
 - Se utiliza tanto para tareas de **clasificación** (predecir una etiqueta) como de **regresión** (predecir un valor numérico).
 - k-NN no aprende una función matemática durante una fase de entrenamiento. Simplemente memoriza los datos de entrenamiento: cuando se le presenta un nuevo dato, busca cuáles son los elementos más parecidos a él para tomar una decisión.

K-Nearest Neighbors (KNN)

- Funcionamiento general:
 - **Definir k:** Se elige un número entero (por ejemplo, k=3). Este número indica cuántos vecinos consultará el algoritmo.
 - **Calcular la distancia:** El algoritmo mide la distancia entre el nuevo punto y todos los demás puntos del conjunto de datos. La métrica más común es la Distancia Euclidiana.

$$d(X, Y) = \left(\sum_{i=1}^n (x_i - y_i)^2 \right)^{1/2}$$

K-Nearest Neighbors (KNN)

- Funcionamiento general:
 - **Definir k:** Se elige un número entero (por ejemplo, k=3). Este número indica cuántos vecinos consultará el algoritmo.
 - **Calcular la distancia:** El algoritmo mide la distancia entre el nuevo punto y todos los demás puntos del conjunto de datos. La métrica más común es la Distancia Euclidiana (2-D).

$$d(X, Y) = \left(\sum_{i=1}^n (x_i - y_i)^2 \right)^{1/2}$$

Así se vería para mas de dos dimensiones:

$$d = [(x_1 - y_1)^2 + (x_2 - y_2)^2 + \dots + (x_n - y_n)^2]^{0.5}$$

K-Nearest Neighbors (KNN)

- Distancia aplicada a un dataset:

	x_1	x_2	x_3	...	x_n
1	x_{11}	x_{21}	x_{31}	...	x_{n1}
2	x_{12}	x_{22}	x_{32}	...	x_{n2}
3	x_{13}	x_{23}	x_{33}	...	x_{n3}
...	...	...	...	...	...
K	x_{1K}	x_{2K}	x_{3K}	...	x_{nK}

$$D_{13} = \left((x_{11} - x_{13})^2 + (x_{21} - x_{23})^2 + (x_{31} - x_{33})^2 + \dots + (x_{n1} - x_{n3})^2 \right)^{1/2}$$

K-Nearest Neighbors (KNN)

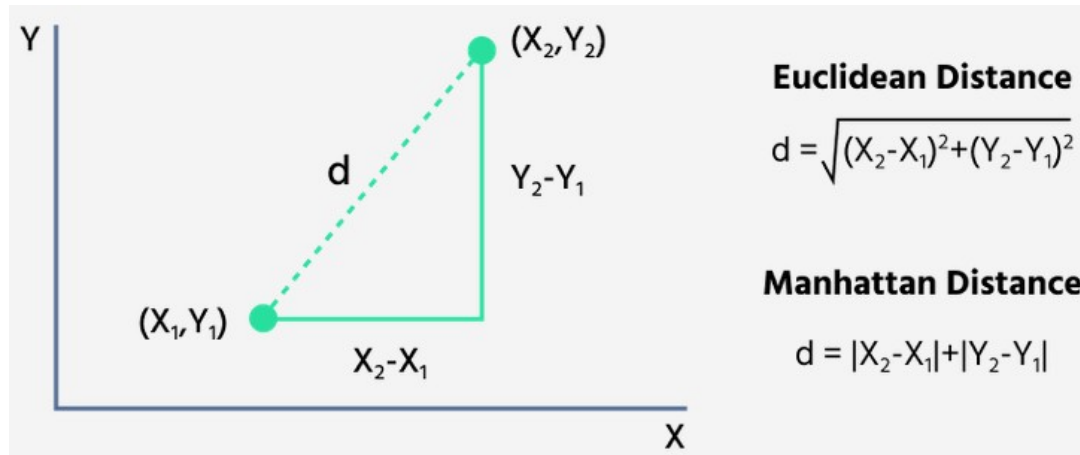
- Distancia aplicada a un dataset:

		x_1	x_2	x_3	...	x_n
i	1	x_{11}	x_{21}	x_{31}	...	x_{n1}
	2	x_{12}	x_{22}	x_{32}	...	x_{n2}
j	3	x_{13}	x_{23}	x_{33}	...	x_{n3}
	...	...	...	...	...	...
	...	...	...	...	...	...
	...	...	...	...	...	...
	K	x_{1K}	x_{2K}	x_{3K}	...	x_{nK}

$$D_{ij} = \left((x_{1i} - x_{1j})^2 + (x_{2i} - x_{2j})^2 + (x_{3i} - x_{3j})^2 + \dots + (x_{ni} - x_{nj})^2 \right)^{1/2}$$

K-Nearest Neighbors (KNN)

- Info extra sobre la distancia:
 - También podemos usar la distancia Manhattan.



K-Nearest Neighbors (KNN)

- Info extra sobre la distancia:
 - Expresión general para la distancia:

$$d(x, y) = \left(\sum_{i=1}^n |x_i - y_i|^p \right)^{1/p}$$

- Si $p=1$: Distancia Manhattan
- Si $p=2$: Distancia Euclidiana

K-Nearest Neighbors (KNN)

- Info extra sobre la distancia:
 - Comparación entre la Euclidiana y la *Manhattan.

Característica	Euclidiana (L_2)	Manhattan (L_1)
Trayectoria	Línea recta (diagonal).	Forma de "escalera" o rejilla.
Sensibilidad	Muy sensible a grandes diferencias (por el cuadrado).	Menos sensible a valores extremos.
Cálculo	Más costosa (requiere raíces y potencias).	Muy rápida (solo sumas y restas).
Geometría	El camino más corto posible.	Suma de catetos.

*También se le conoce como Distancia de “Bloques de Ciudad”.

K-Nearest Neighbors (KNN)

- Funcionamiento general:
 - **Identificar los vecinos:** Seleccionar los k puntos que tengan la distancia más corta hacia el nuevo dato.
 - **Votación** (Clasificación) o **Promedio** (Regresión):
 - En clasificación, el nuevo punto se asigna a la clase más frecuente entre sus k vecinos.
 - En regresión, se calcula el promedio de los valores de los k vecinos para dar un resultado.

K-Nearest Neighbors (KNN)

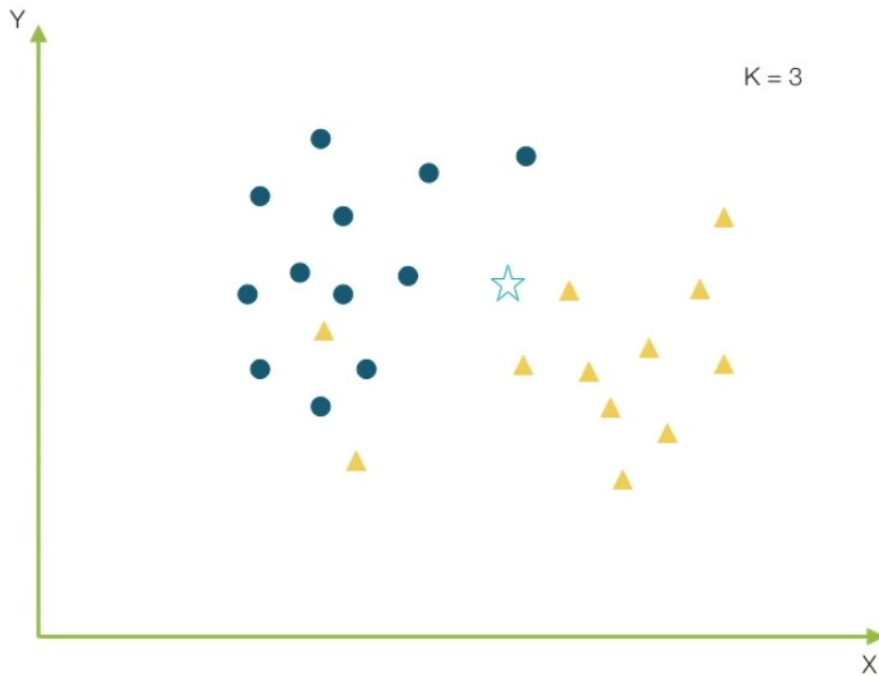
- Ejemplo:



Supongamos que tenemos un dataset con dos clases objetivo (círculo y triángulo)

K-Nearest Neighbors (KNN)

- Ejemplo:

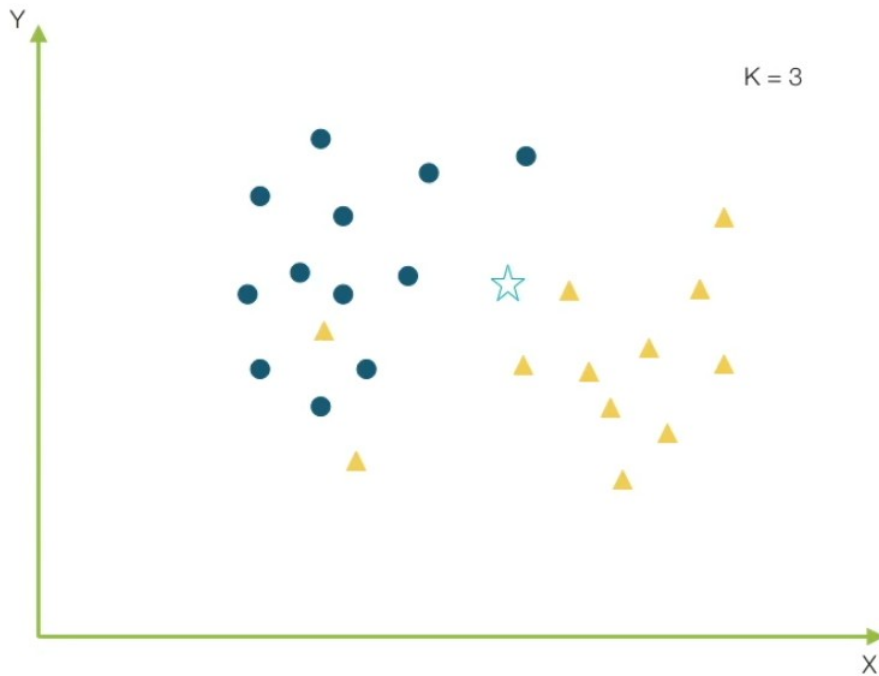


Supongamos que tenemos un dataset con dos clases objetivo (círculo y triángulo)

Además, el dataset posee dos características: X y Y, cuyos valores podrían ser discretos o continuos

K-Nearest Neighbors (KNN)

- Ejemplo:



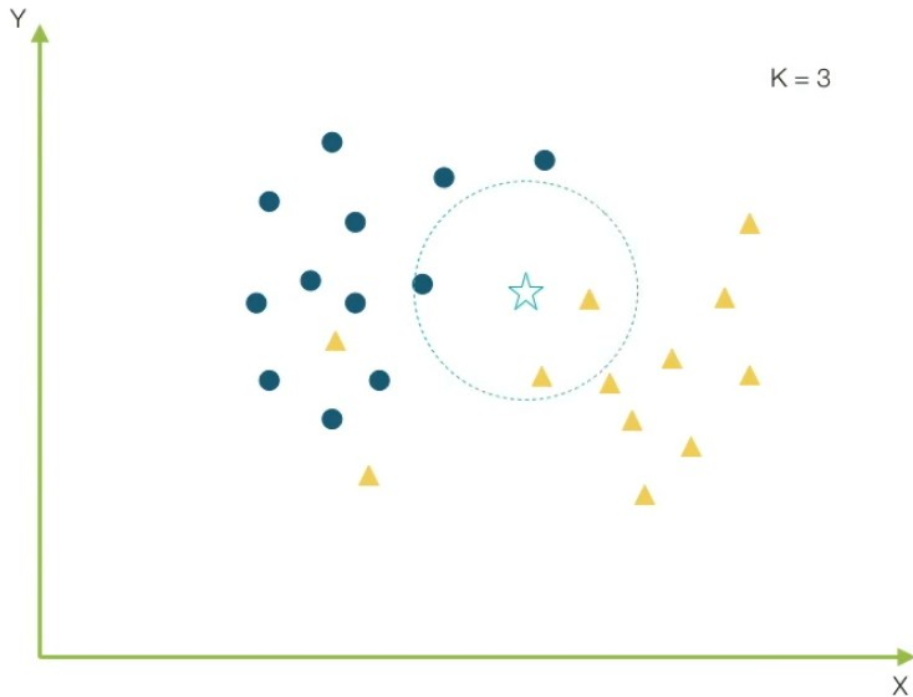
Supongamos que tenemos un dataset con dos clases objetivo (círculo y triángulo)

Además, el dataset posee dos características: X y Y, cuyos valores podrían ser discretos o continuos

Queremos determinar la clase de una muestra del conjunto de test: la estrella.

K-Nearest Neighbors (KNN)

- Ejemplo:



Supongamos que tenemos un dataset con dos clases objetivo (círculo y triángulo)

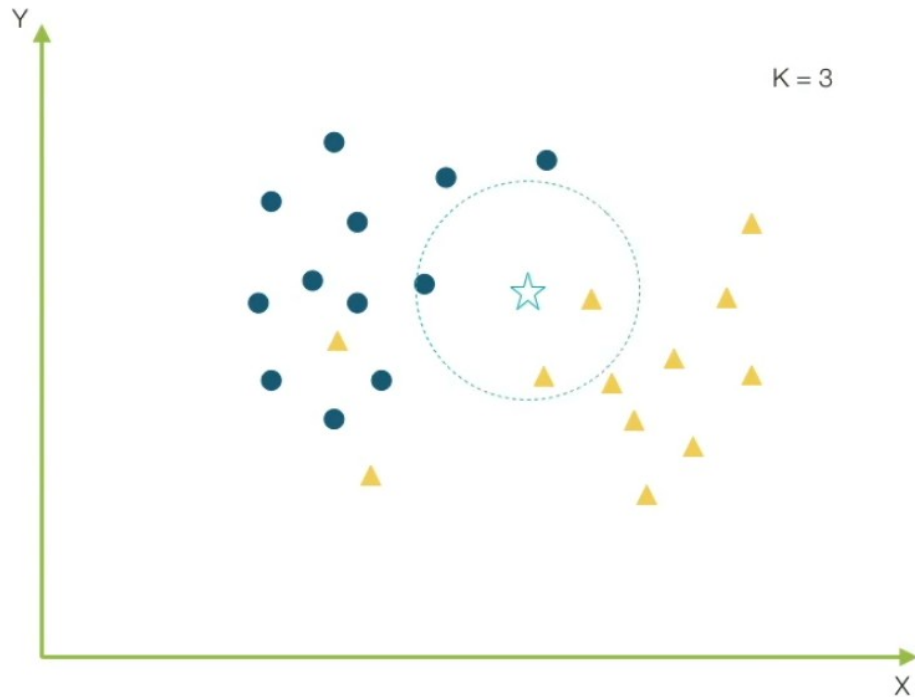
Además, el dataset posee dos características: X y Y, cuyos valores podrían ser discretos o continuos

Queremos determinar la clase de una muestra del conjunto de test: la estrella.

Si $K=3$: entonces escogemos los 3 vecinos más cercanos a la estrella y por votación mayoritaria predecimos la clase que debería tener la estrella.

K-Nearest Neighbors (KNN)

- Ejemplo:



Supongamos que tenemos un dataset con dos clases objetivo (círculo y triángulo)

Además, el dataset posee dos características: X y Y, cuyos valores podrían ser discretos o continuos

Queremos determinar la clase de una muestra del conjunto de test: la estrella.

Si $K=3$: entonces escogemos los 3 vecinos más cercanos a la estrella y por votación mayoritaria predecimos la clase que debería tener la estrella.

Para hallar la distancia podríamos usar la distancia euclidiana.

K-Nearest Neighbors (KNN)

- Tips:
 - **Importancia de k:**
 - Si k es muy pequeño (ej. $k=1$), el modelo es muy sensible al ruido y puede generar overfitting.
 - Si k es muy grande, el modelo se vuelve muy general y puede ignorar patrones locales importantes (underfitting).

K-Nearest Neighbors (KNN)

- Tips:
 - **Normalización de datos**
 - Como k-NN se basa en distancias, es fundamental escalar o normalizar las variables. Si una variable tiene un rango de 0 a 1,000 y otra de 0 a 1, la primera dominará el cálculo de la distancia, aunque la segunda sea más importante.

K-Nearest Neighbors (KNN)

- Tips:
 - **Lazy Learner**
 - A medida que el conjunto de datos crece, k-NN se vuelve más lento. Esto se debe a que, para cada predicción, debe calcular la distancia contra todos los puntos existentes.

K-Nearest Neighbors (KNN)

- Aplicaciones comunes:
 - **Sistemas de recomendación:** "A los usuarios que les gustó X, también les gustó Y".
 - **Reconocimiento de patrones:** Identificación de dígitos escritos a mano.
 - **Detección de valores atípicos (Outliers):** Si un punto está muy lejos de todos sus vecinos, puede considerarse una anomalía.

CONTINUARÁ...